408 详解 · 2009 年选择题 · 升级版

本册每题先给出原卷题干与选项全文（转录自公开渠道真题，来源与核对见《2009-升级日志》），再按 6 节讲解；讲解重心放在方法、推理、易混点上。

升级版说明：每题标题下新增"**题干与选项：**"节（原卷题干与选项全文）；6 节之后新增 7 类增补小节（【图像】【定位】【易混对比】【坑位】【套路】【类比】【口诀】，无适用内容的小节省略）；重点句按全卷统一配色：红＝关键结论、蓝＝定位与套路关键动作、橙＝坑位警告、绿＝口诀；【口诀】附考频标注（该知识点在 2009–2022 年解析中的出现次数＋高/中/低频档位，高频红标）。

第 1 题 · 缓冲区在设备管理中的先进先出作用与速度不匹配问题（正确答案：B）

题干与选项：

为解决计算机主机与打印机之间速度不匹配问题，通常设置一个打印数据缓冲区，主机将要输出的数据依次写入该缓冲区，而打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应该是。

- A. 栈
- B. 队列
- C. 树
- D. 图

入手点： 先想缓冲区的活：主机先把要打的东西交出来，打印机慢慢打，交出来的顺序不能乱。

专有名词从零解释：

- 先想“缓冲区”：一个临时放东西的地方，像食堂打饭窗口前的一段排队区，先来的先被处理。
- 再想“先进先出”：先交进来的先被拿走，后交进来的后被拿走，顺序不变。
- 再想“速度不匹配”：主机交东西快，打印机打东西慢，两边合不上拍，中间得有个地方先存着。

考查知识点：

- 先想为什么是先进先出：主机先交的第一份文件，就该先被打出来；如果顺序乱了，后交的文件先打出来，用户拿到手的顺序就全错了。
- 再想四个选项：A 是栈，B 是队列，C 是树，D 是图；栈是后进先出，树和图是用来表示层次和网状关系的，都不是“排队等着被处理”的结构。
- 拿生活例子验算：打印机前排队，先交的作业先打，后交的后打，这就是排队；要是按栈来，先交的作业会被压在最底下、最后才打出来，顺序全反了。
- 边界情况：缓冲区只解决“先存着、按顺序慢慢打”，不解决“打得快”；打印机本身的速度没变，只是主机不用在旁边干等。
- 易混对子：不要把“临时存东西”都当成栈——栈是后进先出，缓冲区要的是先进先出；不要把“存很多东西”当成树或图——树和图管的是结构关系，不是排队顺序。

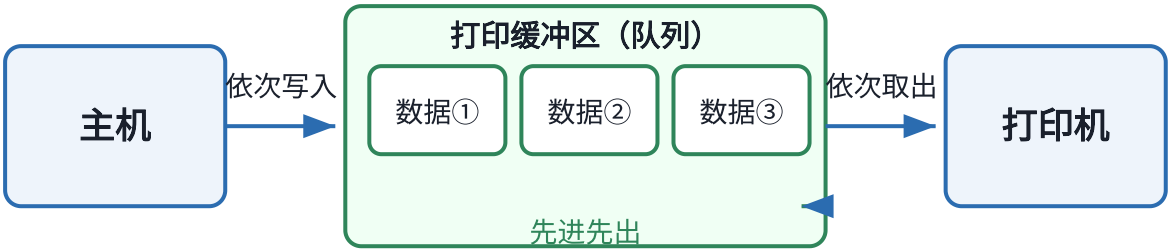
正确选项为什么对： 队列就是先进先出，先交进来的先被打印机取走，顺序不变，又让主机不用等打印机，就是 B，选 B。

错误选项为什么错＋举例：

- A（栈）：栈是后进先出，先交的文件会被压在底下、最后才打，顺序反了。教训：先想顺序要求，再挑结构。
- C（树）：树表示的是一对多的层次关系，不是排队。教训：结构要和用途对上，存东西不等于排队。
- D（图）：图表示的是多对多的网状关系，也不是排队。教训：图管的是“谁和谁连着”，不是“谁先谁后”。

笔记本一句话： 第 1 题＋缓冲区：主机快打印机慢，中间排队按顺序打，先进先出就是队列。

【图像】



先写入的数据先被取出：谁先进，谁先出

主机依次写入、打印机依次取出的打印缓冲区队列示意

【定位】 数据结构 · 第 3 章栈、队列和数组 · 缓冲区的先进先出与队列逻辑结构

【易混对比】

- 栈和队列是两回事：栈是后进先出，队列是先进先出；缓冲区要求"先写入的先被取出"，只有队列符合。
- 逻辑结构和存储方式是两回事：题问的是数据进出的先后规则（逻辑结构），不是用数组还是链表来实现。

谁先写进去谁先被取走＝先进先出＝队列。

【坑位】

看到"缓冲区"别往"栈"上靠：若用栈，先进入的数据最后才被打印，打印顺序全反了。

主机"依次写入"，打印机"依次取出"，两端顺序一致。栈的出栈顺序是入栈的逆序——打印机拿到的第一份会是主机最后写入的那一份，与题意矛盾。C 树、D 图没有线性先后顺序的概念，更不沾边。

【套路】

- 第一步，抓出题干里的顺序描述词："依次写入""依次取出"→两端顺序一致。
- 第二步，把顺序词翻译成结构性质：先进先出→队列，后进先出→栈。
- 第三步，用反例验算错项：栈会让第一个写入的最后打印，排除 A；树、图无线性顺序概念，排除 C、D。

顺序词→结构性质→反例验算错项。

【类比】

像排队取号：先到的人先被窗口叫号，后到的人在后面等着。打印机就是窗口，按排队的先后一个一个"叫号"打印，谁也不能插队。

【口诀】 依次写入依次取，先进先出队列替。

【考频】 「队列」在 2009–2022 年解析中出现 48 次 · 高频

第 2 题 · 用栈实现队列输出时栈的最小容量（正确答案：C）

题干与选项：

设栈 S 和队列 Q 的初始状态均为空，元素 a,b,c,d,e,f,g 依次进入栈 S。若每个元素出栈后立即进入队列 Q，且 7 个元素出队的顺序是 b,d,c,f,e,a,g，则栈 S 的容量至少是。

- A. 1
- B. 2
- C. 3
- D. 4

入手点： 把入栈出栈的过程一步一步走一遍，数一数栈里最多同时有几个元素，那个最大值就是最少容量。

专有名词从零解释：

- 先想“队列”：先进先出，像排队，先排的先走。
- 再想“栈”：后进先出，像一摞盘子，后放的先拿。
- 再想“容量”：栈最多能装几个元素；模拟过程中栈里同时出现的元素个数，不能超过这个数。

考查知识点：

- 先想关键关系：队列的出队顺序，就是栈的出栈顺序；要模拟出这个顺序，入栈出栈的次序是定死的。
- 再想模拟过程：a 入、b 入、b 出、c 入、d 入、d 出、c 出、e 入、f 入、f 出、e 出、a 出、g 入、g 出。
- 再想栈里元素个数的变化：a 入是 1 个，b 入是 2 个，b 出回落到 1 个，c 入是 2 个，d 入是 3 个，d 出回落到 2 个，c 出回落到 1 个，e 入是 2 个，f 入是 3 个，f 出回落到 2 个，e 出回落到 1 个，a 出回落到 0 个，g 入是 1 个，g 出回落到 0 个。
- 拿生活例子验算：一摞盘子最多摞到 3 个高，盘子架最少也得能放 3 个；架子只能放 2 个的话，摞到第 3 个就没地方放了。
- 边界情况：容量是“最多同时出现”的个数，不是“总共进栈”的个数；总共进栈 7 个元素，但同一时刻最多只有 3 个在栈里。
- 易混对子：不要把“总共几个元素”当成容量——容量看的是同一时刻的峰值；不要以为栈要装下全部元素——元素是边进边出的。

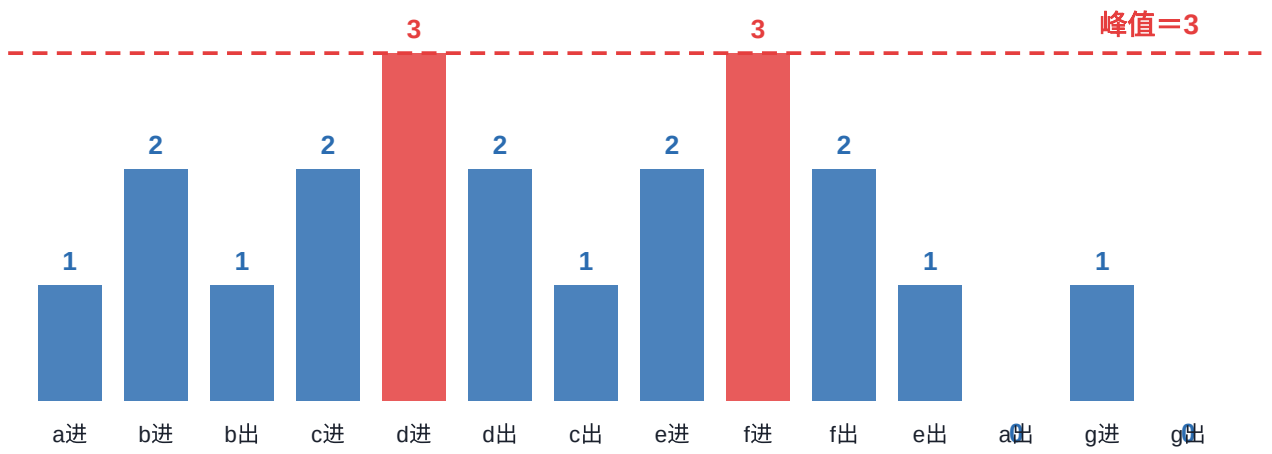
正确选项为什么对： 模拟下来栈里最多同时有 3 个元素，所以容量最少是 3，就是 C，选 C。

错误选项为什么错＋举例：

- 比 3 小的容量：摞到 3 个的时候就装不下了，模拟走不下去。教训：容量看峰值，不是看平均。
- 比 3 大的容量：装是装得下，但题目问的是最少，多了就不符合“最少”。教训：先算峰值，再对“最少”。

笔记本一句话： 第 2 题＋栈模拟队列：一步一步走，栈里最多同时 3 个，容量最少就是 3。

【图像】



栈内深度随模拟过程变化：容量至少为 3（选 C）

模拟出队序列 b,d,c,f,e,a,g 时栈内深度变化，峰值为 3

【定位】 数据结构 · 第 3 章栈、队列和数组 · 栈模拟队列与栈容量下限

【易混对比】

- 栈的容量和"进过栈的元素总数"不是一回事：容量是同一时刻栈里同时存在的最大元素个数，不是总共 7 个元素。
- 出队顺序和进栈顺序不是一回事：进栈顺序固定是 a～g，出队顺序由每个元素何时出栈决定。

栈 S 的出栈顺序就是队列 Q 的出队顺序；容量看栈内同时存在的峰值。

【坑位】

别数元素总数：7 个元素进过栈不代表容量是 7，容量只看同一时刻栈里最多挤了几个。

逐个模拟：a 进、b 进（栈内 a,b）、b 出→Q 得 b；c 进、d 进（栈内 a,c,d，深度 3）、d 出→Q 得 d、c 出→Q 得 c；e 进、f 进（栈内 a,e,f，深度 3）、f 出、e 出；a 出→Q 得 a；g 进、g 出。栈内深度峰值是 3（a,c,d 与 a,e,f 两个时刻），所以容量至少为 3。选 A(1)/B(2) 的，是把"同一时刻"想成了"前后累计"。

【套路】

- 第一步，按出队顺序在脑内放一个栈：要出队 b，必须先让 a、b 进栈，再出 b（后进先出）。
- 第二步，边模拟边数栈内实时个数：进一个 +1，出一个 −1，记下最大值。
- 第三步，最大值就是容量下限：峰值 3→容量至少 3，选 C。

放一个栈模拟出队过程，实时计数取峰值。

【类比】

像电梯限载：7 个人先后进出过电梯，不代表电梯要装 7 个人；容量看的是同一时刻电梯里最多挤了几个人，峰值 3 人，电梯限载就是 3。

【口诀】 模拟进出数峰值，容量就看最大值。

【考频】 「栈」在 2009–2022 年解析中出现 48 次 · 高频

第 3 题 · 由遍历序列判断非基本遍历方式（正确答案：D）

题干与选项：

给定二叉树如右图所示。设 N 代表二叉树的根，L 代表根结点的左子树，R 代表根结点的右子树。若遍历后的结点序列是 3，1，7，5，6，2，4，则其遍历方式是。

- A. LRN
- B. NRL
- C. RLN
- D. RNL

> 注：插图以原卷为准

入手点： 先看根结点在序列里的位置，再看左右子树的先后，两个信息一拼，遍历方式就定了。

专有名词从零解释：

- 先想“遍历”：把树里的每个结点按某种规矩走一遍，不重不漏。
- 再想三种基本遍历：前序是根在最前（根、左、右），中序是根在中间（左、根、右），后序是根在最后（左、右、根）。
- 再想题目里的记法：R 表示根，L 表示左子树，N 表示右子树——这里 N 不是“中”，是“右”的代号；RNL 就是先根、再右子树、再左子树。

考查知识点：

- 先想根的位置：序列里根结点在中间被访问，前序（根最前）和后序（根最后）先排除。
- 再想左右顺序：右子树结点在左子树之前被访问，中序是左先于右，也排除了。
- 再想剩下的组合：根在中间、右先于左，就是 RNL；它不在前序、中序、后序三种基本遍历里，所以答案是“不是基本遍历方式”的那一项。
- 拿生活例子验算：一家人合影站位，先站中间那位（根），再站右边那排（右子树），最后站左边那排（左子树）——这个站法不在“先中间再两边按左右”那三种常规站法里。
- 边界情况：判断遍历方式只看两点——根在前、中、后哪个位置，左右谁先谁后；两个都对上才能定下来。
- 易混对子：不要一看到“根在中间”就说是中序——中序还要求左先于右，这里右先于左，就不是中序；不要把 RNL 的 N 当成“中”——这里 N 指右子树。

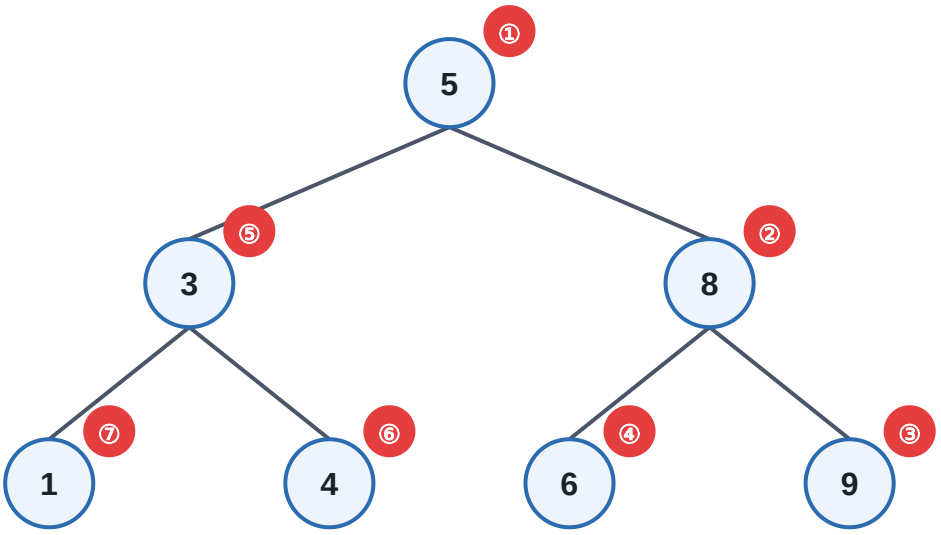
正确选项为什么对： 根在中间、右先于左，拼出来是 RNL，不在三种基本遍历里，就是 D，选 D。

错误选项为什么错＋举例：

- 说前序的选项：前序根在最前，题里根在中间，对不上。教训：先看根的位置。
- 说中序的选项：中序根在中间没错，但要求左先于右，题里是右先于左，对不上。教训：根的位置对了，还要看左右顺序。
- 说后序的选项：后序根在最后，题里根在中间，对不上。教训：两个条件都要验。

笔记本一句话： 第 3 题＋遍历方式：根在中间、右先于左，拼出来是 RNL，不是基本遍历。

【图像】



RNL＝先访问根，再右子树，最后左子树

（示例树，非原图）

示例二叉树的 RNL 遍历示意：先访问根，再右子树，最后左子树（示例树，非原图）

【定位】 数据结构 · 第 5 章树与二叉树 · 非常规遍历方式的识别

【易混对比】

- 根在中间访问和"中序遍历"是两回事：三种基本遍历只有 NLR、LNR、LRN；RNL 的根位置虽和中序一样在中间，但左右访问顺序反了，不是基本遍历。
- NRL、RNL、RLN 和基本遍历是两回事：它们只是把基本遍历的左右互换，统称非常规遍历方式。

根在中间访问，且右子树结点先于左子树→RNL；它不是三种基本遍历方式之一。

【坑位】

看到"根在中间"就选"中序（LNR）"是错的：中序要求先左后右，题中右子树结点先于左子树出现，只能是 RNL。

序列是 3，1，7，5，6，2，4：根 N 在第二位被访问（根在中间），其后访问的结点中右子树的排在左子树之前。A 的 LRN 是后序（根最后），B 的 NRL 根最先，C 的 RLN 根最后，都不符合"根在中间"。RNL 里"根在中间"恰好与中序同位，只是左右访问顺序反了——这正是 D 为"非基本遍历"的落点。

【套路】

- 第一步，看根在序列中的位置：最前→先序类（NLR/NRL），中间→中序类（LNR/RNL），最后→后序类（LRN/RLN）。
- 第二步，看左右谁先被访问：左先→L 开头的，右先→R 开头的。
- 第三步，对照三种基本遍历：只有 NLR、LNR、LRN 是基本的，NRL、RNL、RLN 都是非常规。

根定前后（先序/中序/后序类），左右定谁先，基本只有 NLR、LNR、LRN。

【类比】

像报菜名：先报主菜再报凉菜热菜（NLR），主菜夹在中间报（LNR/RNL），最后报主菜（LRN/RLN）。RNL 就是"主菜中间报，但先报热菜再报凉菜"——标准菜单上没有这个套餐，只有自定义的。

【口诀】 根定类来左右定先，基本三式要记全。

【考频】 「二叉树遍历」在 2009–2022 年解析中出现 54 次 · 高频

第 4 题 · 用左右子树高度差判据识别平衡二叉树（正确答案：B）（部分核对）

题干与选项：

下列二叉排序树中，满足平衡二叉树定义的是_____。

- A.（图，见原卷）
- B.（图，见原卷）
- C.（图，见原卷）
- D.（图，见原卷）

> 注：插图以原卷为准

入手点： 不用整棵树一眼看，先给每个非叶子结点算平衡因子：左子树高度减右子树高度，绝对值超过 1 的，这棵树就不平衡。

专有名词从零解释：

- 先想“高度”：从这个结点往下，到最远的叶子要走几层；叶子自己的高度是 1，只有根一棵光杆树高度也是 1。
- 再想“平衡因子”：左子树高度减去右子树高度，结果可正可负；平衡二叉树要求每个结点的平衡因子绝对值都不超过 1。
- 再想“平衡二叉树”：整棵树左右长得比较均匀，没有哪一边特别深；判断时每个结点都要过关，有一个不过关，整棵树就不平衡。

考查知识点：

- 先想判据：任一结点左右子树高度差的绝对值不超过 1；不太明显的时候，逐个算出非叶结点的平衡因子，一个一个验。
- 再想四个选项的树形（据原卷补抄的文字概括，D 图最下结点连线未看清，本节作部分核对）：A 是 3 个结点、向一侧延伸的单链式树，无分支；B 是 5 个结点，根左右各有一子，左右子又各向外再延伸一层，左右大体对称、等高；C 是 4 个结点、单条路径、无分支的链式树，先向左下、再向右下延伸；D 是 6 个结点，左侧分支多层延伸，右侧只有一层，左侧明显深于右侧。
- 再想逐棵验算：A 向一侧单链，根的一边高度是 2、另一边是 0，高度差是 2，超过 1，不平衡；C 单条路径从左下拐到右下，同样一边深、一边浅，高度差超过 1，不平衡；D 左侧多层、右侧只有一层，左右高度差超过 1，不平衡；B 左右大体对称、等高，每个结点的左右高度差都不超过 1，平衡。
- 拿生活例子验算：天平两边盘子挂的东西差不多高才平衡；A、C、D 都是一边挂得老长、一边空着，天平歪了；只有 B 两边差不多高。
- 边界情况：高度差等于 1 是允许的，只有超过 1 才不平衡；判断要每个结点都验，不能只看根结点。
- 易混对子：不要把“结点多”当成平衡——D 有 6 个结点，比 B 还多，但一边倒，照样不平衡；不要只看根结点——有的树根结点看着平衡，下面的结点已经歪了。

正确选项为什么对： B 左右对称、等高，每个结点的左右高度差都不超过 1，符合平衡二叉树的判据，就是 B，选 B。

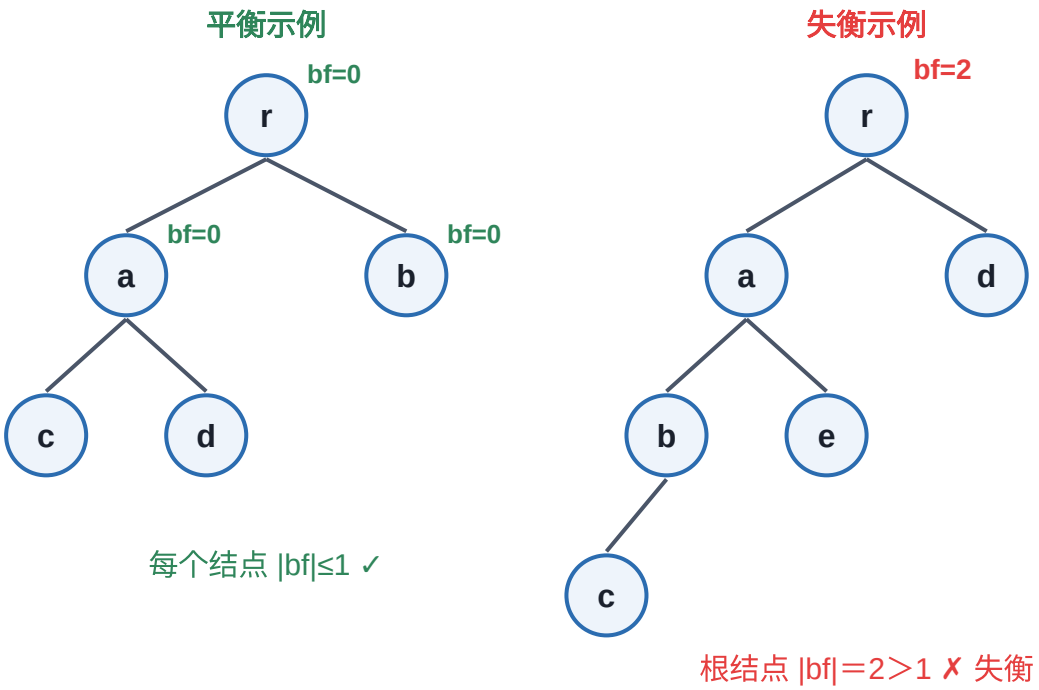
错误选项为什么错＋举例：

- A（3 结点单链）：向一侧单链，根的一边高 2、一边高 0，差 2，超了。教训：单链树一边倒，肯定不平衡。
- C（4 结点单链）：单条路径先左下再右下，还是一边倒，高度差超 1。教训：拐个弯还是单链，照样不平衡。

- D（6 结点左深右浅）：左侧多层、右侧一层，左右高度差超 1。教训：结点多不等于平衡，看的是左右高度差。

笔记本一句话： 第 4 题＋平衡二叉树：每个结点左右高度差绝对值都不超过 1，B 左右对称等高，A、C、D 一边倒。

【图像】



逐个结点算平衡因子（左高−右高），|bf|＞1 即失衡（示意，非原卷四图）

平衡判定示意：逐个结点计算平衡因子，绝对值超过 1 即失衡（示意，非原卷四图）

【定位】 数据结构 · 第 7 章查找 · 平衡二叉树的定义判据（图题，部分核对）

【易混对比】

- 平衡二叉树和二叉排序树是两回事：排序树只要求左小右大，平衡树还要每个结点的左右高度差不超过 1；本题四个图都是排序树，排序规则筛不掉任何选项。
- 看起来对称和满足判据是两回事：判据是每个非叶结点的平衡因子绝对值都不超过 1，要逐个算，不是看整体顺不顺眼。

判据只有一个：任一结点左右子树高度差的绝对值不超过 1，逐个结点验算平衡因子。

【坑位】

别只看"左右高度大体一致"就定答案：D 图左侧明显深于右侧（差超过 1），A、C 是无分支的单链式，根的平衡因子绝对值超过 1，只有 B 每个结点的平衡因子都不超标。

按原卷微型补抄的关键特征：A 是 3 个结点、向一侧延伸的单链式树，无分支；C 是 4 个结点、无分支的链式树（先向左下、再向右下延伸），两者的根结点左右高度差都超过 1；D 是 6 个结点，左侧分支多层延伸、右侧只有一层，左侧明显深于右侧，差超过 1；B 是 5 个结点，根左右各有一子，左右子又各向外再延伸一层，左右大体对称、等高，每个结点的平衡因子绝对值都不超过 1。不太明显时，就把每个非叶结点的平衡因子逐个算出来。

【套路】

- 第一步，确认都是排序树后不再纠结大小关系，只看高度。
- 第二步，从下往上算每个非叶结点的平衡因子（左高减右高）。
- 第三步，出现绝对值超过 1 的结点就淘汰该图；四个图只剩 B。

逐个算非叶结点平衡因子，有一个超 1 就淘汰。

【类比】

像检查一组天平：每根横梁两边挂的高度差不能超过一格，有一根超了整组就不合格。不能只看最顶上那根顺不顺眼，每根横梁都要量一遍。

【口诀】 逐个结点算因子，差超一就出局。

【考频】 「平衡二叉树」在 2009–2022 年解析中出现 8 次 · 中频

第 5 题 · 由第 6 层叶结点数求完全二叉树的最大结点数（正确答案：C）

题干与选项：

已知一棵完全二叉树的第 6 层（设根为第 1 层）有 8 个叶结点，则该完全二叉树的结点个数最多是_____。

- A. 39
- B. 52
- C. 111
- D. 119

入手点： 先想完全二叉树长什么样，再想“最多”对应哪种高度：高度越大，结点越多。

专有名词从零解释：

- 先想“完全二叉树”：除了最底层，其他层都是满的；最底层可以缺，但缺只能缺在右边，左边的结点都得在。
- 再想“叶结点”：下面没有孩子的结点，像树梢。
- 再想“高度”：从根到最远叶子的层数；高度为 6 的满二叉树一共 63 个结点，高度为 7 的满二叉树一共 127 个结点。

考查知识点：

- 先想第 6 层有叶结点时，高度可能是 6，也可能是 7：如果高度是 6，第 6 层就是最底层；如果高度是 7，第 6 层的结点还可以有孩子。
- 再想为什么取 7：高度 7 的树比高度 6 的树多一整层，结点更多；题目问“最多”，就取高度 7。
- 再想高度 7 时的算法：前 6 层是满的，一共 63 个结点；第 6 层有 32 个结点，其中 8 个是叶结点（没有孩子），剩下 24 个各有两个孩子，第 7 层就有 48 个结点；总数是 63＋48＝111。
- 再想另一种算法：高度 7 的满二叉树是 127 个结点；第 6 层的 8 个叶结点，每个少了 2 个孩子，第 7 层缺 8×2＝16 个结点；127−16＝111。
- 拿生活例子验算：六层楼住满了人，七楼只住了 48 间；六楼有 8 间是封顶的（不住人也不往上盖），七楼就少了 16 间；127 间满的减掉 16 间，就是 111 间。
- 边界情况：只有最后两层有叶结点；第 6 层的叶结点数不能超过 32（第 6 层一共就 32 个位置）。
- 易混对子：不要直接拿满二叉树的 127 当答案——第 6 层那 8 个叶结点没有孩子，第 7 层缺了 16 个；不要取高度 6——高度 6 只有 63 个结点，不是最多。

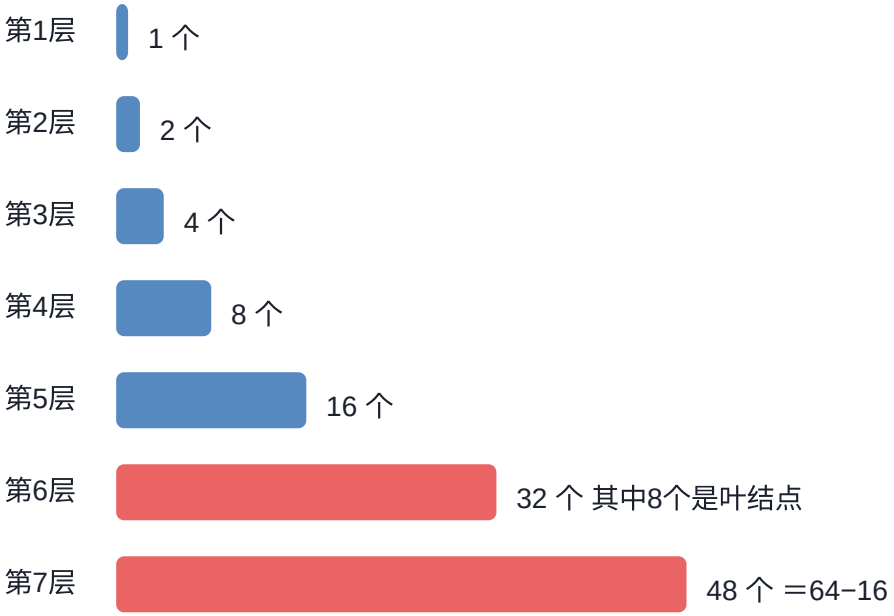
正确选项为什么对： 高度取 7，前 6 层满 63 个，第 7 层 48 个，一共 111，就是 C，选 C。

错误选项为什么错＋举例：

- 说 63 的选项：那是高度 6 的满二叉树，没算第 7 层。教训：问“最多”要取最大的高度。
- 说 127 的选项：那是高度 7 的满二叉树，没扣掉第 7 层缺的 16 个。教训：叶结点没有孩子，缺的那部分要减掉。
- 其他数值的选项：要么高度取错了，要么缺的结点数算错了。教训：先定高度，再算缺口。

笔记本一句话： 第 5 题＋完全二叉树最多结点：高度取 7，满树 127 减掉第 7 层缺的 16 个，得 111。

【图像】



前6层满＝63，第7层＝48，总数＝63＋48＝111（选 C）

完全二叉树层数与结点数示意：第 6 层有 8 个叶结点时高度取 7 结点数最多

【定位】 数据结构 · 第 5 章树与二叉树 · 完全二叉树的层数与结点数极值

【易混对比】

- 第 6 层有叶结点和高度是 6 是两回事：叶结点在第 6 层，树还可以有第 7 层（第 6 层的非叶结点继续往下长）。
- 求最多和求最少是两回事：高度取 7 且第 7 层尽量塞满才是"最多"；只算高度 6 的是算少了。

第 6 层有叶结点→高度可为 6 或 7；求最多取高度 7，且第 6 层的 8 个叶结点让第 7 层少长 16 个结点。

【坑位】

"最多"二字决定高度取 7：高度 6 时最多 63 个结点（满二叉树），高度 7 时把第 7 层尽量塞满才是 111；按高度 6 的非满情形算是根本没往"最多"上想。

完全二叉树只在最底层右侧缺结点，最底层之上全是满的，且叶结点只出现在最后两层。第 6 层有 8 个叶结点→前 6 层为满二叉树（63 个结点）；这 8 个叶结点在第 7 层本来能长出 8×2＝16 个孩子，因为它们是叶结点，第 7 层就少了这 16 个：第 7 层最多有 64−16＝48 个结点。总数＝63＋48＝111。验算：(2^7−1)−16＝127−16＝111。

【套路】

- 第一步，由"第 k 层有叶结点"定高度范围：高度可为 k 或 k＋1。
- 第二步，"最多"取大高度，前 k 层按满二叉树算（2^k−1 个结点）。
- 第三步，第 k 层的叶结点数 m→第 k＋1 层少 m×2 个结点，加上去得总数。

高度取大，前 k 层满算，叶结点 m 让下一层少 2m。

【类比】

像盖楼：前 6 层是标准层全盖满了，第 6 层有 8 个房间封顶不再往上盖（叶结点），第 7 层就比满层少了 16 个房间。要"住的人最多"，楼就盖 7 层高，第 7 层能盖多少盖多少。

【口诀】 有叶取大高，前满减二倍。

【考频】 「完全二叉树」在 2009–2022 年解析中出现 42 次 · 高频

第 6 题 · 森林转二叉树后父子关系在原森林中的可能含义（正确答案：B）

题干与选项：

将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 的父结点的父结点，则在原来的森林中，u和 v 可能具有的关系是_____。

Ⅰ．父子关系 Ⅱ．兄弟关系Ⅲ．u 的父结点与 v 的父结点是兄弟关系

- A. 只有 Ⅱ
- B. Ⅰ和 Ⅱ
- C. Ⅰ和Ⅲ
- D. Ⅰ、Ⅱ和Ⅲ

入手点： 先把转换规则想清楚——左孩子右兄弟，再看二叉树里的父子在原森林里能对应上哪几种关系。

专有名词从零解释：

- 先想“森林”：好几棵树放在一起，每棵树有自己的根。
- 再想“左孩子右兄弟”规则：森林转二叉树时，每个结点的第一个孩子变成它的左孩子，紧挨着的下一个兄弟变成它的右孩子；兄弟之间用右孩子一串串连起来。
- 再想题目里的三句话：Ⅰ是父子关系，Ⅱ是兄弟关系，Ⅲ是 u、v 各自的父结点互为兄弟。

考查知识点：

- 先想规则怎么走：一个结点的第一个孩子，在二叉树里是它的左孩子；第二个孩子、第三个孩子，不直接挂在父亲下面，而是挂在前一个孩子的右孩子位置上，一串连着。
- 再想第一种可能（Ⅰ，父子关系）：如果 v 是 u 的第一个孩子，转成二叉树后 v 就是 u 的左孩子，二叉树里的父子正好就是原森林里的父子。
- 再想第二种可能（Ⅱ，兄弟关系）：如果 v 是 u 的第二个孩子，转成二叉树后 v 变成 u 第一个孩子的右孩子；这时候二叉树里的一对父子（第一个孩子和 v），在原森林里是兄弟；再比如 u 和 v 是兄弟、中间还隔着兄弟 k，v 变成 k 的右孩子、k 变成 u 的右孩子，二叉树里的父子在原森林里也是兄弟。
- 再想第三种为什么不行（Ⅲ）：如果 u 的父结点和 v 的父结点互为兄弟，转成二叉树后 u、v 不在同一条路径上，做不成二叉树里的一对父子，所以 Ⅲ 不是可能的含义。
- 拿生活例子验算：一家人排队，第一个孩子站父亲左手边，其他孩子一个接一个站哥哥右手边；二叉树里“左手边那位”和“右手边那位”站在一起，在原家里可能是父子，也可能是兄弟，但不可能是“两家人的孩子”。
- 边界情况：右孩子链表示的是兄弟关系，不是父子关系；看到二叉树里的右孩子，先想“兄弟”，再想“父子”。
- 易混对子：不要以为二叉树里的父子一定是原森林里的父子——右孩子那条线表示的是兄弟；不要把“各自的父结点是兄弟”也算进来——那种情形下两个结点在二叉树里根本不在一条路径上。

正确选项为什么对： 二叉树里的父子，在原森林里可能是父子（第一个孩子），也可能是兄弟（右孩子链），Ⅰ、Ⅱ都对，就是 B，选 B。

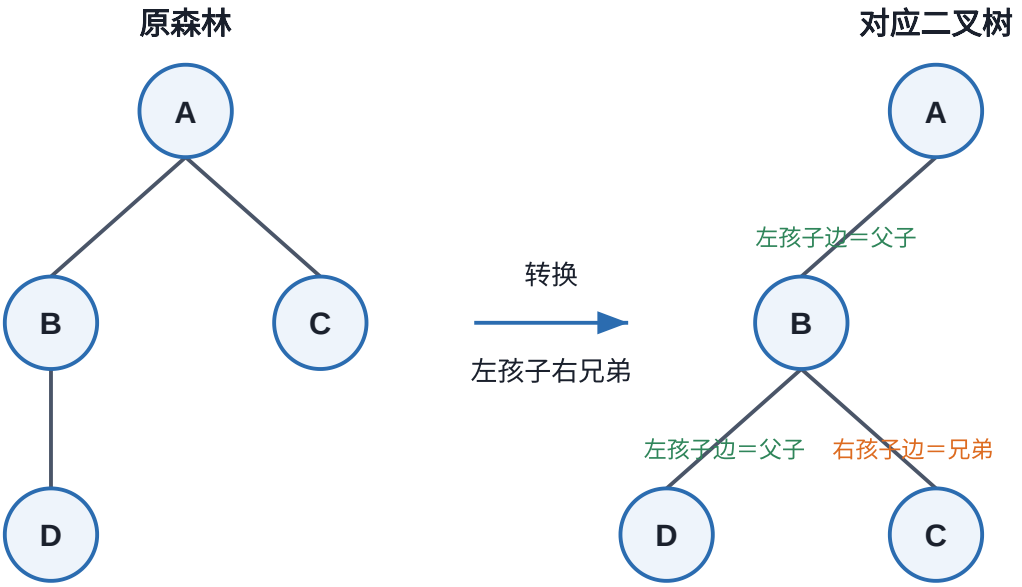
错误选项为什么错＋举例：

- A（仅Ⅱ）：漏了 Ⅰ——第一个孩子转成左孩子，那就是原来的父子关系。教训：左孩子那条线就是父子，别漏了。

- C（Ⅰ和Ⅲ）：Ⅲ不成立——各自的父结点是兄弟时，两个结点在二叉树里不在一条路径上。教训：不在一条路径上，就做不成父子。
- D（Ⅰ、Ⅱ和Ⅲ）：多了不成立的Ⅲ。教训：组合题要一项一项验，多一项错的，整个组合就错。

笔记本一句话： 第 6 题＋森林转二叉树：左孩子是父子，右孩子链是兄弟，B 是 Ⅰ、Ⅱ。

【图像】



u 是 v 的祖父：Ⅰ父子 ✓（经左孩子边） Ⅱ兄弟 ✓（经右孩子边）
Ⅲ（父结点互为兄弟）不在同一路径上 ✗ → 选 B

森林转二叉树"左孩子右兄弟"规则示意：左孩子边对应父子，右孩子边对应兄弟

【定位】 数据结构 · 第 5 章树与二叉树 · 森林与二叉树的转换规则

【易混对比】

- 二叉树中的父子边和森林中的父子关系是两回事：二叉树的父子边可能来自森林的兄弟关系（右孩子边），也可能来自真正的父子关系（左孩子边）。
- 左孩子边和右孩子边是两回事：左孩子边＝原森林的父子关系，右孩子边＝原森林的兄弟关系。

左孩子右兄弟：二叉树中的左孩子边对应森林的父子，右孩子边对应森林的兄弟。

【坑位】

u 是 v 的"父结点的父结点"（祖父），不是"父结点"：v 的父结点可能是 u 的左孩子（→v 是 u 的孙辈，Ⅰ对），也可能是 u 的右孩子（→v 与 u 在兄弟链上，Ⅱ对）；但 u 的父结点与 v 的父结点互为兄弟（Ⅲ）时，u、v 根本不在同一条路径上，不可能一个是另一个的祖父——这是 C、D 的陷阱。

情形一：v 是 u 的第二个孩子→v 变成 u 第一个孩子的右孩子，u 是 v 的祖父，u、v 是父子关系，Ⅰ成立。情形二：u 与 v 为兄弟且中间隔着兄弟 k→v 变成 k 的右孩子、k 是 u 的右孩子，u 是 v 的祖父，u、v 是兄弟关系，Ⅱ成立。情形三：u 的父结点与 v 的父结点为兄弟→转换后 u、v 不在同一条路径上，Ⅲ不成立。所以只有Ⅰ、Ⅱ正确，选 B。

【套路】

- 第一步，画出祖父关系在二叉树中的两种走法：u→左孩子→左/右孩子，或 u→右孩子→左/右孩子。
- 第二步，把每条边按"左孩子右兄弟"翻译回森林：左孩子边＝父子，右孩子边＝兄弟。
- 第三步，逐项核对 Ⅰ、Ⅱ、Ⅲ：Ⅰ（父子）成立、Ⅱ（兄弟）成立、Ⅲ（父结点互为兄弟）不成立。

祖父两条走法，边边翻译，逐项核对。

【类比】

像家谱改画法：规定"长子画在爸爸正下方（左孩子），其他儿子依次画在长子右边（右孩子）"。新图谱里"某人正下方再往下"是亲儿子，"某人右边再往右"可能是亲兄弟。Ⅲ那种情形，相当于两个人分属两个分支，怎么数都数不成祖孙。

【口诀】 左孩父子右兄弟，祖父两步来翻译。

【考频】 「森林」在 2009–2022 年解析中出现 44 次 · 高频

第 7 题 · 图的顶点度数之和、连通图边数与完全图顶点度数的判断（正确答案：A）

题干与选项：

下列关于无向连通图特性的叙述中，正确的是_____。

I. 所有顶点的度之和为偶数II. 边数大于顶点个数减 1III. 至少有一个顶点的度为 1

- A. 只有 I
- B. 只有 II
- C. I 和 II
- D. I 和III

入手点： 三句话一项一项验：度数之和看每条边的贡献，连通图的边数想树的情况，完全图想每个顶点的度数。

专有名词从零解释：

- 先想“度数”：一个顶点连着几条边，度数就是几。
- 再想“完全图”：每两个顶点之间都直接连一条边的图；N 个顶点的完全图里，每个顶点的度数都是 N−1。
- 再想“连通图”：任意两个顶点之间都有路能走通的图；树就是最省的连通图。

考查知识点：

- 先想 I：每条边连接两个顶点，给度数之和贡献 2；所有边加起来，度数之和就是边数的两倍，一定是偶数，I 对。
- 再想 II：n 个顶点、n−1 条边就可以构成连通图（树就是例子），所以“边数大于顶点数减 1”不是普遍成立的，II 错。
- 再想 III：完全图里每个顶点的度数都是 N−1；只要 N 不小于 3，每个顶点度数都大于 1；就算只看一个环，每个顶点度数都是 2，也没有度数为 1 的顶点；所以“至少有一个顶点的度数为 1”不对，III 错。
- 拿生活例子验算：7 个人互相握手，每握一次手，两个人的握手次数各加 1，总次数一定是双数；7 个人最少握 6 次手就能连成一串（树），“多于 6 次”不是必须的；7 个人互相都握过手，每个人都握了 6 次，没人只握 1 次。
- 边界情况：N＝1 的完全图只有一个顶点、度数为 0，也没有度数为 1 的顶点；N＝2 的完全图两个顶点度数都是 1，那是特例，不是“至少有一个”的普遍结论。
- 易混对子：不要把“树有 n−1 条边”当成“边数大于 n−1”——树恰好是 n−1，不大于；不要以为完全图里一定有度数为 1 的顶点——完全图里每个顶点都和所有其他顶点相连，度数是 N−1。

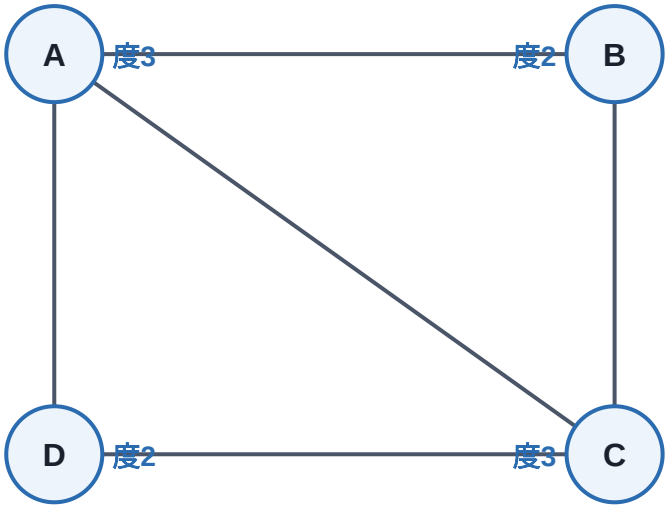
正确选项为什么对： 只有 I 对，II、III 都错，仅 I 正确就是 A，选 A。

错误选项为什么错＋举例：

- B（仅 II）：II 错——树用 n−1 条边就连通了，边数不必大于 n−1。教训：想一个反例就能推翻一句普遍断言。
- C（I 和 II）：I 对，但 II 错，组合里混进一句错的，整个组合就错。教训：组合题一项一项验。
- D（I 和III）：I 对，但 III 错——完全图和环里都没有度数为 1 的顶点。教训：找反例，环就是现成的。

笔记本一句话： 第 7 题＋图的三句话：度数之和是边数两倍一定是偶数，树 n−1 条边就连通，完全图没有度数为 1 的顶点，仅 I 对。

【图像】



度数之和＝3＋2＋3＋2＝8＝2×4条边（握手定理）

恒为偶数 → Ⅰ正确

无向图握手定理示意：每条边给两个端点的度数各贡献 1

【定位】 数据结构 · 第 6 章图 · 无向图的度数性质与连通图边数下界

【易混对比】

- 度数之和和边数是两回事：每条边贡献 2 个度，度数之和＝2×边数，恒为偶数。
- 连通和"边数大于顶点数减 1"是两回事：n 个顶点的连通图最少 n−1 条边（树），"大于 n−1"不成立。

握手定理：所有顶点度数之和＝2×边数，恒为偶数；连通图边数至少为 n−1。

【坑位】

Ⅱ的"大于 n−1"是错的：n 个顶点、n−1 条边的树就是连通图，"大于"不成立，应该是"大于等于"；Ⅲ更错：星形树才有度为 1 的顶点，但环等连通图里一个度为 1 的顶点都没有，"至少有一个"是全称命题，一个反例就推翻。

Ⅰ：每条边连接两个顶点，给两端各贡献 1 度→度数之和＝2e，偶数，成立。Ⅱ：n 个顶点 n−1 条边可构成树，树连通→"边数大于 n−1"不成立。Ⅲ：取 n≥3 的环图，连通且每个顶点度数为 2，没有度为 1 的顶点→"至少有一个度为 1"是错的。只有Ⅰ正确，选 A。

【套路】

- 第一步，把Ⅰ翻译成握手定理：度数之和＝2×边数，偶数恒成立。
- 第二步，对Ⅱ、Ⅲ各找一个反例：Ⅱ用"树（n−1 条边连通）"反驳"大于"；Ⅲ用"环图无度 1 顶点"反驳"至少"。
- 第三步，组合只有Ⅰ的选项→A。

Ⅰ握手定理直接判，Ⅱ Ⅲ各找一个反例。

【类比】

像数握手：每两个人握一次手，两个人各记一次"握手数"，总人次永远是偶数。连通像"所有人通过握手链能联系上"，最少 n−1 次握手就够（排成一串）。

【口诀】 度数求和两倍边，连通至少 n 减一。

【考频】 「度数」在 2009–2022 年解析中出现 11 次 · 高频

第 8 题 · 区分 B-树特征与 B+树特征（正确答案：D）

题干与选项：

下列叙述中，不．符合 m 阶 B 树定义要求的是_____。

- A. 根节点最多有 m 棵子树
- B. 所有叶结点都在同一层上
- C. 各结点内关键字均升序或降序排列
- D. 叶结点之间通过指针链接

入手点： 先把 B-树和 B+树的区别想清楚，再看哪个选项说的是 B+树那边独有的特征。

专有名词从零解释：

- 先想“B-树”：一种多路平衡查找树，关键字分布在所有结点里，查一个关键字可能在中间某层就找到了。
- 再想“B+树”：B-树的变种，非叶结点只起索引作用，不真正存数据；所有关键字都在叶结点里，叶结点之间用指针连成一条链表。
- 再想“叶结点链表”：所有叶结点像糖葫芦一样串起来，顺着指针能从头走到尾，方便按顺序扫一遍。

考查知识点：

- 先想 B-树这边：关键字可以出现在任何一层，非叶结点里存的是真正的关键字和数据，叶结点之间没有指针相连。
- 再想 B+树这边：非叶结点只放索引（用来指路），真正的数据都在叶结点；各叶结点之间通过指针相互链接，顺着链能按顺序访问全部数据。
- 再想四个选项：A、B、C 说的都是 B-树那边的特征，D 说的是“各叶结点之间通过指针相互链接”，这是 B+树独有的特征。
- 拿生活例子验算：B-树像一本把内容分散在各章节的书，每章里都有正文；B+树像一本目录章只写标题、正文全在最后一章、最后一章各页还用线串起来的书，顺着线能从头翻到尾。
- 边界情况：B+树的非叶结点不是没有关键字，而是不存数据，只存索引；叶结点链表是 B+树顺序访问的本事，B-树没有。
- 易混对子：不要把“多路平衡”当成区分点——B-树和 B+树都是多路平衡的；区分点在数据放哪一层、叶结点有没有链表。

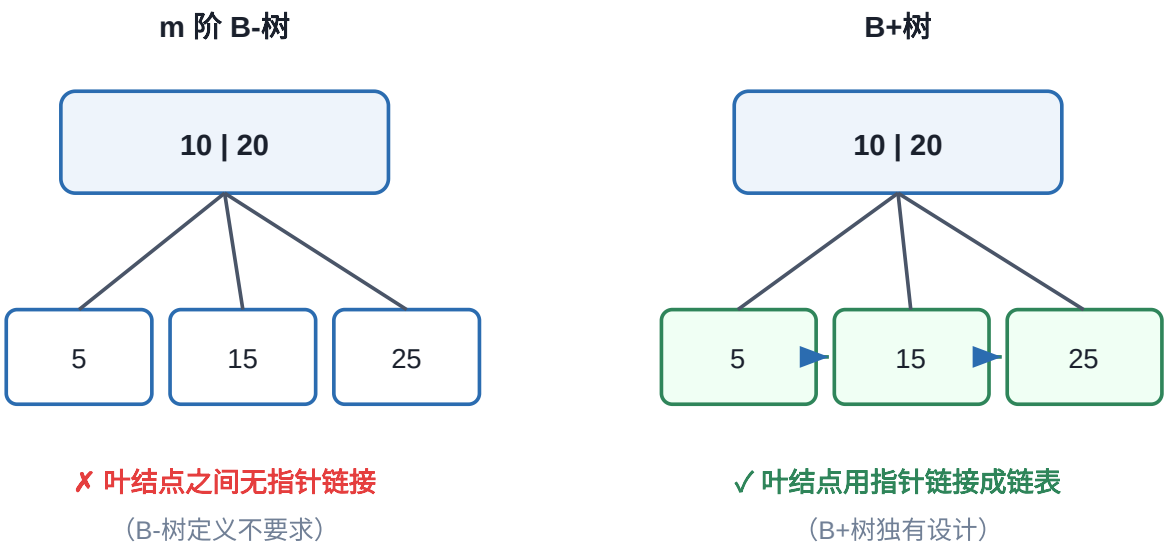
正确选项为什么对： D 说的是叶结点之间用指针链接，这是 B+树的特征，就是 D，选 D。

错误选项为什么错＋举例：

- 说 B-树特征的选项：关键字分布在各层、非叶结点存数据、叶结点没有链表，这些都是 B-树那边的样子，不是 B+树独有的。教训：先分清两边，再挑独有的那一个。
- 其他 B-树特征的选项：道理一样，只要说的是 B-树那边的特征，就不是答案。教训：三个 B-树特征先排除，剩下的就是 B+树。

笔记本一句话： 第 8 题＋B-树和 B+树：B+树数据全在叶结点，叶结点用指针串成链表，D 就是这条。

【图像】



叶结点指针链接是 B+树独有特征，不符合 m 阶 B-树定义要求 → 选 D

B-树与 B+树特征对比示意：叶结点指针链接是 B+树独有的设计

【定位】 数据结构 · 第 7 章查找 · B-树与 B+树的定义特征区分

【易混对比】

- B-树和 B+树是两回事：两者都要求所有叶结点在同一层、结点内关键字有序；但只有 B+树要求叶结点之间用指针链接成链表。
- 所有叶结点在同一层和叶结点互相链接是两回事：前者是两者共有的平衡要求，后者是 B+树独有的顺序遍历设计。

叶结点之间通过指针链接是 B+树的特征，不是 m 阶 B-树的要求→选 D。

【坑位】

A、B、C 都是 B-树和 B+树共有的特征，别被"根最多 m 棵子树""叶结点同层""关键字有序"迷惑——题问的是"不符合 B-树定义"，要找的是 B+树独有的那一条。

A：m 阶 B-树根结点最多 m 棵子树，是 B-树特征，成立。B：所有叶结点都在同一层上（B-树的平衡性要求），成立。C：各结点内关键字升序或降序排列，成立。D：叶结点之间通过指针链接——这是 B+树为了顺序遍历加的链表，m 阶 B-树的定义里没有这一条，不成立。所以不符合 B-树定义要求的是 D。

【类比】

像两套档案柜：B-树和 B+树都是"每层抽屉对齐、抽屉内文件按序排"；但 B+树在最底层抽屉之间加了一条传送带（指针链表），方便按顺序一口气取完。题问的是"B-树档案柜的标配"，传送带不是标配。

【口诀】 叶链指针 B+ 有，B-树定义无此条。

【考频】 「B-树」在 2009–2022 年解析中出现 4 次 · 中频

第 9 题 · 向小顶堆插入关键字后的向上调整结果（正确答案：A）

题干与选项：

已知关键字序列 5，8，12，19，28，20，15，22 是小根堆（最小堆），插入关键字 3，调整后得到的小根堆是_____。

- A. 3，5，12，8，28，20，15，22，19
- B. 3，5，12，19，20，15，22，8，28
- C. 3，8，12，5，20，15，22，28，19
- D. 3，12，5，8，28，20，15，22，19

入手点： 插入就两步：先把新关键字放到堆的末尾，再往上比——比父亲小就往上换，换到父亲不比它大为止。

专有名词从零解释：

- 先想“小顶堆”：一棵完全二叉树，每个父亲都不比它的孩子大；堆顶就是最小的那个。
- 再想“插入”：新元素先放到完全二叉树的最后一个空位，再一路往上比较交换，直到父亲不比它大。
- 再想“向上调整”：新元素和父亲比大小，小就往上换，换到合适的位置停下来。

考查知识点：

- 先想原堆：按原关键字序列建成小顶堆，每个父亲都不比孩子大，堆顶是最小的。
- 再想插入 3：把 3 放在小顶堆末端，再向上调整——3 比父亲小就往上换，一路换到堆顶附近；调整后的小顶堆序列为 3、5、12、8、28、20、15、22、19。
- 再想为什么是这个序列：3 是新插入的最小关键字，一路换到堆顶；其他结点之间的父子大小关系没变，只是位置跟着 3 的上移挪了挪。
- 拿生活例子验算：排队按个子从矮到高站，新来一个最矮的，从队尾一路往前换，换到队首；其他人相对顺序不变。
- 边界情况：插入后堆还是完全二叉树，形状不变，只换位置；向上调整最多换到堆顶，不会换出堆外。
- 易混对子：不要把新元素直接放到堆顶——要先放末尾再往上换；不要往下调整——插入是往上走，删除堆顶才是往下走。

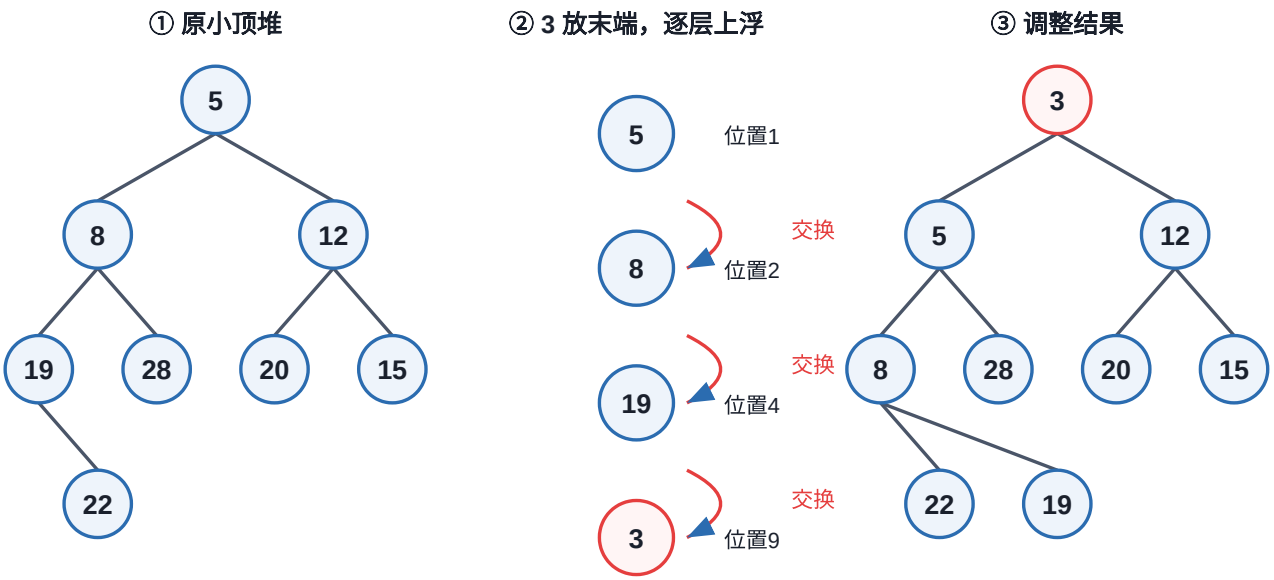
正确选项为什么对： 3 放末尾往上换到堆顶，序列变成 3、5、12、8、28、20、15、22、19，就是 A，选 A。

错误选项为什么错＋举例：

- 3 没换到堆顶的序列：向上调整没走完，父亲还比 3 大，违反小顶堆的规矩。教训：插入要一路换到父亲不比它大为止。
- 3 换错位置、其他结点顺序乱了的序列：调整只动 3 这一条线上的结点，不该动的地方动了，堆的规矩就坏了。教训：一次只换一条线，别的地方不动。
- 其他序列：按定义逐项核对，只要有一个父亲比孩子大，就不是小顶堆。教训：最后把每个父亲和孩子都比一遍。

笔记本一句话： 第 9 题＋小顶堆插入：新元素放末尾往上换，3 一路换到堆顶，序列是 3、5、12、8、28、20、15、22、19。

【图像】



3，5，12，8，28，20，15，22，19（选 A）

小顶堆插入关键字 3 的向上调整过程：3 与 19、8、5 逐层交换后到达堆顶

【定位】 数据结构 · 第 5 章树与二叉树 · 小顶堆的插入与向上调整

【易混对比】

- 插入和建堆是两回事：插入是把新关键字放末端再向上调整；建堆是从最后一个非叶结点开始向下调整。
- 向上调整和向下调整是两回事：插入用向上（新结点在末端，只可能比父小）；删除堆顶用向下。

插入：新关键字放堆末端，再与父结点比较、上浮，直到父结点不大于它为止。

【坑位】

别把 3 一步换到堆顶：调整是逐层交换，3 先后与 19、8、5 交换；B 的 3，5，12，19，20，…是只换了一层的半成品，D 的 3，12，5，…是左右孩子关系搞混的。

堆的数组表示（下标从 1 起）：5(1)，8(2)，12(3)，19(4)，28(5)，20(6)，15(7)，22(8)。3 插入到末端位置 9：3 小于父 19(4)，交换→3 到位置 4；3 小于父 8(2)，交换→3 到位置 2；3 小于父 5(1)，交换→3 到位置 1。此时父结点 5 不再大于 3，停止。结果：3，5，12，8，28，20，15，22，19＝A。

【套路】

- 第一步，新关键字放堆末端（数组最后一位），不破坏完全二叉树形状。
- 第二步，与父结点比较：比父小就交换（小顶堆），上浮一层。
- 第三步，重复直到父结点不大于它或到达堆顶。

放末端、比父小就换、换到顶或父更小为止。

【类比】

像班级按身高排队（矮的在前）：新来一个最矮的同学先站到队尾，然后一个一个往前"插队"——只要前面的人比他高就交换位置，一直换到队首。

【口诀】 插入放末端，小了往上换。

【考频】 「堆」在 2009–2022 年解析中出现 19 次 · 高频

第 10 题 · 由一趟或两趟后的序列特征识别排序算法（正确答案：B）

题干与选项：

若数据元素序列 11，12，13，7，8，9，23，4，5 是采用下列排序方法之一得到的第二趟排序后的结果，则该排序算法只能是_____。

- A. 起泡排序
- B. 插入排序
- C. 选择排序
- D. 二路归并排序

入手点： 每种排序算法每趟过后，序列都有自己的“印记”：冒泡和选择每趟定一个元素的最终位置，插入每趟让前面一段有序，归并每趟产生若干有序小段。

专有名词从零解释：

- 先想“冒泡排序”：每趟把最大的像气泡一样顶到最后面，一趟定一个最终位置。
- 再想“选择排序”：每趟从剩下里挑最小的放到前面，一趟定一个最终位置。
- 再想“插入排序”：每趟把下一个元素插进前面已经排好的那一段，前面一段始终有序。
- 再想“二路归并排序”：先把相邻两个合成有序小段，再把小段两两合并，第一趟过后应该出现若干两两有序的小段。

考查知识点：

- 先想冒泡和选择：每趟能确定一个元素的最终位置；题中前两个元素和后两个元素都不是最小或最大的两个元素按序排列，没有元素被定在最终位置上，所以冒泡和选择先排除。
- 再想二路归并：第一趟后应得到若干有序子序列，而题中序列当时并无两元素有序序列，所以 D 的二路归并也排除。
- 再想插入排序：每趟后能确定前面若干元素有序；题中第二趟后前三个元素有序，正好符合插入排序的印记。
- 拿生活例子验算：整理扑克牌，左手捏着的一叠始终是排好的，每新拿一张插进去，左手那叠还是排好的——这就是插入；冒泡和选择是每轮把一张牌彻底放好，归并是先两两配对再合并。
- 边界情况：判断时看的是“趟过后的印记”，不是最终结果；前几个元素有序，不等于它们已经在最终位置上。
- 易混对子：不要把“前面一段有序”当成冒泡——冒泡定的是最终位置，插入定的是前面一段有序；不要把“没有两元素有序段”不当回事——归并第一趟一定有两两有序的小段，没有就能排除。

正确选项为什么对： 第二趟后前三个元素有序，符合插入排序每趟让前面一段有序的印记，就是 B，选 B。

错误选项为什么错＋举例：

- 说冒泡排序的选项：冒泡每趟定一个元素的最终位置，题中前后元素都不是最值按序排列，没有元素被定下来。教训：冒泡看的是最终位置，不是前面有序。
- 说选择排序的选项：选择每趟也是定一个最终位置，题中同样没有元素被定在最终位置上。教训：选择和冒泡一样，一趟定一个。
- D（二路归并排序）：第一趟后应有若干两元素有序子序列，题中一个都没有。教训：归并的印记是成对的有序小段，没有就能排除。

笔记本一句话： 第 10 题＋认排序算法：冒泡选择一趟定一个最终位置，归并一趟出成对有序小段，插入是前面一段有序，第二趟前三有序就是插入。

【图像】



最终有序：4，5，7，8，9，11，12，13，23

没有一个元素在最终位置上 → 只能是插入排序（选 B）

四种排序趟后特征对比：插入排序前段有序、冒泡选择看落位、归并看有序段

【定位】 数据结构 · 第 8 章排序 · 由趟后序列特征识别排序算法

【易混对比】

- 前段有序和有元素落位是两回事：插入排序每趟让前面一段有序（但这些元素位置未必最终）；冒泡/选择每趟让一个元素到达最终位置。
- 第几趟和有序段长度是两回事：插入排序第 k 趟后前 k＋1 个元素有序；二路归并第 k 趟后有序段长度为 2^k。

第二趟后前三个元素有序（11，12，13），且没有元素在最终位置→插入排序的特征。

【坑位】

别被"11，12，13 递增"带偏去选冒泡：冒泡/选择每趟必有一个元素落到最终位置，题中最终有序序列是 4，5，7，8，9，11，12，13，23，对照发现没有一个元素已经在最终位置上，冒泡和选择直接排除；归并第二趟后应出现长度 4 的有序段，题中也没有。

最终有序：4，5，7，8，9，11，12，13，23。逐个对照题中序列 11，12，13，7，8，9，23，4，5：没有一个元素的位置与最终位置重合→排除 A（冒泡）、C（选择）。D 二路归并：第一趟后相邻两两成有序对、第二趟后形成长度 4 的有序段，题中前四个 11，12，13，7 不是有序段→排除。只有 B 插入排序：第二趟后前 3 个元素有序（11，12，13 成立），特征吻合。

【套路】

- 第一步，先写出完全有序的序列，对照趟后序列数有几个元素已在最终位置。
- 第二步，有元素落位→冒泡/选择类（落位数等于趟数）；无落位→看前段是否有序。
- 第三步，前 k＋1 个有序→插入排序；出现 2^k 长的有序段→归并排序。

先数落位元素，再看前段有序还是有序段。

【类比】

像整理扑克牌：插入排序是"每次拿一张插进已排好的手牌里"，两轮后手里前三张是顺的；冒泡/选择是"每轮把最大（小）的扔到一头"，两轮后两头各钉死一张。题里两头都没钉死、手牌前三张是顺的，只能是插牌。

【口诀】 落位看冒泡选择，前段有序是插入。

【考频】 「排序」在 2009–2022 年解析中出现 46 次 · 高频

第 11 题 · CPU 区分指令与数据的依据（正确答案：C）

题干与选项：

冯·诺依曼计算机中指令和数据均以二进制形式存放在存储器中， CPU 区分它们的依据是。

- A. 指令操作码的译码结果
- B. 指令和数据的寻址方式
- C. 指令周期的不同阶段
- D. 指令和数据所在的存储单元

入手点： 先想 CPU 干活的节奏：取指阶段和执行阶段是分开的，处在哪个阶段，就决定了取出来的是什么。

专有名词从零解释：

- 先想“指令周期”：CPU 执行一条指令要经过的几个阶段，先取指令，再执行指令。
- 再想“取指阶段”：CPU 到存储器里取指令的阶段，这个阶段取出来的就是指令。
- 再想“执行阶段”：CPU 按指令的要求去干活的阶段，这个阶段到存储器里取的，就是指令要操作的数据。
- 再想“译码”：CPU 把取出来的指令的操作码翻译成控制信号，告诉各部件该干什么。

考查知识点：

- 先想核心依据：CPU 按指令周期的不同阶段区分——取指阶段取出的是指令，执行阶段取出的是数据；阶段不同，取出来的东西身份就不同。
- 再想为什么二进制长得一样也没关系：指令和数据在存储器里都是 0 和 1，单看内容分不出来；分出来的办法不是看内容，而是看“什么时候取的”。
- 再想常踩的坑：CPU 只有在确定取出的是指令后，才把操作码送去译码；所以不能反过来“依据译码结果区分指令和数据”——译码是后面的事，不能当区分的依据。
- 拿生活例子验算：厨房里盐和糖长得像，厨师不是靠尝来分的，而是靠“什么时候拿”——做菜前从调料架拿的是调料，装盘后从柜子里拿的是盛器；阶段不同，东西的身份就不同。
- 边界情况：同一块存储器里指令和数据混着放，靠阶段区分才不出错；如果靠内容猜，0 和 1 长得一样，猜不出来。
- 易混对子：不要把“操作码”当成区分依据——操作码是取出来之后才译码的；不要把“存放位置”当成依据——指令和数据可以存在同一个存储器里。

正确选项为什么对： 按阶段区分——取指阶段取的是指令，执行阶段取的是数据，就是 C，选 C。

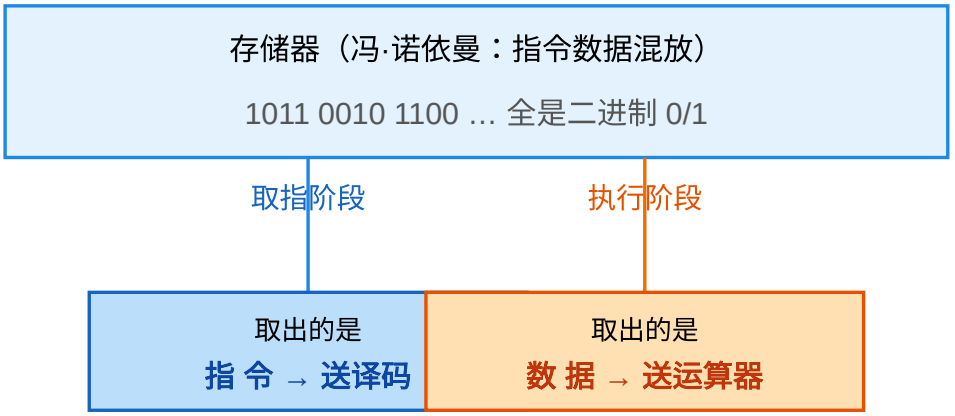
错误选项为什么错＋举例：

- 说“依据译码结果区分”的选项：译码发生在确定是指令之后，拿后面的事当区分依据，顺序反了。教训：先认定是指令，才送去译码。
- 说“看内容区分”的选项：指令和数据都是二进制，内容长得一样，分不出来。教训：分不出来的时候，想想“什么时候取的”。
- 其他表述的选项：只要不是“按阶段区分”，就没说到点子上。教训：阶段不同，身份不同，这是本题的钥匙。

笔记本一句话： 第 11 题＋指令和数据：不看内容看阶段，取指阶段取的是指令，执行阶段取的是数据。

【图像】

Q11 指令与数据长得一样，CPU 靠“阶段”区分



译码是区分之后的事，不能拿译码结果当区分依据 → 选 C

指令与数据按指令周期阶段区分：取指阶段取出的是指令，执行阶段取出的是数据

【定位】 计算机组成原理 · 第 1 章计算机系统概述 · 指令周期（取指/执行阶段区分指令与数据）

【易混对比】

- "译码结果"和"按阶段区分"：CPU 只有在取指阶段取出、确认这是"指令"之后，才敢把操作码送去译码；译码是区分之后的事，不能反过来当区分依据。
- "寻址方式"和"按阶段区分"：取指令和取数据都可以用直接寻址等方式，寻址方式相同说明不了取的是什么。
- "所在存储单元"和"按阶段区分"：冯·诺依曼机指令和数据混放在同一存储器，地址上长得一模一样，靠单元地址区分不了。

指令和数据在存储器里长得一模一样，CPU 区分靠"在什么时候取出来"，不靠"长什么样"。

【坑位】

别被"译码"二字忽悠——译码是区分之后的事。

最容易踩的坑是选 A"指令操作码的译码结果"。想一想：CPU 怎么知道现在送进译码器的就是操作码？只有先按阶段确定了"这次取的是指令"，才敢把取来的字节送去译码。拿译码结果当区分依据，是把果当成了因，因果颠倒。

【套路】

- 第一步看时间：取指阶段取出的就是指令，执行阶段取出的就是数据。
- 第二步看顺序：先按阶段确认是指令，再译码；译码结果不能反过来当依据。
- 第三步看存放：冯·诺依曼混放，地址和单元都帮不上忙。

什么时候取的，决定取的是什么。

【类比】

像火车站安检：同一条传送带上的包（存储器里的二进制串）长得都一样，安检口（取指阶段）过去的按"行李"处理，出口（执行阶段）拿出来的按"物品"处理——靠的是在哪个口子经过，而不是包上贴了什么标签（译

码)。

【口诀】 取指取来是指令，执行取来是数据；先区分后译码，因果别颠倒。

【考频】 「指令周期」在 2009–2022 年解析中出现 10 次 · 高频

第 12 题 · 不同字长整数相加时的符号扩展与结果（正确答案：D）

题干与选项：

一个 C 语言程序在一台 32 位机器上运行。程序中定义了三个变量 x、y 和 z，其中 x 和 z 为 int 型，y 为 short型。当 x=127，y=-9 时，执行赋值语句 z=x+y 后，x、y 和 z 的值分别是。

- A. x=0000007FH，y=FFF9H，z=00000076H
- B. x=0000007FH，y=FFF9H，z=FFFF0076H
- C. x=0000007FH，y=FFF7H，z=FFFF0076H
- D. x=0000007FH，y=FFF7H，z=00000076H

入手点： 先把两个数都变成 32 位再相加：短的那个要做符号扩展，符号位是 1 就补 1，是 0 就补 0。

专有名词从零解释：

- 先想“补码”：计算机里表示带符号整数的办法，负数是正数按位取反再加 1。
- 再想“字长”：一个数占几个二进制位；int 是 32 位，short 是 16 位。
- 再想“符号扩展”：把短数拉长到和长数一样长时，前面补的位要和符号位一样——负数补 1，正数补 0，这样数值不变。

考查知识点：

- 先想两个数：x 的十六进制是 0000007FH（就是 127），y 的十六进制是 FFF7H；y 的最高位是 1，说明 y 是负数，算出来是 −9。
- 再想符号扩展：y 从 16 位升到 32 位，符号位是 1，前面补 16 个 1，变成 FFFFFFF7H；不补 1 的话，y 就变成正数了，值就变了。
- 再想相加：0000007FH 加 FFFFFFF7H，得 00000076H；最高位进位自然丢弃，结果是 118，和 127−9＝118 对得上。
- 再想排除法：x 在四个选项中相同，不用算；z＝127−9＝118 大于 0，前 4 个字节一定是 0，能排除两个高字节不是 0 的选项；只算 y＝−9 的十六进制是 FFF7H，能排除没做符号扩展的选项。
- 拿生活例子验算：借钱记账，−9 元写成 16 位是 FFF7H；要和 32 位的账本对齐，前面得补 1，写成 FFFFFFF7H，补 0 就变成一大笔正数了，账就错了。
- 边界情况：符号扩展只看符号位，不看数值大小；正数补 0，负数补 1，补错了数值就变了；相加时最高位进位丢弃是正常的，不算溢出。
- 易混对子：不要把符号扩展当成“前面补 0”——补 0 是无符号数的做法，带符号负数要补 1；不要把“最高位进位”当成溢出——丢弃是补码加法的正常处理。

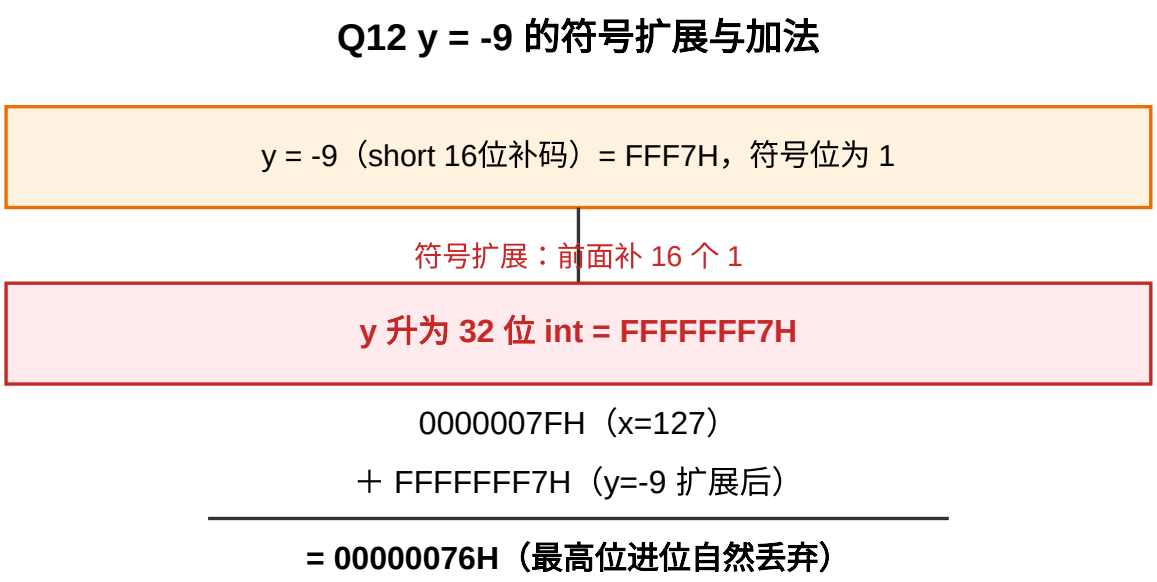
正确选项为什么对： y 符号扩展成 FFFFFFF7H，再加 x 得 00000076H，就是 D，选 D。

错误选项为什么错＋举例：

- 没做符号扩展的选项：y 前面补了 0，变成正数，相加结果就错了。教训：负数扩展补 1，不是补 0。
- 高字节不是 0 的选项：z＝118 大于 0，前 4 个字节一定是 0，高字节不是 0 的，数值就不可能是 118。教训：先算正负和大概值，再对高字节。
- 其他细节错的选项：x 相同不用算，盯住 y 的扩展和相加结果验。教训：先定正负，再算细节。

笔记本一句话： 第 12 题＋符号扩展：负数 short 升 int，前面补 16 个 1，y 变成 FFFFFFF7H，再加得 00000076H。

【图像】



【定位】 计算机组成原理 · 第 2 章数据的表示和运算 · 符号扩展

【易混对比】

- "符号扩展"和"零扩展"：有符号数短字长转长字长，看符号位补位——符号位为 1 则前面补 16 个 1（FFF7H→FFFFFFF7H），为 0 则补 0；零扩展是无符号数才用的，前面一律补 0。
- "进位丢弃"和"溢出"：32 位加法最高位的进位自然丢弃，赋值结果只看低 32 位，这不是错误，更不是溢出。
- "先验符号"和"直接算"：z＝127−9＝118＞0，结果必为正数，高 4 字节必为 0——先验符号能秒杀 B、C 两个选项。

看 y 的符号位：1 就补 1，0 就补 0；算完丢进位。

【坑位】

y＝-9 的十六进制是 FFF7H 不是 FFF9H，先把符号扩展算对再加。

最容易踩的坑有两个：一是把 -9 写成 FFF9H（那是 -7 的补码）；二是把 y 当 32 位数直接和 x 相加，忘了先符号扩展成 FFFFFFF7H。-9 的补码：9＝0009H→按位取反得 FFF6H→末位加 1 得 FFF7H。符号扩展时 y 的符号位为 1，前面补 16 个 1。

【套路】

- 第一步：把 y 写成 16 位补码：-9 → FFF7H。
- 第二步：符号扩展到 32 位：符号位为 1 → 前面补 16 个 1 → FFFFFFF7H。
- 第三步：相加，最高位进位丢弃：0000007FH＋FFFFFFF7H＝00000076H。
- 第四步验算：z＝127−9＝118＞0，高位全 0，秒杀 B、C。

符号位定补位，加完丢进位，符号先验算。

【类比】

像把一张小纸贴到大纸中央：纸原来是什么底色（符号位），四周就补什么底色——黑的（1）补黑边，白的（0）补白边。要是乱补底色，整张纸的"正负"就变了。

【口诀】 短转长看符号，1 补 1、0 补 0；加完丢掉最高位，正负先验算。

【考频】 「符号扩展」在 2009–2022 年解析中出现 18 次 · 高频

第 13 题 · 浮点数加法的对阶、尾数相加、规格化与溢出判断（正确答案：D）

题干与选项：

浮点数加、减运算过程一般包括对阶、尾数运算、规格化、舍入和判溢出等步骤。设浮点数的阶码和尾数均采用补码表示，且位数分别为 5 位和 7 位（均含 2 位符号位）。若有两个数 X=27 ×29/ 32，Y=25 ×5/8，则用浮点加法计算 X+Y 的最终结果是。

- A. 00111 1100010
- B. 00111 0100010
- C. 01000 0010001
- D. 发生溢出

入手点： 四步一步一步走：先对阶，再加尾数，再规格化，最后判溢出；少走一步，结果就不是最终结果。

专有名词从零解释：

- 先想“阶码”：浮点数里表示数量级的部分，像科学计数法里的指数。
- 再想“尾数”：浮点数里表示有效数字的部分，像科学计数法里的系数。
- 再想“对阶”：两个数的阶码不一样时，把小的那个向大的看齐，阶码加几，尾数就右移几位。
- 再想“规格化”：尾数要调整成标准形状；尾数相加后如果符号位进了位，就要右规（尾数右移一位、阶码加 1）。
- 再想“溢出”：阶码超出了能表示的范围，结果就溢出了。

考查知识点：

- 先想第一步对阶：X 的阶码是 00,111，Y 的阶码是 00,101，X 的阶码比 Y 大 2；按小阶向大阶看齐，Y 的阶码加 2、尾数右移 2 位，Y 变成阶码 00,111、尾数 00,00101。
- 再想第二步尾数相加：X 的尾数 00,11101 加 Y 的尾数 00,00101，得 01,00010；符号位进了位，变成 01，说明要右规。
- 再想第三步规格化：尾数右移 1 位、阶码加 1，得阶码 01,000、尾数 00,10001。
- 再想第四步判溢出：阶码符号位是 01，说明阶码溢出了，结果溢出，就是 D。
- 再想另一种算法：直接算出结果是 17/32×2^8；阶码用补码表示、数位为 3 位时最大只能表示 7，阶码 8 超出表示范围，也说明溢出。
- 拿生活例子验算：两个科学计数法的数相加，先把指数对齐，再加系数，系数进位了就调整指数，指数超了格子就装不下——溢出。
- 边界情况：对阶是“小阶向大阶看齐”，不是大阶向小阶；尾数相加符号位进位要右规，不是左规；判溢出看的是阶码的符号位。
- 易混对子：不要把中间结果当成最终结果——B 少了第三、第四步，C 少了第四步，算对了前面不等于算完了；不要把“尾数进位”当成溢出——尾数进位只要右规，只有阶码超范围才是溢出。

正确选项为什么对： 四步走完，阶码符号位是 01，溢出，就是 D，选 D。

错误选项为什么错＋举例：

- B：本身计算没错，但少了第三步规格化和第四步判溢出，停在中间结果上。教训：四步走完才算完。
- C：做了规格化，但少了第四步判溢出，没看出阶码超了范围。教训：规格化后还要判溢出。
- 其他表述的选项：只要没走到“阶码溢出”这个结论，就不是最终结果。教训：溢出看阶码，不看尾数。

笔记本一句话： 第 13 题＋浮点加法：对阶、尾数相加、右规、判溢出四步走完，阶码符号位 01 就是溢出。

【图像】

Q13 浮点加法四步：$X=2^{7}\times29/32$，$Y=2^{5}\times5/8$

① 对阶	小阶向大阶看齐 Y：阶 00,101→00,111，尾数右移2位	**00,111；00,00101**
② 尾数相加	00,11101＋00,00101 得 01,00010 → 尾数溢出	**需右规**
③ 规格化（右规）	尾数右移1位，阶码＋1 01,000；00,10001	**阶码双符号位 01**
④ 判溢出	阶码 01 = 上溢 阶码最大只到 7（00,111）	**发生溢出 → 选 D**

浮点加法四步：对阶、尾数相加、规格化右规、判阶码溢出

【定位】 计算机组成原理 · 第 2 章数据的表示和运算 · 浮点数加减运算

【易混对比】

- "对阶"和"规格化"：对阶是小阶向大阶看齐（Y 阶＋2、尾数右移 2 位，右移会丢精度）；规格化分左规（尾数左移、阶码−1）和右规（尾数右移、阶码＋1），本题是右规。
- "尾数溢出"和"阶码溢出"：尾数相加出现 01/10 叫尾数溢出，靠右规解决；只有阶码双符号位出现 01/10，才是真正的溢出，没得救。
- "B、C 和 D"：B、C 本身计算无误但不是最终结果——B 停在尾数相加（少了第三、第四步），C 停在规格化（少了第四步）；真正的考点是第四步判阶码。

尾数溢出右规能救，阶码溢出没得救。

【坑位】

算到 01000 0010001 就收手会错选 C——最后一步判阶码才是考点。

最容易踩的坑：前三步都算对了，得到阶码 01,000（十进制 8）、尾数 00,10001，看着像个正常结果就选了 C。但阶码是 5 位补码（含 2 位符号位），双符号位 01 说明上溢——阶码最大只能表示到 7（00,111），8 已经超了表示范围。少了第四步，前功尽弃。

【套路】

- 第一步对阶：小阶向大阶看齐：Y 阶 00,101→00,111，尾数右移 2 位 → 00,111；00,00101。
- 第二步尾数相加：00,11101＋00,00101＝01,00010，尾数溢出。
- 第三步规格化（右规）：尾数右移 1 位、阶码＋1 → 01,000；00,10001。
- 第四步判溢出：阶码双符号位 01 → 上溢 → D。

对阶、相加、规格化、判溢出，四步一步不能少。

【类比】

像两个人合伙记账：先把账本单位统一（对阶），再把钱加起来（尾数相加），加完发现位数写不下了就进一位（右规），最后看账本格子还装不装得下（判溢出）——格子装不下，账就爆了。

【口诀】 对阶相加再右规，阶码 01 就溢出。

【考频】 「浮点数」在 2009–2022 年解析中出现 13 次 · 高频

第 14 题 · 二路组相联 Cache 中主存块到 Cache 组的映射（正确答案：C）

题干与选项：

某计算机的 Cache 共有 16 块，采用 2 路组相联映射方式（即每组 2 块）。每个主存块大小为 32 字节，按字节编址。主存 129 号单元所在主存块应装入到的 Cache 组号是。

- A. 0
- B. 1
- C. 4
- D. 6

入手点： 先算 Cache 分了几组，再算这个单元在主存第几块，最后用块号对组数取余，组号就出来了。

专有名词从零解释：

- 先想“Cache”：CPU 和主存之间的一层快存，CPU 先到这里找，找不到再去主存。
- 再想“组相联”：Cache 里的块分成若干组，主存的块只能落到某一组里，但组里的哪个位置都可以。
- 再想“二路组相联”：每组里有 2 个位置，主存的块落到组里后，两个位置任选其一。
- 再想“主存块”：主存按固定大小切成的一块一块，每块大小一样。

考查知识点：

- 先想 Cache 分了几组：Cache 共有 16 块，二路组相联就是每组 2 块，一共 16÷2＝8 组，组号是 0、1、2、…、7。
- 再想主存块怎么映射：主存某一块只能落到某一组，主存第 0、8、16、…块落到第 0 组，第 1、9、17、…块落到第 1 组，依此类推，块号对 8 取余就是组号。
- 再想 129 号单元在第几块：每个主存块大小是 32 字节，从 0 开始数，129÷32＝4 余 1，所以 129 号单元在第 4 块。
- 再想第 4 块落到哪组：4 对 8 取余是 4，所以落到第 4 组的任一块里，就是 C。
- 拿生活例子验算：16 个抽屉两两一组编成 8 组，主存的第 4 号箱子只能放进第 4 组，组里两个抽屉任选其一。
- 边界情况：块号从 0 开始数，不是 1；单元号除以块大小，商是块号，余数是块内偏移；组号是块号对组数取余，不是单元号直接取余。
- 易混对子：不要拿单元号直接对组数取余——要先算出块号；不要把“组里的任一块”当成“固定的一块”——组相联是组内任选，不是全相联也不是直接映射。

正确选项为什么对： 129 号单元在第 4 块，第 4 块对 8 取余是 4，落到第 4 组，就是 C，选 C。

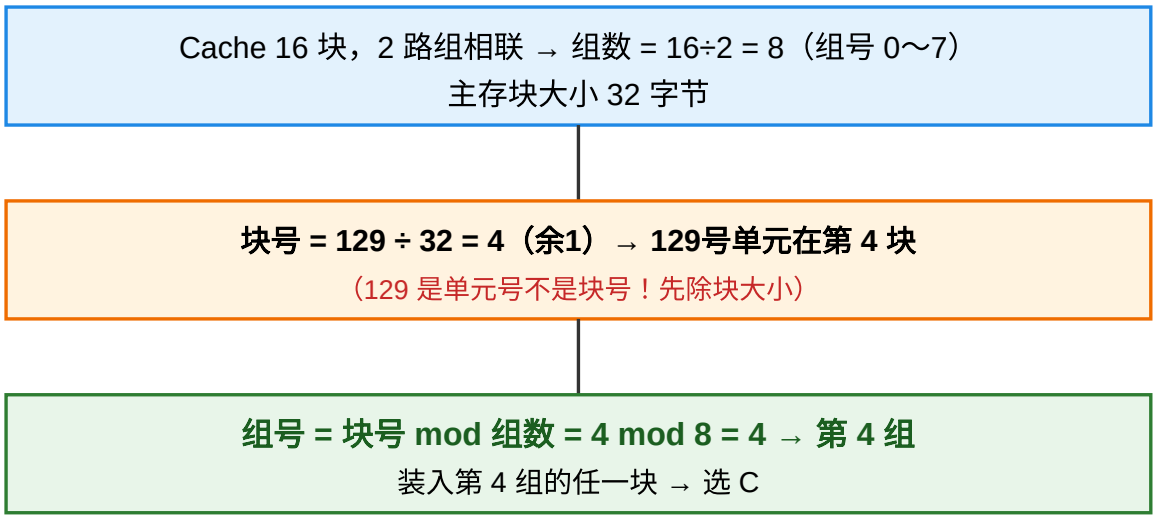
错误选项为什么错＋举例：

- 说其他组号的选项：要么块号算错了（比如从 1 开始数），要么直接拿单元号取余，没先算块号。教训：先算块号，再算组号。
- 说“固定某一块”的选项：组相联是组内任选，不是固定位置。教训：组相联管的是组，不是具体块。

笔记本一句话： 第 14 题＋组相联映射：16 块二路分 8 组，129 号单元在第 4 块，4 对 8 取余是 4，落第 4 组。

【图像】

Q14 129号单元 → 第4块 → 第4组



129号单元先除块大小得第4块，再对组数取模得第4组

【定位】 计算机组成原理 · 第 3 章存储系统 · Cache 组相联映射

【易混对比】

- "单元号"和"块号"：129 是字节单元号，不是块号。先 129÷32＝4（余 1）→ 第 4 块（从 0 开始编号）；直接拿 129 mod 8 是错的。
- "组数"和"块数"：16 块、2 路组相联 → 组数＝16÷2＝8，不是 16；组号＝块号 mod 组数。
- "从 0 开始"和"从 1 开始"：主存块从 0 编号，129÷32 商 4 就是第 4 块；若按"第几块"从 1 数会错一位。

单元号先除块大小得块号，块号再 mod 组数得组号。

【坑位】

129 不是块号！先除以 32 得块号 4，再 mod 8 得组号 4。

最容易踩的坑：拿 129 直接 mod 8（得 1，错选 B），或者忘了 2 路组相联要先算组数＝16÷2＝8。记住两步走：单元号→块号（除以块大小），块号→组号（mod 组数），一步都不能省。

【套路】

- 第一步：组数＝Cache 块数 ÷ 路数＝16÷2＝8。
- 第二步：块号＝单元号 ÷ 块大小＝129÷32＝4（余 1）。
- 第三步：组号＝块号 mod 组数＝4 mod 8＝4，装入第 4 组的任一块。

先除块大小，再 mod 组数，两步都别省。

【类比】

像 8 个抽屉（组）、每个抽屉 2 格：先看第 129 颗糖属于第几包（每包 32 颗 → 第 4 包），再看第 4 包该放进第几个抽屉（4 mod 8＝第 4 个）。不能直接拿糖数去除抽屉数。

【口诀】 块数除路得组数，单元除块得块号，块号取余定组号。

【考频】 「组相联」在 2009–2022 年解析中出现 19 次 · 高频

第 15 题 · ROM 与 RAM 芯片的数量计算及字扩展、位扩展方式（正确答案：D）

题干与选项：

某计算机主存容量为 64KB，其中 ROM 区为 4KB，其余为 RAM 区，按字节编址。现要用 2K×8 位的 ROM芯片和 4K×4 位的 RAM 芯片来设计该存储器，则需要上述规格的 ROM 芯片数和 RAM 芯片数分别是。

- A. 1、15
- B. 2、15
- C. 1、30
- D. 2、30

入手点： 先算总容量除以单片容量得片数，再看缺的是位数还是字数：缺字数用字扩展，缺位数用位扩展，两样都缺就两种一起用。

专有名词从零解释：

- 先想“字扩展”：单片的位数够了，但字数不够，就把几片芯片的字数拼起来，地址线多出来的部分用来选片。
- 再想“位扩展”：单片的字数够了，但位数不够，就把几片芯片的位数拼起来，拼成要的数据宽度。
- 再想“芯片规格”：比如 2K×8 位，就是 2K 个字、每个字 8 位；4K×4 位，就是 4K 个字、每个字 4 位。

考查知识点：

- 先想 ROM 区：ROM 区是 4KB，选用 2K×8 位的 ROM 芯片；位数 8 位已经够了，字数 2K 不够 4K；需要 4K×8÷(2K×8)＝2 片，用字扩展方式。
- 再想 RAM 区：RAM 区是 60KB，选用 4K×4 位的 RAM 芯片；字数 4K 不够 60K，位数 4 位不够 8 位，两样都缺；需要 60K×8÷(4K×4)＝30 片，用字和位同时扩展方式。
- 再想片数算法：总位数除以单片位数——ROM 是 4K×8＝32K 位，除以单片 2K×8＝16K 位，得 2 片；RAM 是 60K×8＝480K 位，除以单片 4K×4＝16K 位，得 30 片。
- 拿生活例子验算：拼积木，ROM 是宽度够了、长度不够，并排接 2 块就行；RAM 是宽度不够、长度也不够，既要并排接、又要叠起来，一共 30 块。
- 边界情况：算片数要按总位数算，不能只算字数或只算位数；字扩展和位扩展可以一起用，片数是两种扩展的乘积。
- 易混对子：不要把“字数不够”当成位扩展——字数不够是字扩展，位数不够才是位扩展；不要只算一种扩展的片数——两样都缺时，片数要按总位数算。

正确选项为什么对： ROM 2 片字扩展、RAM 30 片字位扩展，就是 D，选 D。

错误选项为什么错＋举例：

- ROM 片数或方式错的选项：ROM 位数够了，不用位扩展；字数差一倍，2 片就够。教训：先看位数够不够，再看字数差几倍。
- RAM 片数或方式错的选项：RAM 字数和位数都缺，只用一种扩展拼不出来；30 片是按总位数算出来的。教训：两样都缺，两种扩展一起上。
- 其他组合的选项：只要片数或方式有一处对不上，就不是答案。教训：片数和方式分开验。

笔记本一句话： 第 15 题＋存储器扩展：ROM 2 片字扩展，RAM 30 片字位扩展，片数按总位数除以单片位数算。

【图像】

Q15 存储器扩展：ROM 纯字扩展，RAM 字位同扩

ROM 区 4KB

4K×8 ÷ (2K×8) = 2 片

位数 8→8 够 → 纯字扩展

2 片地址译码并联

RAM 区 60KB（64−4）

60K×8 ÷ (4K×4) = 30 片

位数 4→8、容量 4K→60K

字和位同时扩展

片数 = 总容量(位) ÷ 单片容量(位)，别忘了 ×8

RAM 先减 ROM：64−4=60KB，直接拿 64 算会得 32（无此选项）

ROM 2 片字扩展、RAM 30 片字位扩展 → 选 D

ROM 2片字扩展与 RAM 30片字位同时扩展示意图

【定位】 计算机组成原理 · 第 3 章存储系统 · 存储器扩展

【易混对比】

- "字扩展"和"位扩展"：字扩展增加地址容量（位数不变），芯片地址线并联、片选译码区分；位扩展增加数据位数（容量不变），芯片地址线和片选全并联、数据线拼接。
- "ROM 和 RAM 的扩展方式"：ROM：2K×8→4K×8，位数已够 8 位、容量差一倍 → 纯字扩展；RAM：4K×4→60K×8，位数差一倍（×2）、容量差 15 倍 → 字和位同时扩展。
- "片数公式"：总容量（位）÷ 单片容量（位）。RAM：60K×8÷(4K×4)＝30。注意先把 KB 换算成位，别漏了 ×8。

位数不够就位扩展，容量不够就字扩展，两样都不够就一起扩。

【坑位】

RAM 区是 60KB 不是 64KB——先减掉 ROM 的 4KB 再算。

最容易踩的坑：直接拿 64KB 算 RAM 片数（得 32 片，根本无此选项），或者忘了 ×8（得 15，错选 B）。RAM 容量＝64−4＝60KB；片数＝60K×8÷(4K×4)＝30。题干给 64KB 是总量，ROM 的 4KB 要先扣掉。

【套路】

- 第一步定容量：RAM 区＝64−4＝60KB。
- 第二步算片数：ROM＝4K×8÷(2K×8)＝2 片；RAM＝60K×8÷(4K×4)＝30 片。
- 第三步定方式：ROM 位数正好 → 纯字扩展；RAM 位数 4→8、容量 4K→60K → 字和位同时扩展。

先定容量、再算片数、最后看位数定扩展方式。

【类比】

像拼乐高：字扩展是横向多拼几块板子（地址变多），位扩展是纵向把板子加厚（数据变宽）。ROM 只差长度，横向拼 2 块；RAM 又短又薄，横向拼 15 组、每组纵向叠 2 块，一共 30 块。

【口诀】 容量先减 ROM，位换别忘 ×8；位数不够位扩展，容量不够字扩展。

【考频】 「字扩展」在 2009–2022 年解析中出现 4 次 · 中频

第 16 题 · 相对寻址中取指后 PC 值的计算（正确答案：C）

题干与选项：

某机器字长 16 位，主存按字节编址，转移指令采用相对寻址，由两个字节组成，第一字节为操作码字段，第二字节为相对位移量字段。假定取指令时，每取一个字节 PC 自动加 1。若某转移指令所在主存地址为2000H，相对位移量字段的内容为 06H，则该转移指令成功转移后的目标地址是_____。

- A. 2006H
- B. 2007H
- C. 2008H
- D. 2009H

入手点： 先算取指后 PC 是多少：指令占几个字节，PC 就加几；再把 PC 和位移加起来，就是有效地址。

专有名词从零解释：

- 先想“PC”：程序计数器，里面放的是下一条要取的指令的地址；每取一个字节，PC 自动加 1。
- 再想“相对寻址”：有效地址等于 PC 的值加上一个位移，位移可正可负。
- 再想“取指后 PC”：把整条指令取完之后 PC 的值，不是取指之前的 PC。

考查知识点：

- 先想取指后 PC：转移指令由两个字节组成，每取一个字节 PC 自动加 1；取指前 PC 是 2000H，取完两个字节，PC 是 2000H＋2H＝2002H。
- 再想有效地址：相对寻址 EA＝(PC)＋A，取指后 PC 是 2002H，位移 A 是 06H；EA＝2002H＋06H＝2008H，就是 C。
- 拿生活例子验算：看书看到第 2000 页，书签夹在 2000 页；一口气读完两页，书签挪到 2002 页；再往后翻 6 页，就到了 2008 页。
- 边界情况：相对寻址用的 PC 是取指后的 PC，不是取指前的；指令占几个字节，PC 就加几，不能只加 1。
- 易混对子：不要拿取指前的 PC 直接加位移——那样少算了指令本身的长度；不要把“每取一个字节加 1”记成“每条指令加 1”——加几看指令占几个字节。

正确选项为什么对： 取指后 PC 是 2002H，再加位移 06H，得 2008H，就是 C，选 C。

错误选项为什么错＋举例：

- A：没有考虑 PC 自动更新，相当于拿取指前的 PC 直接加位移，少算了指令长度。教训：相对寻址先算取指后 PC。
- B：虽然考虑了自动更新，但只按加 1 计算；指令是两个字节，该加 2。教训：加几看指令占几个字节。
- 其他表述的选项：只要 PC 或位移有一处算错，有效地址就错。教训：先算 PC，再加位移。

笔记本一句话： 第 16 题＋相对寻址：先算取指后 PC（2000H＋2H＝2002H），再加位移 06H，得 2008H。

【图像】

Q16 相对寻址：基准是取指后的 PC



$$EA = (PC) + A = 2002H + 06H = 2008H$$

两个经典坑位

A：2000H＋06H = 2006H —— 忘了 PC 自动更新

B：2001H＋06H = 2007H —— 指令 2 字节，只加了 1

选 C

相对寻址 PC 更新流程：2000H 取指后得 2002H 再加位移 06H 得 2008H

【定位】 计算机组成原理 · 第 4 章指令系统 · 相对寻址

【易混对比】

- "取指前 PC"和"取指后 PC"：相对寻址的基准是取指完成后的 PC（2002H），不是指令首地址（2000H）。用 2000H＋06H＝2006H 是 A 坑。
- "指令占 2 字节"和"占 1 字节"：本题转移指令由两个字节组成，每取一字节 PC＋1 → 取指后 PC＝2000H＋2＝2002H；若只加 1 得 2007H 是 B 坑。
- "相对寻址"和"直接寻址"：直接寻址的 EA 就是形式地址 A；相对寻址的 EA＝(PC)＋A，PC 必须是取指后的值。

相对寻址的 PC 永远是"取指后"的 PC。

【坑位】

2000H 是指令首地址，不是算 EA 用的 PC——先 ＋2 得到 2002H 再加位移。

最容易踩的坑：A 直接用 2000H＋06H（忘了 PC 自动更新），B 只加了 1（没看清指令是 2 字节）。题干给了"每取一个字节 PC 自动加 1"和"由两个字节组成"，两处信息都要用上，缺一个就掉坑。

【套路】

- 第一步：取指后 PC＝2000H＋2＝2002H。
- 第二步：EA＝(PC)＋A＝2002H＋06H＝2008H。

先更新 PC，再加位移。

【类比】

像接力赛交接棒：起跑线（2000H）只是起点，下一棒从接棒点（跑完第一棒后的位置 2002H）开始算距离——不能回起跑线重算，也不能只跑半棒就交接。

【口诀】 相对寻址先取指，PC 加长再加位移。

【考频】 「相对寻址」在 2009–2022 年解析中出现 17 次 · 高频

第 17 题 · 识别不是 RISC 特点的表述（正确答案：A）

题干与选项：

下列关于 RISC 的叙述中，错误.. 的是。

- A. RISC 普遍采用微程序控制器
- B. RISC 大多数指令在一个时钟周期内完成
- C. RISC 的内部通用寄存器数量相对 CISC 多
- D. RISC 的指令数、寻址方式和指令格式种类相对 CISC 少

入手点： 先把 RISC 的特点列出来，再看哪个选项和这些特点对不上——对不上的就是答案。

专有名词从零解释：

- 先想“CISC”：复杂指令集，指令又多又复杂，一条指令能干很多事。
- 再想“RISC”：精简指令集，指令又少又简单，每条指令干的事少，干得快。
- 再想“控制器”：CPU 里发号施令的部件；硬布线控制器是用专门的逻辑电路直接发信号，微程序控制器是把控制信号存在存储器里、一条一条取出来用。

考查知识点：

- 先想 RISC 的特点：指令条数少；指令长度固定，指令格式和寻址种类少；只有取数、存数指令访问存储器，其余指令的操作都在寄存器之间进行；CPU 中通用寄存器多；大部分指令在一个或小于一个机器周期内完成；以硬布线逻辑为主，不用或少用微程序控制。
- 再想四个选项：B、C、D 说的都是上面这些 RISC 特点；A 说的是 RISC 普遍使用微程序控制器，这和“以硬布线逻辑为主”对不上，是错的。
- 再想为什么 RISC 用硬布线：RISC 指令简单规整，用逻辑电路直接发信号最快；微程序要一条一条取控制信号，慢，适合指令复杂、不好直接用电路实现的 CISC。
- 拿生活例子验算：快餐店菜单简单，服务员直接按固定流程上菜最快；大饭店菜式复杂，才需要翻菜谱（微程序）一道一道对。
- 边界情况：RISC 是“不用或少用”微程序，不是绝对不用；判断时看“普遍使用”这种绝对化的说法对不对。
- 易混对子：不要把“速度快”和“用微程序”连起来——RISC 快，恰恰是因为用硬布线；不要把 CISC 的特点安到 RISC 头上。

正确选项为什么对： A 说 RISC 普遍使用微程序控制器，和 RISC 以硬布线为主的特点相反，所以 A 是“不是 RISC 特点”的那一项，就是 A，选 A。

错误选项为什么错＋举例：

- B、C、D：说的都是 RISC 的特点——指令少、定长、只有取数存数访存、寄存器多、基本一个周期、硬布线为主；这些都是对的，所以不是答案。教训：先列特点，再一项一项对。
- 其他表述的选项：只要说的是 RISC 的特点，就不是“不是特点”的那一项。教训：题目问的是“不是”，别顺手选了“是”。

笔记本一句话： 第 17 题＋RISC 特点：指令少定长、只有取数存数访存、寄存器多、硬布线为主；说 RISC 普遍用微程序的 A 是错的。

【图像】

Q17 RISC 特点辨析：A 是 CISC 的帽子

A. 普遍采用微程序控制器 RISC 用硬布线，不用/少用微程序	✗ 错误
B. 大多数指令一个时钟周期完成 RISC 设计目标之一	✓ 正确
C. 内部通用寄存器数量相对多 操作多在寄存器间进行	✓ 正确
D. 指令数、寻址方式、格式种类少 指令简单、长度固定	✓ 正确

RISC = 简单指令＋硬布线＋多寄存器；微程序是 CISC 的标配 → 选 A

RISC 四个叙述辨析：A 微程序控制器是 CISC 的帽子

【定位】 计算机组成原理 · 第 4 章指令系统 · RISC 特点

【易混对比】

- "RISC 的控制器"和"CISC 的控制器"：RISC 以硬布线逻辑为主，不用或少用微程序控制；CISC 指令复杂，常采用微程序控制器。A 把 CISC 的帽子扣在了 RISC 头上。
- "指令条数"和"指令长度"：RISC 指令条数少、长度固定、格式和寻址种类少；CISC 指令条数多、长度可变、格式复杂。
- "访存指令"：RISC 只有取数、存数指令访问存储器，其余指令的操作均在寄存器之间进行，通用寄存器数量多。

RISC＝简单指令＋硬布线＋多寄存器；微程序是 CISC 的标配。

【坑位】

看到"微程序控制器"先想想是谁的标配——RISC 嫌它慢，根本不用。

最容易踩的坑：把 CISC 的特点记混到 RISC 上。RISC 追求"大多数指令在一个时钟周期内完成"，微程序每条指令都要访问控制存储器、速度慢，和 RISC 的设计目标背道而驰，所以 RISC 普遍采用硬布线控制器。A 的表述方向完全反了。

【类比】

像快餐店（RISC）vs 私房菜馆（CISC）：快餐菜单少、流程固定、现做现卖（硬布线）；私房菜菜品复杂，每道菜都要翻菜谱（微程序）慢慢做。A 说"快餐店每道菜都要翻菜谱"，跟快餐店的定位直接矛盾。

【口诀】 RISC 少固定多寄存，硬布线来不用微。

【考频】 「RISC」在 2009–2022 年解析中出现 2 次 · 低频

第 18 题 · 流水线时钟周期的取值依据（正确答案：A）

题干与选项：

某计算机的指令流水线由四个功能段组成，指令流经各功能段的时间（忽略各功能段之间的缓存时间）分别为 90 ns、80 ns、70 ns、和 60 ns，则该计算机的 CPU 时钟周期至少是。

- A. 90 ns
- B. 80 ns
- C. 70 ns
- D. 60 ns

入手点： 先想时钟周期的作用：每个时钟周期，各段同时往前推一步；周期太短，用时最长的那段就干不完。

专有名词从零解释：

- 先想“流水线”：把一条指令的执行拆成几段，像工厂流水线，每段同时干不同的活。
- 再想“时钟周期”：流水线往前推一步的时间间隔；每个周期，各段把手头的活交到下一段。
- 再想“最长的执行时间”：各段里用时最久的那一段的时间。

考查知识点：

- 先想为什么是最长：每个周期各段都要交活；周期比最长那段还短，最长那段就干不完，活交不出去，流水线就乱了。
- 再想四个选项：A 是最长执行时间；其他选项的数值或表述都没有说到“最长”这个点上。
- 拿生活例子验算：流水线上有快有慢的工人，传送带的速度得按最慢的那个工人来定；定快了，最慢的工人还没干完，活就被推走了。
- 边界情况：时钟周期可以比最长那段长一点（留余量），但不能比它短；周期定得越长，流水线跑得越慢。
- 易混对子：不要按最短的段来定——最短的干完了，最长的还没干完；不要按平均来定——平均比最长的小，最长的还是干不完。

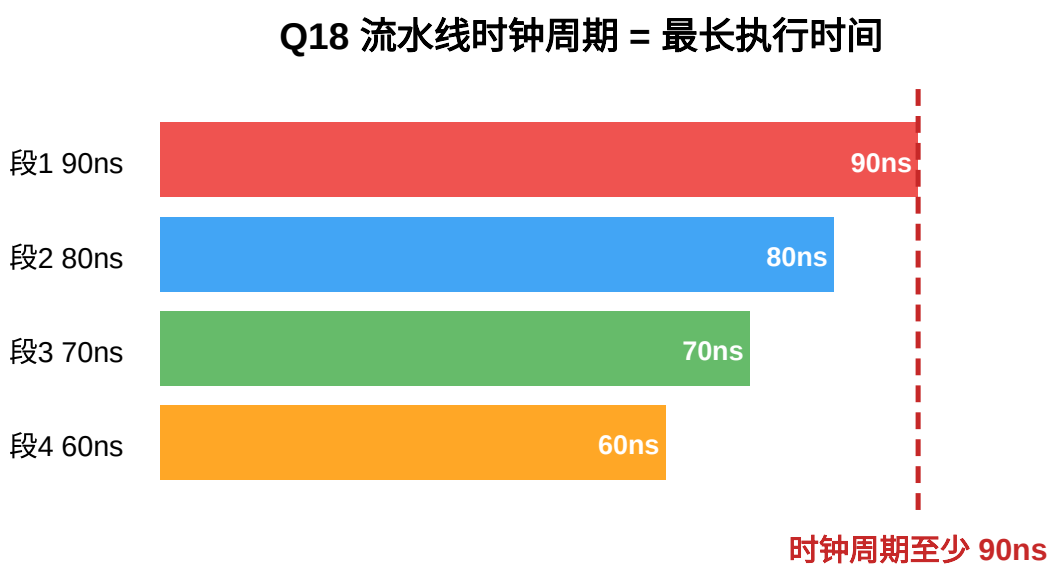
正确选项为什么对： 时钟周期应以最长的执行时间为准，就是 A，选 A。

错误选项为什么错＋举例：

- 说最短执行时间的选项：最短的干完了，最长的没干完，活交不出去。教训：周期看的是瓶颈，不是看最快的。
- 说平均时间的选项：平均比最长的小，最长的那段还是干不完。教训：瓶颈段的时间才是下限。
- 其他数值或表述的选项：只要没说“最长”，就没说到点子上。教训：流水线的节奏由最慢的一段定。

笔记本一句话： 第 18 题＋流水线时钟周期：按最长那段的时间定，短了最长那段干不完。

【图像】



四段流水线执行时间 90/80/70/60ns，时钟周期取最长 90ns

【定位】 计算机组成原理 · 第 5 章中央处理器 · 指令流水线

【易混对比】

- "最长"和"平均/最短"：时钟周期必须迁就最慢的段。取 80ns（B）或更小，90ns 的段在一个周期内做不完，功能不能正确完成。
- "时钟周期"和"吞吐率"：周期由最长段决定；流水线稳定后每个周期出一项结果，吞吐率＝1/周期。周期不是越小越好，小到慢段做不完就错了。
- "忽略缓存时间"：题干说忽略各功能段之间的缓存时间，所以周期就等于最长执行时间，不用再加缓冲。

木桶效应：流水线时钟周期由最慢的那段说了算。

【坑位】

别算平均、别取最短——90ns 的段在 80ns 的周期里根本做不完。

最容易踩的坑：看到四个数就想求平均或取最小。记住：时钟是全流水线统一的节拍，节拍必须够最慢的段用完，否则它的功能不能正确完成。至少 90ns，只能大不能小。

【类比】

像四人接力跑，每棒距离固定、速度不同：发令间隔必须按最慢那一棒的时间定，否则他还没跑完下一棒就出发了，全队乱套。

【口诀】 流水节拍看最长，慢段做完才交班。

【考频】 「流水线」在 2009–2022 年解析中出现 33 次 · 高频

第 19 题 · 微程序控制器与硬布线控制器的特点对比（正确答案：D）

题干与选项：

相对于微程序控制器，硬布线控制器的特点是。

- A. 指令执行速度慢，指令功能的修改和扩展容易
- B. 指令执行速度慢，指令功能的修改和扩展难
- C. 指令执行速度快，指令功能的修改和扩展容易
- D. 指令执行速度快，指令功能的修改和扩展难

入手点： 先把两种控制器的特点各列一边，再看哪个选项两边都对得上——一边对、一边错的组合都不行。

专有名词从零解释：

- 先想“微程序控制器”：把控制信号编成一段段微程序，存在控制存储器里；执行一条机器指令，就等于执行一段微程序。
- 再想“硬布线控制器”：不用存微程序，直接用专门的逻辑电路产生控制信号，像把流程焊死在电路里。
- 再想“灵活性”：改起来容不容易；微程序改存储器里的内容就行，硬布线改电路就难了。

考查知识点：

- 先想微程序控制器：采用“存储程序”原理，每条机器指令对应一个微程序；修改和扩充容易、灵活性好；但每条指令执行都要访问控制存储器，所以速度慢。
- 再想硬布线控制器：采用专门逻辑电路实现，速度主要取决于逻辑电路延迟，所以速度快；但修改和扩充困难、灵活性差。
- 再想四个选项：D 的组合是“指令执行速度快，但指令功能的修改和扩展困难”，这正是硬布线控制器的特点，和微程序控制器那边对上号后，两边都说对了。
- 拿生活例子验算：微程序像翻菜谱做菜，换菜谱容易，但每道菜都要翻一遍，慢；硬布线像把流程刻在机器上，做得快，但要换流程就得重新刻。
- 边界情况：速度和灵活性是反着来的——快的难改，好改的慢；判断组合时两边都要验，不能只看一边。
- 易混对子：不要把“快”安到微程序头上——微程序要访问控制存储器，慢；不要把“易改”安到硬布线头上——硬布线改电路，难。

正确选项为什么对： D 说的是硬布线控制器“指令执行速度快，但指令功能的修改和扩展困难”；硬布线用专门逻辑电路实现，速度取决于电路延迟，所以快，改电路困难，所以难改，D 说对了；微程序那边是易改但慢，D 没有把两边说反，就是 D，选 D。

错误选项为什么错＋举例：

- 把微程序说成“快”的选项：微程序每条指令都要访问控制存储器，慢。教训：微程序的代价就是速度。
- 把硬布线说成“易改”的选项：硬布线改的是逻辑电路，难。教训：硬布线的好处是快，不是易改。
- 其他组合错的选项：只要两边有一边说反了，整个组合就错。教训：组合题两边分开验。

笔记本一句话： 第 19 题＋两种控制器：微程序易改但慢，硬布线快但难改，D 说对了硬布线这边。

【图像】

Q19 微程序 vs 硬布线：慢而活， 快而死

	微程序控制器	硬布线控制器
指令执行速度	慢（每条指令访控存）	快（逻辑电路延迟）
修改/扩展	容易（改微指令）	困难（改电路）

题干问“相对于微程序，硬布线”→ 速度快、修改扩展难 → 选 D

别把 A（微程序的特点）当答案：先定参照系

微程序与硬布线控制器对比：速度与灵活性的取舍

【定位】 计算机组成原理 · 第 5 章中央处理器 · 控制器（微程序/硬布线）

【易混对比】

- "微程序的速度"和"硬布线的速度"：微程序每条机器指令都要访问控制存储器 → 慢；硬布线是专门逻辑电路实现 → 速度只取决于电路延迟 → 快。
- "微程序的灵活性"和"硬布线的灵活性"：微程序采用"存储程序"原理，改微指令就行 → 修改和扩充容易；硬布线改的是电路 → 修改和扩充困难。
- "和 Q17 的联动"：RISC 指令简单、用硬布线（快）；CISC 指令复杂、用微程序（好实现）。两题互相印证。

微程序：慢而活；硬布线：快而死。

【坑位】

题干问的是"相对于微程序，硬布线"——别把微程序的特点当答案。

最容易踩的坑：A 把"指令执行速度慢，修改和扩展容易"（微程序的特点）直接选了。先定参照系：题干问硬布线相对微程序怎么样 → 速度变快了、灵活性变差了 → D。读题先看"相对于谁"。

【类比】

像做菜：微程序是照着菜谱一步步做（慢，但换菜谱就能换菜）；硬布线是专用模具一次成型（快，但换花样要重开模具）。问"模具相对菜谱怎么样"，答案只能是"快而死"。

【口诀】 微程慢来修改易，硬线快却难扩展。

【考频】 「微程序」在 2009–2022 年解析中出现 6 次 · 中频

第 20 题 · 由总线周期与传输字节数计算总线带宽（正确答案：B）

题干与选项：

假设某系统总线在一个总线周期中并行传输 4 字节信息，一个总线周期占用 2 个时钟周期，总线时钟频率为 10MHz，则总线带宽是_____。

- A. 10M B/S
- B. 20M B/S
- C. 40MB/ S
- D. 80M B/S

入手点： 带宽就是单位时间传多少字节：先算一个总线周期是多长时间，再用这个周期里传的字节数除以时间。

专有名词从零解释：

- 先想“总线带宽”：单位时间内总线上传输数据的量，通常用每秒传送信息的字节数衡量，单位是 B/s。
- 再想“总线周期”：总线上传输一次数据要的时间；本题里 1 个总线周期等于 2 个时钟周期。
- 再想“时钟周期”：时钟频率的倒数；频率是 10MHz，时钟周期就是 1/10MHz＝0.1μs。

考查知识点：

- 先想一个总线周期多长：1 个总线周期等于 2 个时钟周期，时钟周期是 0.1μs，所以一个总线周期是 0.2μs。
- 再想一个周期传多少：1 个总线周期里传输了 4 字节信息。
- 再想带宽：4B÷0.2μs＝20MB/s，就是 B。
- 拿生活例子验算：传送带每 0.2 秒送过来 4 箱货，一秒就送 20 箱；先算一趟多长时间，再算一秒几趟。
- 边界情况：总线周期和时钟周期不是一回事，本题里一个总线周期是两个时钟周期；单位换算要对齐，μs 和 s 别混了。
- 易混对子：不要把总线周期当成时钟周期——那样时间少算一半，带宽就多算一倍；不要把“字节数”当成“位数”——带宽单位是 B/s，不是 b/s。

正确选项为什么对： 4B÷(2×0.1μs)＝20MB/s，就是 B，选 B。

错误选项为什么错＋举例：

- 把总线周期当成时钟周期的选项：时间少算一半，带宽多算一倍。教训：先看一个总线周期是几个时钟周期。
- 单位换算错的选项：μs 和 s 混了，或者字节和位混了，数值就差出数量级。教训：单位先对齐再算。
- 其他数值的选项：只要周期时间或字节数有一处算错，结果就错。教训：先算周期时间，再除字节数。

笔记本一句话： 第 20 题＋总线带宽：一个总线周期 0.2μs 传 4B，4B÷0.2μs＝20MB/s。

【图像】

Q20 总线带宽 = 字节 ÷ 总线周期

时钟频率 10MHz → 时钟周期 T = 1/10MHz = 0.1μs

1 总线周期 = 2 时钟周期 → 总线周期 = 0.2μs（不是 0.1μs！）

带宽 = 4B ÷ 0.2μs = 20MB/s → 选 B

坑：直接用 4B÷0.1μs=40MB/s 会错选 C

总线带宽计算流程：时钟周期 0.1μs→总线周期 0.2μs→带宽 20MB/s

【定位】 计算机组成原理 · 第 6 章总线 · 总线带宽

【易混对比】

- "总线周期"和"时钟周期"：1 总线周期＝2 时钟周期。直接拿 0.1μs 当总线周期，会算出 40MB/s 错选 C。
- "带宽单位"：总线带宽通常用 B/s（字节/秒）衡量，不是 b/s（位/秒）。4 字节＝32 位，别混。
- "频率和周期"：时钟周期 T＝1/f＝1/10MHz＝0.1μs。频率 10MHz 别直接当周期用。

先把"周期"统一成总线周期，再算字节除以时间。

【坑位】

1 总线周期＝2 时钟周期——分母是 0.2μs 不是 0.1μs。

最容易踩的坑：直接用 4B÷0.1μs＝40MB/s（C）。题干明确"一个总线周期占用 2 个时钟周期"：时钟周期 0.1μs → 总线周期 0.2μs → 4÷0.2μ＝20MB/s。周期翻倍、分母翻倍，答案减半。

【套路】

- 第一步：时钟周期 T＝1/f＝1/10MHz＝0.1μs。
- 第二步：总线周期＝2T＝0.2μs。
- 第三步：带宽＝4B÷0.2μs＝20MB/s。

周期先看清是几倍时钟，再除。

【类比】

像货车拉货：每 2 分钟（一趟＝2 个 1 分钟）拉 4 箱，平均每分钟 2 箱——不能按 1 分钟 4 箱算，那是把"趟"和"分钟"搞混了。

【口诀】 带宽等于字节除以总线周期，周期先看清几倍时钟。

【考频】 「总线带宽」在 2009–2022 年解析中出现 25 次 · 高频

第 21 题 · 由 Cache 命中与缺失次数计算命中率（正确答案：D）

题干与选项：

假设某计算机的存储系统由 Cache 和主存组成，某程序执行过程中访存 1 000 次，其中访问 Cache 缺失（未命中）50 次，则 Cache 的命中率是。

- A. 5%
- B. 9.5%
- C. 50%
- D. 95%

入手点： 先把题设的数字看清楚：给的是缺失次数，不是命中次数；命中次数要拿总数减缺失，再除以总数。

专有名词从零解释：

- 先想“命中”：CPU 要找的东西在 Cache 里，直接拿到了，这叫命中。
- 再想“缺失”：要找的东西不在 Cache 里，得去主存拿，这叫缺失。
- 再想“命中率”：命中次数除以总访问次数，越高说明 Cache 越管用。

考查知识点：

- 先想题设数字：总访存 1000 次，Cache 缺失（未命中）50 次；题设没有直接给命中次数。
- 再想命中次数：总共 1000 次，缺失 50 次，命中就是 1000−50＝950 次。
- 再想命中率：950÷1000＝95%，就是 D。
- 拿生活例子验算：去小卖部买 1000 次东西，50 次没货（缺失），950 次有货（命中），有货的比例就是 95%。
- 边界情况：命中率的分母是总访问次数，不是缺失次数；分子是命中次数，不是缺失次数。
- 易混对子：不要把“缺失 50 次”当成“命中 50 次”——那样算出来是 5%，正好掉进坑里；不要拿缺失次数除以总数当命中率——那是缺失率，不是命中率。

正确选项为什么对： 命中 950 次，总共 1000 次，950÷1000＝95%，就是 D，选 D。

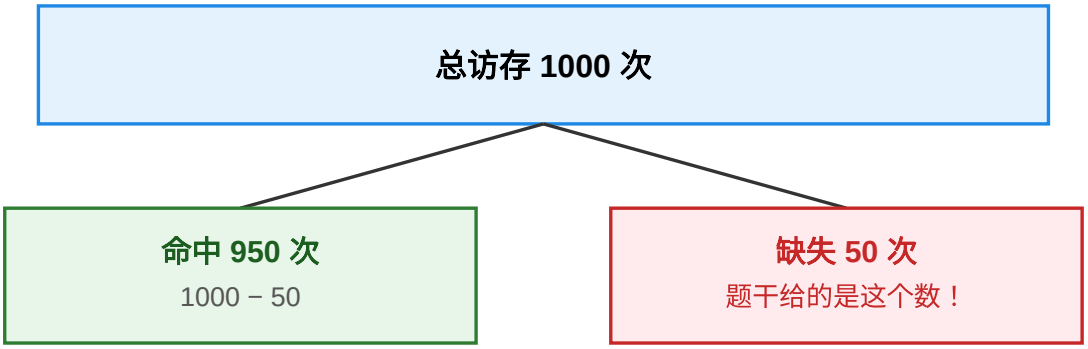
错误选项为什么错＋举例：

- 说 5% 的选项：那是把缺失 50 当成命中 50 算出来的，分子分母弄反了。教训：先看清题设给的是缺失还是命中。
- 其他百分数的选项：要么总数用错了，要么命中次数没减缺失。教训：先算命中次数，再除以总数。

笔记本一句话： 第 21 题＋Cache 命中率：总共 1000 次、缺失 50 次，命中 950 次，950÷1000＝95%。

【图像】

Q21 命中率：50 是缺失数，不是命中数



命中率 = 950 ÷ 1000 = 95% → 选 D

看成“命中 50 次”会得 5%（A 坑）

1000次访存分解：命中950次与缺失50次，命中率95%

【定位】 计算机组成原理 · 第 3 章存储系统 · Cache 命中率

【易混对比】

- "缺失 50 次"和"命中 50 次"：题干说的是缺失（未命中）50 次，命中＝1000−50＝950。看成命中 50 次会算出 5%（A 坑）。
- "命中率"和"缺失率"：命中率＝命中次数÷总访问次数＝95%；缺失率＝50÷1000＝5%。两个率别搞反。
- "命中次数要不要算"：题干没直接给命中次数，要拿总数减缺失数——这一步是本题唯一的计算。

先分清 50 是缺失数，命中数要拿总数去减。

【坑位】

50 是缺失次数不是命中次数——命中率的分子是 950，分母是总访问 1000。

最容易踩的坑：扫一眼"50 次"就按命中 50 算，得 5%（A）。题干写的是"缺失（未命中）50 次"：命中＝1000−50＝950，命中率＝950÷1000＝95%。先读题，再列式。

【套路】

- 第一步：命中次数＝总访问−缺失＝1000−50＝950。
- 第二步：命中率＝950÷1000＝95%。

缺失转命中先做减法，再除以总数。

【类比】

像考试 1000 道题，错了 50 道——正确率是 950÷1000＝95%，不是 50÷1000。错题数和对题数，先分清再算。

【口诀】 缺失不是命中数，总数减去再去除。

【考频】 「Cache」在 2009–2022 年解析中出现 20 次 · 高频

第 22 题 · 识别能产生外部中断的事件（正确答案：A）

题干与选项：

下列选项中，能引起外部中断的事件是。

- A. 键盘输入
- B. 除数为 0
- C. 浮点运算下溢
- D. 访存缺页

入手点： 先想外部中断的定义：CPU 执行指令以外的事件产生的中断，通常来自 CPU 和内存以外的设备。

专有名词从零解释：

- 先想“外部中断”：CPU 外面的设备有事要 CPU 处理，发来的中断请求，比如键盘敲了一个键。
- 再想“内中断（异常）”：CPU 执行指令时自己内部出的事，比如除数为 0、访存缺页。
- 再想“下溢”：浮点运算结果太小，小到没法表示，就按机器零处理。

考查知识点：

- 先想 A 键盘输入：键盘是 CPU 外面的输入设备，每次键盘输入，CPU 都需要执行中断来读入数据，属于外部事件，能引起外部中断。
- 再想 B 除数为 0：这是 CPU 执行除法指令时内部出的事，属于异常，也就是内中断，发生在 CPU 内部。
- 再想 C 浮点运算下溢：将按机器零处理，不会产生中断。
- 再想 D 访存缺页：这是 CPU 执行指令时产生的中断，也不属于外部中断。
- 拿生活例子验算：门外有人敲门（外部设备），屋里人要停下手头的活去开门；屋里自己绊了一跤（内部事件），那不是门外来的。
- 边界情况：外部中断看的是事件来自哪里——CPU 和内存以外，才是外部；执行指令过程中出的事，都是内部。
- 易混对子：不要把“缺页”当成外部中断——缺页是执行指令时访存出的事，是内部的；不要把“下溢”当成中断——下溢按机器零处理，不产生中断。

正确选项为什么对： 键盘输入是 CPU 外面的输入设备的事件，能引起外部中断，就是 A，选 A。

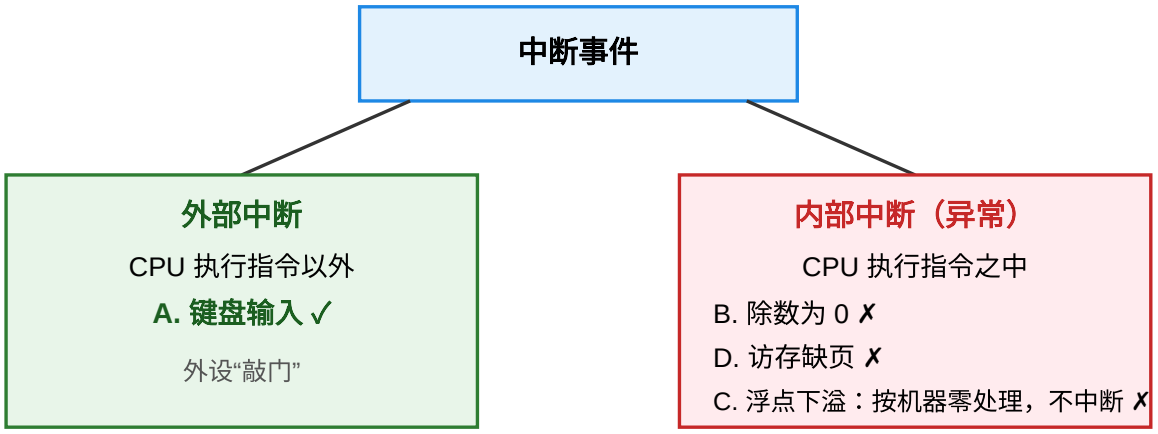
错误选项为什么错＋举例：

- B（除数为 0）：执行指令时 CPU 内部的异常，是内中断，不是外部中断。教训：内部出的事，不算外部。
- C（浮点运算下溢）：按机器零处理，不产生中断。教训：下溢有专门的处理办法，不用中断。
- D（访存缺页）：执行指令时访存出的事，是内中断，不是外部中断。教训：访存是执行指令的一部分。

笔记本一句话： 第 22 题＋外部中断：键盘输入是外面设备的事，能引起外部中断；除零、缺页是内部的，下溢不产生中断。

【图像】

Q22 内/外中断：看事件发生在“执行之外”还是“之中”



之外为外，之中为内 → 选 A

内外部中断分类：键盘输入是外部中断，除零缺页是内中断

【定位】 计算机组成原理 · 第 1 章计算机系统概述 · 中断分类

【易混对比】

- "外部中断"和"内部中断（异常）"：外部中断由 CPU 执行指令以外的事件引起（键盘输入、打印机请求）；内中断由 CPU 执行指令时内部产生（除数为 0、缺页、溢出）。
- "下溢"和"中断"：浮点运算下溢按机器零处理，不产生中断——不是"异常就一定中断"。
- "缺页"：访存缺页发生在 CPU 执行访存指令的过程中 → 内中断，不是外部中断。"外部"的"外"指 CPU 之外。

看事件发生在"执行指令之外"还是"执行指令之中"。

【坑位】

缺页、除零都是 CPU 执行指令时自己踩的坑——那是内中断，不是外部。

最容易踩的坑：把"访存缺页"当外部中断。缺页是 CPU 执行访存指令时发现的，属于执行过程中的内部事件；外部中断的"外部"指 CPU 与内存之外——键盘、鼠标、打印机这些外设的请求。

【套路】

- 第一步：问"这事发生在 CPU 执行指令之外吗"——键盘输入是人在外面敲的 → 外部中断。
- 第二步：除零、缺页都是执行指令时 CPU 内部发现的 → 内中断，排除。
- 第三步：浮点下溢按机器零处理，不产生中断 → 排除。

之外为外，之中为内。

【类比】

像上课：外部中断是有人敲门（外面来的事）；内中断是自己写作业写错了（做题过程中出的事）。键盘输入就是"有人敲门"，CPU 得停下来去开门（读入数据）。

【口诀】 外设敲门是外断，执行踩坑是内断；下溢归零不中断。

【考频】 「中断」在 2009–2022 年解析中出现 60 次 · 高频

第 23 题 · 单处理机系统中进程、通道与设备之间的并行关系（正确答案：D）

题干与选项：

单处理机系统中，可并行的是。

Ⅰ 进程与进程 Ⅱ 处理机与设备 Ⅲ 处理机与通道 Ⅳ 设备与设备

- A. Ⅰ、Ⅱ和Ⅲ
- B. Ⅰ、Ⅱ和Ⅳ
- C. Ⅰ、Ⅲ和Ⅳ
- D. Ⅱ、Ⅲ和Ⅳ

入手点： 先想单处理机是什么意思：同一时刻只能有一个进程占用处理机；再想通道和设备是不是独立于 CPU 干活的。

专有名词从零解释：

- 先想“单处理机系统”：只有一个处理机（不包含多核的情况），同一时刻只能干一件事。
- 再想“通道”：独立于 CPU 的、专门控制输入输出的设备，有自己的控制能力，能和 CPU 同时干活。
- 再想“并行”：同一时刻一起干活，不是轮流干。

考查知识点：

- 先想进程和进程：同一时刻只能有一个进程占用处理机，进程之间不能并行执行。
- 再想处理机和设备：设备是独立于 CPU 干活的，处理机算它的，设备传它的，两者可以并行。
- 再想处理机和通道：通道是独立于 CPU 的控制输入输出的设备，两者可以并行。
- 再想设备和设备：设备之间也是可以并行的，各传各的，互不耽误。
- 再想四个选项：D 的组合是 Ⅱ、Ⅲ和Ⅳ，即处理机与设备、处理机与通道、设备与设备三组并行；Ⅰ是进程之间并行，是错项。
- 拿生活例子验算：一个厨师（处理机）同一时刻只能炒一个菜（进程之间不能并行）；但厨师炒菜的同时，烤箱烤着东西、洗碗机洗着碗（设备和通道可以并行）。
- 边界情况：这里说的是单处理机、不包含多核；多核是另一回事，不能拿来套。
- 易混对子：不要把“并发”和“并行”混了——并发是轮流干，看着像一起；并行是同一时刻一起干；不要以为设备也要排队等 CPU——设备和通道有自己的控制能力。

正确选项为什么对： 进程之间不能并行，处理机与设备、处理机与通道、设备与设备三组可以并行，Ⅱ、Ⅲ、Ⅳ都对，就是 D，选 D。

错误选项为什么错＋举例：

- 含“进程之间并行”的选项：单处理机同一时刻只能一个进程占处理机，进程之间并行不了。教训：单处理机是瓶颈，进程只能轮流。
- 漏了某一组并行的选项：通道和设备独立于 CPU，设备之间也独立，漏掉任何一组，组合就不完整。教训：三组并行关系一项一项验。
- 其他组合的选项：只要组合里混进错的、或者漏掉对的，就不是答案。教训：组合题逐项验。

笔记本一句话： 第 23 题＋单处理机并行：进程之间不能并行，处理机与设备、处理机与通道、设备与设备三组可以并行，D 是 Ⅱ、Ⅲ、Ⅳ。

【图像】



单处理机系统中进程与进程串行、处理机与通道并行、处理机与设备并行、设备与设备并行的关系示意图

【定位】 操作系统 · 第1章 概述 · 单处理机系统的并行关系

【易混对比】

- 并行和并发的区别：并行是同一时刻真正同时做，单处理机里只有 CPU 之外的部件之间才做得到；并发是宏观上同时、微观上轮流，单处理机上的多个进程之间只能并发。
- 进程与进程和"处理机与设备"的区别：前者争的是同一个 CPU，单处理机同一时刻只能跑一个进程；后者是两套独立硬件各干各的，CPU 算它的、设备传它的数据。
- 通道和普通设备控制器的区别：通道是独立于 CPU 的 I/O 专用处理机，能和 CPU 并行工作，这正是"处理机与通道并行"的底气。

单处理机： Ⅰ进程与进程不能并行； Ⅱ、Ⅲ、Ⅳ都能并行，选 D。

【坑位】

看到"并行"就想到多进程同时跑，是单处理机题里最常见的翻车点。

单处理机只有一个 CPU，同一时刻只能有一个进程占用处理机，进程之间只能并发、不能并行，所以 Ⅰ 是错的。错选往往出在把"并发"当"并行"，或者想当然觉得多道程序就是并行。记住：单处理机 + 进程与进程 → 先判"不能并行"，再去看其他三组。

【套路】

- 第一步，先看有没有"单处理机"这个定语：有 → 进程与进程直接判错。

- 第二步，看剩下的三组是不是独立硬件：处理机与设备、处理机与通道、设备与设备，都是各干各的 → 都能并行。
- 第三步，组合： Ⅱ、Ⅲ、Ⅳ对 → 选 D。

单处理机先杀进程并行，再认独立硬件皆可并行。

【类比】

像一个厨师（单处理机）在厨房：他要同时做两道菜，只能切一下这道、炒一下那道（进程之间并发，不能真并行）；但他炒菜的同时，电饭煲自己煮饭（处理机与设备并行）、洗碗机自己洗碗（设备与设备并行），互不耽误。术语贴回去：厨师是"CPU"，电饭煲、洗碗机是"设备"，那个能自己按程序洗碗的智能洗碗机就是"通道"——有独立干活能力的硬件，才能和 CPU 并行。

【口诀】 单机进程只能串，通道设备各自转； Ⅰ 错 Ⅱ Ⅲ Ⅳ对，答案锁定 D。

【考频】 「通道」在 2009–2022 年解析中出现 14 次 · 高频

第 24 题 · 高响应比优先调度算法的响应比公式与性质（正确答案：D）

题干与选项：

下列进程调度算法中，综合考虑进程等待时间和执行时间的是_____。

- A. 时间片轮转调度算法
- B. 短进程优先调度算法
- C. 先来先服务调度算法
- D. 高响应比优先调度算法

入手点： 先把响应比公式写出来，再看这个公式同时照顾了谁：等待时间和执行时间都照顾到了。

专有名词从零解释：

- 先想“响应比”：一个进程“等了多久”加上“要跑多久”，再除以“要跑多久”；等得越久、跑得越短，响应比越高。
- 再想“调度”：操作系统决定下一个让哪个进程上处理机。
- 再想“高响应比优先”：每次挑响应比最高的进程投入执行。

考查知识点：

- 先想公式：响应比 R＝(等待时间＋执行时间)÷执行时间；也可以写成 1＋等待时间÷执行时间。
- 再想它照顾了谁：既看等待时间，也看执行时间；对于同时到达的长进程和短进程，短进程的响应比高，会优先执行，提高系统吞吐量；长进程的响应比会随等待时间的增加而提高，不会一直等不到，饿不死。
- 再想它和别的算法的区别：只看等待时间是先来先服务，长进程会拖住短进程；只看执行时间是短进程优先，长进程可能饿死；高响应比把两者揉在一起，两边都照顾。
- 拿生活例子验算：排队买票，等得久的人优先（照顾等待），买得快的人也优先（照顾执行）；等得越久，优先级涨得越快，排到最后的人也不会永远排不上。
- 边界情况：响应比是动态的——等待时间越长，响应比越大；刚到的进程响应比是 1（等待时间是 0）。
- 易混对子：不要把响应比记成“等待时间÷执行时间”——分子是等待加执行，不是只有等待；不要把高响应比当成短进程优先——短进程优先是它的一个效果，不是它的全部，长进程靠等待时间涨响应比。

正确选项为什么对： 高响应比优先选响应比最高的进程执行，公式和性质都对得上，就是 D，选 D。

错误选项为什么错＋举例：

- 只看等待时间的选项：那是先来先服务，不是高响应比。教训：高响应比的分子是等待加执行。
- 只看执行时间的选项：那是短进程优先，长进程可能饿死。教训：高响应比还看等待时间，长进程不会饿死。
- 其他表述的选项：只要公式或性质有一处说错，就不是答案。教训：公式和性质分开验。

笔记本一句话： 第 24 题＋高响应比优先：R＝(等待时间＋执行时间)÷执行时间，短进程先跑、长进程靠等待涨响应比不饿死。

【图像】

高响应比优先：综合等待时间和执行时间

R =（等待时间 + 执行时间）÷ 执行时间 = 1 + 等待时间 ÷ 执行时间

每次选响应比 R 最大的进程投入执行（非抢占）

例子：同时到达

短进程：等待 10，执行 2 → R = 1 + 10÷2 = 6 → 先执行 ✓

长进程：等待 10，执行 10 → R = 1 + 10÷10 = 2 → 后执行

长进程等到 100：R = 1 + 100÷10 = 11 → 轮到它，不饿死 ✓

短进程优先只看执行 → 长进程饿死；先来先服务只看等待 → 不看执行

结论：既看等待又看执行 → 只有高响应比优先 → 选 D

高响应比优先调度公式 R=(等待时间+执行时间)/执行时间=1+等待时间/执行时间，以及短进程先执行、长进程靠等待提高响应比的示意图

【定位】 操作系统 · 第2章 进程管理 · 高响应比优先调度算法

【易混对比】

- 高响应比优先和短进程优先的区别：短进程优先只看执行时间，长进程可能永远等（饥饿）；高响应比优先把等待时间也算进分子，等得越久响应比越高，长进程迟早轮到。
- 高响应比优先和先来先服务的区别：先来先服务只看等待时间（到达顺序），不管执行时间长短；高响应比优先两者都看。
- 高响应比优先和时间片轮转的区别：时间片轮转是抢占式的、按时间片轮流，高响应比优先是非抢占式的、一次选定响应比最高的投入执行。

响应比公式：R =（等待时间 + 执行时间）/ 执行时间 = 1 + 等待时间 / 执行时间。

既看等待又看执行 → 只有高响应比优先，选 D。

【坑位】

公式分子是"等待 + 执行"，不是只有"等待"。

最容易写错的是 R = 等待时间 / 执行时间，漏了分子的执行时间。记住变形：R = 1 + 等待时间/执行时间，"1 +"就是那个被漏掉的执行时间除以自己。另外注意：响应比是动态长的——长进程等得越久、R 越大，所以它克服了短进程优先的饥饿问题，这是它最大的卖点，考"为什么好"就答这个。

【套路】

- 第一步，看题干关键词："综合考虑等待时间和执行时间" → 直接锁定高响应比优先。
- 第二步，写对公式：R =（等待 + 执行）/ 执行 = 1 + 等待/执行。

- 第三步，选最大 R 的进程执行：同时到达时短进程 R 大先执行；长进程靠等待时间把 R 养大。

综合等待加执行，响应比最大者先行，长进程靠等待翻身。

【类比】

像银行排号叫号：既看你排了多久（等待时间），也看你办业务要花多久（执行时间）。只取个号的小业务（短进程）响应比高、先办，吞吐量上去；要办房贷的大业务（长进程）一开始排后面，但每多等一分钟、它的"优先级分"就涨一截，绝不会被晾到天黑（不饥饿）。术语贴回去：排队时长是"等待时间"，业务耗时是"执行时间"，优先级分就是"响应比 R"。

【口诀】 响应比，一加等待除以执；短进程先上，长等待翻身，综合考虑选 D。

【考频】 「调度」在 2009–2022 年解析中出现 35 次 · 高频

第 25 题 · 用鸽巢原理思想求打印机分配中的极端进程数（正确答案：C）

题干与选项：

某计算机系统中有 8 台打印机，由 K 个进程竞争使用，每个进程最多需要 3 台打印机。该系统可能会发生死锁的 K 的最小值是_____。

- A. 2
- B. 3
- C. 4
- D. 5

入手点： 先想死锁能发生的最挤情况：每个进程手里都拿着 2 台、就差 1 台；8 台分给这样的进程，能分给几个，死锁开始成为可能的最小进程数就是几个。

专有名词从零解释：

- 先想“鸽巢原理”：东西比盒子多的时候，至少有一个盒子里不止一个东西；反过来想最挤的情况，就能算出临界值。
- 再想“极端情况”：最容易出问题、最挤的那种分配——每个进程都差一台就能跑。
- 再想“打印机”：这里是一种独占资源，一个进程占着，别的进程就用不了。

考查知识点：

- 先想每个进程最多需要 3 台：如果每个进程已经占有了 2 台打印机，那么只要还有多的打印机，总能满足一个进程达到 3 台的条件，然后顺利执行。
- 再想极端分配：将 8 台打印机分配给 K 个进程、每个进程有 2 台打印机时，这个情况是极端情况；8÷2＝4，K 为 4。
- 再想为什么是 4：4 个进程各占 2 台，正好把 8 台分完，每个都差 1 台，谁都拿不到第 3 台，死锁就可能发生了；这就是最挤的临界状态。
- 再想为什么不是 3：3 个进程最挤也只占 6 台，还剩 2 台，总能让一个进程拿到第 3 台跑完，所以 3 个进程时死锁不可能发生。注意区分：可能死锁的最小值是 4，保证不死锁的最大值是 3——本题问的是前者。
- 拿生活例子验算：8 把椅子，每个人差一把就能坐满一桌（3 把）；4 个人每人先占 2 把，8 把正好分完，谁都差一把，这就是最挤的时候。
- 边界情况：极端情况看的是“每个进程都差一台”，不是“每个进程都拿满 3 台”；拿满 3 台的进程已经能跑了，不算挤。
- 易混对子：不要把“可能死锁最少 4 个”当成“保证不死锁最多 4 个”——保证不死锁最多是 3 个（3 个最挤占 6 台、还剩 2 台，总能让一个进程跑完）；4 个是死锁开始成为可能的分界线；不要把“每个进程占 2 台”当成最终状态——极端情况是用来找临界值的。

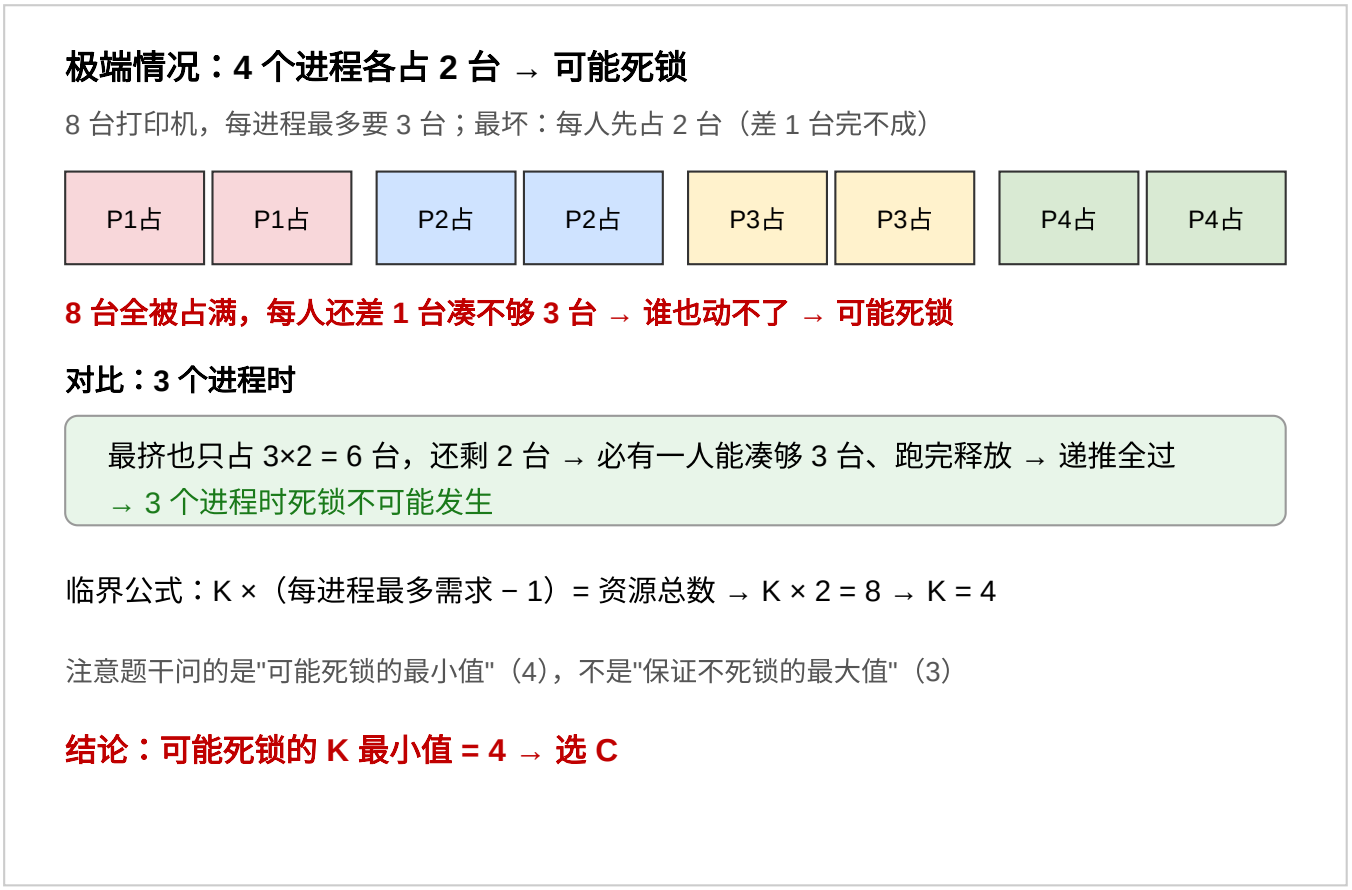
正确选项为什么对： 8 台分给每个占 2 台的进程，8÷2＝4，极端进程数 K 为 4，就是 C，选 C。

错误选项为什么错＋举例：

- 说 3 的选项：3 个进程最挤只占 6 台，还剩 2 台，总能让一个进程拿到第 3 台跑完，死锁不可能发生。教训：可能死锁的最小值，要看最挤时资源够不够分完；3 个进程时资源分不完，死锁就无从谈起。
- 说 5 的选项：5 个进程各占 2 台要 10 台，8 台根本不够。教训：先看总数够不够分。
- 其他数值的选项：只要不是 8÷2＝4，就没算对极端情况。教训：先定极端状态，再除。

笔记本一句话： 第 25 题＋打印机极端进程数：可能死锁最少 4 个（每个占 2 台差 1 台，8÷2＝4）；保证不死锁最多是 3 个。

【图像】



8 台打印机被 4 个进程各占 2 台、每人还差 1 台凑不够 3 台而可能死锁的极端分配示意图

【定位】 操作系统 · 第2章 进程管理 · 死锁的资源分配极端分析

【易混对比】

- "可能死锁的 K 最小值"和"保证不死锁的 K 最大值"的区别：前者是"再多一个就可能出事"的临界点（本题 4），后者是"到这个数都还安全"的上限（本题 3）；两者差 1，题干问哪个答哪个。
- 极端情况和平均情况的区别：算死锁临界永远按最坏情况摆——每个进程都卡在"差 1 台就能完成"的状态，而不是平均分配。
- 占有且等待和一次性全部分配的区别：死锁分析默认进程是逐台申请、拿住不放（占有且等待），不是一次要齐 3 台。

8 台、每进程最多 3 台：K=4 时可能死锁（每人占 2 台占满），K=3 时必不死锁，选 C。

【坑位】

题干问的是"可能死锁的最小值"，答成"保证不死锁的最大值"就差 1 翻车。

最容易误选 B（3）：3 是"保证不死锁的最多进程数"——3 个进程最挤也就占 6 台，还剩 2 台，必有一个进程能凑够 3 台、跑完释放，递推下去谁都不会卡死，所以 3 个时死锁不可能发生。但题干问的是"可能发生死锁的 K 的最小值"，要找的是第一个"可能出事"的数：4 个进程时，每人占 2 台正好占满 8 台，谁也凑不够 3 台 → 可能死锁。所以答案是 4，不是 3。记住：先想"最坏怎么摆"，再看题干问的是"可能死锁最小"还是"保证不死锁最大"。

【套路】

- 第一步，摆最坏情况：每个进程都占"最多需求 − 1"台（本题 2 台），谁都差 1 台完不成。
- 第二步，看资源够不够把所有进程都卡住：K × 2 = 8 → K = 4 时 8 台被占满，谁也动不了。
- 第三步，验证 K−1 是否安全：3 个进程最多占 6 台，剩 2 台必有一人能凑够 3 台 → 不可能死锁，确认 4 是最小值。

每人差一台占满全部资源，资源数除以（需求−1）得临界 K。

【类比】

像 8 个停车位、K 辆大车，每辆要占 3 个车位才能掉头开走。最坏情况：4 辆车每辆先抢 2 个，8 个车位占满，谁都差 1 个、谁也开不走——死锁。3 辆车时最多占 6 个，还剩 2 个，必有一辆能凑到 3 个开走、腾出 3 个，下一辆也走得成。术语贴回去：车位是"打印机"，大车是"进程"，先抢 2 个不放是"占有且等待"，谁也开不走是"死锁"。

【口诀】 每人差一台，资源全占满；八除以二得四，可能死锁最小四。

【考频】 「死锁」在 2009–2022 年解析中出现 11 次 · 高频

第 26 题 · 进程地址空间越界时的地址保护与中断（正确答案：A）

题干与选项：

分区分配内存管理方式的主要保护措施是_____。

- A. 界地址保护
- B. 程序代码保护
- C. 数据保护
- D. 栈保护

入手点： 先想每个进程都有自己的地址空间，出了这个空间就是越界；越界要靠硬件检查，拦下来并产生中断。

专有名词从零解释：

- 先想“地址空间”：一个进程能用的地址范围，像每个住户自己的房间号段。
- 再想“地址越界”：进程要访问的地址不在自己的地址空间里，像住户要开别人家的门。
- 再想“地址保护”：硬件检查每次访问是不是被允许，不允许就拦下来。
- 再想“地址越界中断”：检查出越界后，硬件产生的中断，告诉操作系统来处理。

考查知识点：

- 先想每个进程的独立空间：每个进程都拥有自己独立的进程空间，别的进程的地址不能随便碰。
- 再想检查怎么做：当程序要访问某个内存单元时，由硬件检查是否允许；如果允许就执行，如果不允许就产生地址越界中断。
- 再想为什么是硬件检查：访问内存是每时每刻都在发生的事，靠软件事后查来不及，硬件检查才跟得上。
- 拿生活例子验算：小区门禁，住户刷卡只能开自己那栋楼的门；刷别人家的门，门禁直接拦下来并报警。
- 边界情况：地址保护拦的是“不允许的访问”，不是所有访问；允许的访问正常执行，不产生中断。
- 易混对子：不要把越界当成“直接让它访问”——不拦的话，一个进程能改掉别的进程的数据；不要把中断当成“程序自己发现的”——是硬件检查发现的。

正确选项为什么对： 地址越界要靠地址保护拦下来，硬件检查不允许就产生地址越界中断，就是 A，选 A。

错误选项为什么错＋举例：

- 说“不检查直接访问”的选项：那样一个进程能碰别的进程的内存，系统就乱了。教训：独立空间要靠保护来保证。
- 说“软件事后发现”的选项：访问内存是高频操作，事后查来不及，得硬件当场检查。教训：保护是硬件的活。
- 其他表述的选项：只要没说到“硬件检查、不允许就产生地址越界中断”，就没说到点子上。教训：保护、中断两个词都要有。

笔记本一句话： 第 26 题＋地址越界：硬件检查访问是不是被允许，不允许就产生地址越界中断。

【图像】



分区分配中硬件用上、下界寄存器检查每次访存地址，界内合法、界外产生地址越界中断的示意图

【定位】 操作系统 · 第3章 内存管理 · 分区分配的内存保护

【易混对比】

- 界地址保护和地址越界中断的区别：前者是"事前检查"的保护措施（硬件比对地址是否在界内），后者是"事后处理"（越界发生后 CPU 响应的中断）；题干问的是"主要保护措施"，答措施不答中断。
- 界地址保护和程序代码保护/数据保护/栈保护的区别：后三者是干扰项，分区分配不管代码还是数据，只管"地址有没有跑出这个分区的上下界"。
- 分区分配的保护和分页分段的保护的区别：分页靠页表项的保护位，分段靠段表项的界限与权限，分区分配靠的是一对上、下界寄存器。

分区分配只认上下界，硬件检查越界 → 界地址保护，选 A。

【坑位】

题干问"主要保护措施"，别答成"越界后发生什么"。

"地址越界中断"是越界发生之后的处理动作，不是保护措施本身；保护措施是"用界地址（上、下界寄存器）做检查"这件事。另外三个选项（程序代码保护、数据保护、栈保护）听起来都像保护，但分区分配的粒度是整个分区，保护的就是"别跑出界"，跟代码还是数据无关。

【类比】

像小区的门禁：每户（进程）有自己的楼栋门牌范围（上、下界），保安（硬件）在每次有人进楼时核对"这栋楼是不是你家的范围"，在就放行、不在就拦下（越界中断）。保安不管你是回家睡觉还是开派对（不管代码还是数据），只管你有没有走错楼。术语贴回去：门牌范围是"界地址"，保安是"硬件检查"，拦下是"地址越界中断"。

【口诀】 分区上下界，硬件来把关；越界就中断，保护选 A 项。

【考频】 「内存管理」在 2009–2022 年解析中出现 6 次 · 中频

第 27 题 · 由地址总长度与段号位数求分段存储的最大段长（正确答案：C）

题干与选项：

一个分段存储管理系统中，地址长度为 32 位，其中段号占 8 位，则最大段长是_____。

- A. 28 字节
- B. 216 字节
- C. 224 字节
- D. 232 字节

入手点： 先把地址拆成两段：段号和段内位移；段内位移有几位，最大段长就是 2 的几次方。

专有名词从零解释：

- 先想“分段存储”：逻辑地址分成段号和段内位移两部分，段号指第几段，段内位移指段里的第几个位置。
- 再想“段内位移”：段里的偏移量，有几位，就能表示几个位置。
- 再想“最大段长”：一段里最多能有多少个字节，就是段内位移能表示的位置个数。

考查知识点：

- 先想地址怎么拆：地址长度为 32 位，段号占 8 位，剩下的是段内位移，占 32−8＝24 位。
- 再想最大段长：段内位移 24 位，能表示 2^24 个位置，所以最大段长为 2^24 B。
- 拿生活例子验算：一个小区 32 位门牌号，前 8 位是楼号，后 24 位是房间号；一栋楼里最多有 2^24 间房。
- 边界情况：最大段长看的是段内位移的位数，不是地址总长度；段号位数变了，最大段长跟着变。
- 易混对子：不要拿 32 位直接算 2^32——那是整个地址空间，不是单段的长度；不要拿段号的 8 位算 2^8——那是段的个数，不是段的长度。

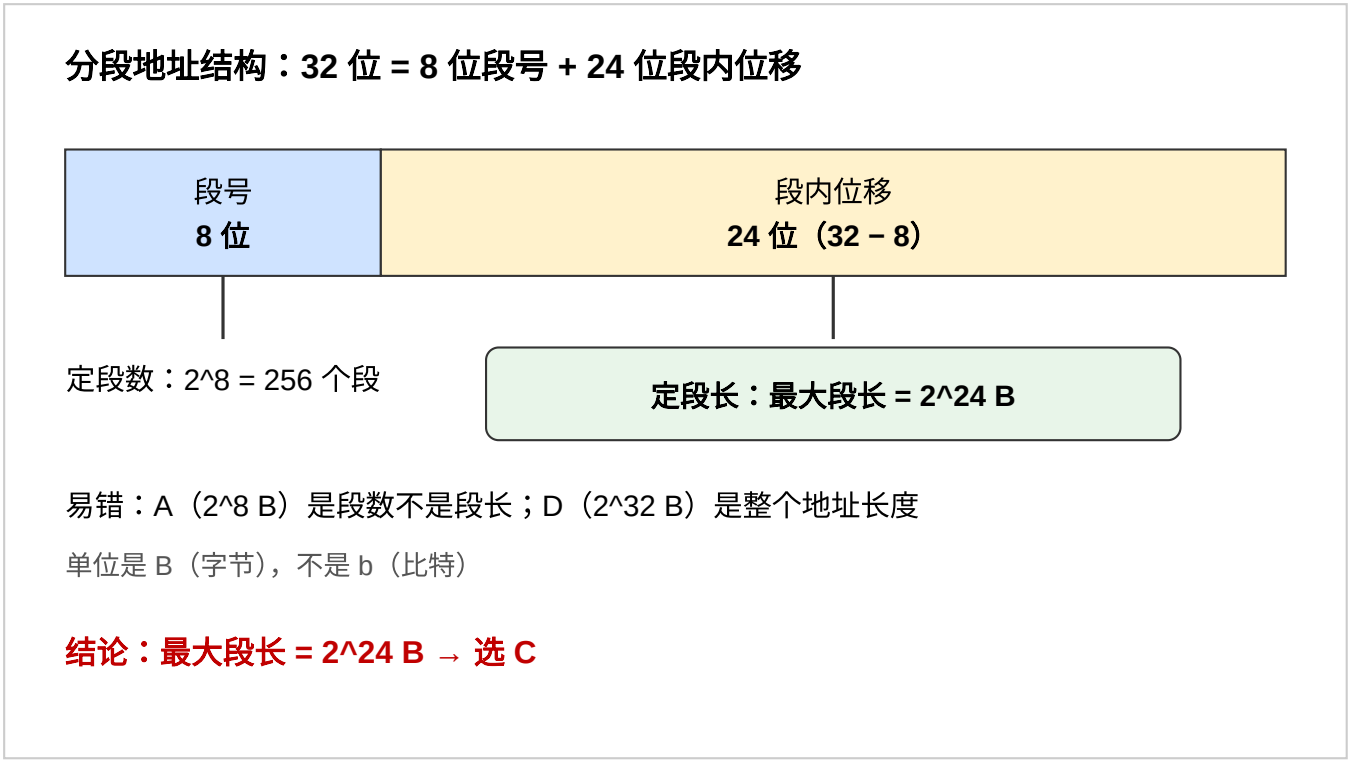
正确选项为什么对： 段内位移 24 位，最大段长 2^24 B，就是 C，选 C。

错误选项为什么错＋举例：

- 说 2^32 B 的选项：那是整个地址空间，不是单段。教训：先拆段号和位移。
- 说 2^8 B 的选项：那是段号能表示的段数，不是段长。教训：段长看位移位数。
- 其他数值的选项：只要位数拆错了，结果就错。教训：32−8＝24，再算 2^24。

笔记本一句话： 第 27 题＋最大段长：32 位地址减 8 位段号剩 24 位位移，最大段长 2^24 B。

【图像】



32 位逻辑地址分为 8 位段号和 24 位段内位移，段号定段数、位移定最大段长 2^24B 的结构示意图

【定位】 操作系统 · 第3章 内存管理 · 分段存储的地址结构

【易混对比】

- 段号位数和段内位移位数的区别：段号定"最多有几个段"（2^8 = 256 个），位移定"一个段最大多长"（2^24 B）；题干问最大段长，看的是位移那 24 位。
- 最大段长和地址总长度的区别：32 位是整个逻辑地址的长度，不是段长；段长只看段内位移部分。
- 分段的最大段长和分页的页面大小的区别：分页的页面大小由页内偏移位数定，分段的段长由段内位移位数定，思路一样、名字不同。

位移 = 32 − 8 = 24 位 → 最大段长 = 2^24 B，选 C。

【坑位】

三处易错：拿段号 8 位当答案（A）、拿总长 32 位当答案（D）、算出 24 却忘了单位 B。

A（2^8 字节）是把"段号位数"当成了"段长"，它实际定的是段数；D（2^32 字节）是把整个地址长度当成了段长，相当于说一个段能占满整个地址空间。另外注意单位：地址位数算出来的是字节数，答案是 2^24 B（字节），不是 2^24 b（比特）。

【套路】

- 第一步，拆地址：段内位移位数 = 地址总长 − 段号位数 = 32 − 8 = 24。
- 第二步，位移定段长：最大段长 = 2^位移位数 = 2^24 B。
- 第三步，核对：问的是段长不是段数，单位是 B，选 C。

总长减段号得位移，二的位移次方即最大段长。

【类比】

像门牌号"3 栋 1201 室"：栋号占几位决定小区最多有几栋（段数 = 2^段号位数），室号占几位决定一栋楼最多有几户（段长 = 2^位移位数）。题目问"一栋楼最多几户"，看的是室号的位数，不是栋号、也不是整个门牌号的长

度。术语贴回去：栋号是"段号"，室号是"段内位移"，一栋的户数上限是"最大段长"。

【口诀】 总长减段号，剩下是位移；二的位移次方，最大段长 2^24。

【考频】 「分段」在 2009–2022 年解析中出现 19 次 · 高频

第 28 题 · 既能随机访问又易于扩展的文件物理结构（正确答案：B）

题干与选项：

下列文件物理结构中，适合随机访问且易于文件扩展的是_____。

- A. 连续结构
- B. 索引结构
- C. 链式结构且磁盘块定长
- D. 链式结构且磁盘块变长

入手点： 先把三种结构各的特点列出来，再看哪个同时满足“随机访问”和“容易扩展”两个要求。

专有名词从零解释：

- 先想“连续结构”：文件的数据一块接一块排着，像一排连着的座位。
- 再想“链式结构”：文件的数据用指针一串串连起来，像糖葫芦。
- 再想“索引结构”：专门有一张索引表，记着每一块数据在哪，像书的目录。
- 再想“随机访问”：想读第几块就直接读第几块，不用从头一串串找。
- 再想“容易扩展”：文件长大了，加新块容不容易。

考查知识点：

- 先想连续结构：数据连着排，随机访问容易（算一下位置就找到了），但文件长大了要加块，得找一大片连着的空地，不容易扩展。
- 再想链式结构：加新块容易（往链子后面挂一块就行），但随机访问不行——要读第几块，得从头顺着指针一串串找过去。
- 再想索引结构：索引表记着每一块的位置，随机访问直接查表就行；加新块也容易，往索引表里加一行、再找一块空地就行，两个要求都满足。
- 拿生活例子验算：连续像一排连座，想加座得拆墙；链式像糖葫芦，想吃中间那颗得从头撸；索引像带目录的书，想翻哪页查目录就行，加页也只要在目录里加一行。
- 边界情况：索引结构要多花一张索引表的空间，这是代价；但题目问的是两个要求，不是代价。
- 易混对子：不要把“连续能随机访问”当成全对——连续扩展难；不要把“链式易扩展”当成全对——链式随机访问难；两个要求要同时满足。

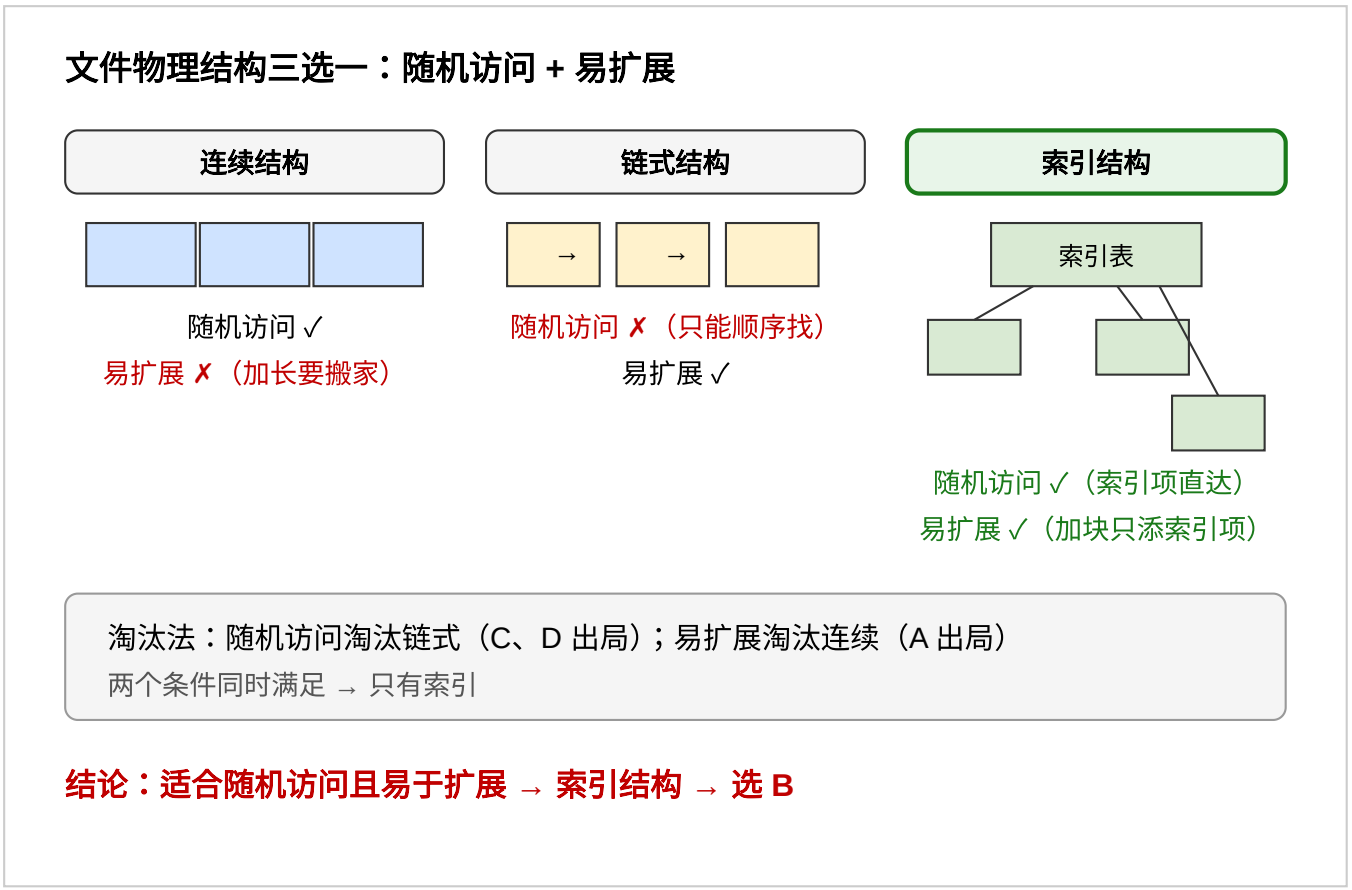
正确选项为什么对： 索引结构随机访问查表就行、扩展加表项就行，两个要求都满足，就是 B，选 B。

错误选项为什么错＋举例：

- 说连续结构的选项：随机访问行，但扩展难，要找连着的空地。教训：两个要求缺一不可。
- 说链式结构的选项：扩展容易，但随机访问要从头顺着找。教训：随机访问要直接定位，不能顺着找。
- 其他表述的选项：只要只满足一个要求，就不是答案。教训：两个要求一起验。

笔记本一句话： 第 28 题＋文件物理结构：连续难扩展，链式难随机访问，索引结构两者都行。

【图像】



连续、链式、索引三种文件物理结构在随机访问与易扩展性上的对比示意图

【定位】 操作系统 · 第4章 文件管理 · 文件的物理结构

【易混对比】

- 索引和连续的区别：连续也要整块连续空间，文件加长时找不到相邻空块就得整块搬家（扩展难）；索引的盘块可以分散存放，加块只添索引项。
- 索引和链式的区别：链式要找第 N 块必须从头顺着指针一块块找（不能随机访问）；索引直接由索引项算出第 N 块的地址。
- 定长块链式和变长块链式的区别：都是链式，都只能顺序访问，变长还多一个"块大小不一、管理更麻烦"的毛病，随机访问同样不行。

随机访问淘汰链式，易扩展淘汰连续 → 索引两者兼得，选 B。

【坑位】

题干两个条件必须同时满足，单看一个条件会误选。

只看"随机访问"会误选 A（连续）：连续确实能随机访问，但文件一加长、找不到连续空块就得搬家，扩展性差。只看"易于扩展"会误选 C/D（链式）：链式加块确实方便，但找第 N 块只能从头顺藤摸瓜，随机访问不行。两个条件一交叉，只剩索引。

【套路】

- 第一步，查随机访问：链式只能顺序找 → C、D 出局。
- 第二步，查易扩展：连续要整块连续空间、加长得搬家 → A 出局。
- 第三步，剩索引：索引项直接定位（随机访问✓），加块只添索引项（易扩展✓）→ 选 B。

随机先杀链式，扩展再杀连续，剩下索引即答案。

【类比】

像找书：索引结构是带目录的书——要看第 50 页直接翻目录（随机访问），加新章节只在目录添一行、内容页塞哪都行（易扩展）；连续结构是从头抄到尾的卷轴，加内容得整卷重抄；链式结构是寻宝图、一张纸条指着下一张，必须一张张找下去。术语贴回去：目录是"索引表"，纸条是"盘块指针"，卷轴是"连续分配"。

【口诀】 随机又扩展，索引来帮忙；连续难扩展，链式不能跳，答案选 B。

【考频】 「文件」在 2009–2022 年解析中出现 60 次 · 高频

第 29 题 · SCAN 磁盘调度算法的磁头移动与服务顺序（正确答案：A）

题干与选项：

假设磁头当前位于第 105 道，正在向磁道序号增加的方向移动。现有一个磁道访问请求序列为 35，45，12，68，110，180，170，195，采用 SCAN 调度(电梯调度)算法得到的磁道访问序列是 _____。

- A. 110,170,180,195,68,45,35,12
- B. 110,68,45,35,12,170,180,195
- C. 110,170,180,195,12,35,45,68
- D. 12,35,45,68,110,170,180,195

入手点： 先想 SCAN 像电梯：先往一个方向走，顺路把这个方向的请求都服务了，再掉头往回走。

专有名词从零解释：

- 先想“磁道”：磁盘上一圈一圈的轨道，编号从小到大。
- 再想“磁头”：读写数据的头，在磁道之间移动。
- 再想“SCAN 算法”：像电梯一样，先往一个方向走到底（顺路服务），再往回走。

考查知识点：

- 先想出发方向：磁头从 105 道向序号增加的方向移动，先服务所有大于 105 的磁道请求，按从小到大的顺序：110、170、180、195。
- 再想掉头：大于 105 的都服务完了，磁头往回走，按从大到小的顺序服务剩下的请求：68、45、35、12。
- 再想完整顺序：110、170、180、195、68、45、35、12，就是 A。
- 拿生活例子验算：电梯从 105 楼往上走，顺路停 110、170、180、195 楼；到顶了再往下走，顺路停 68、45、35、12 楼。
- 边界情况：SCAN 掉头后不是从最小的开始往上，而是从大的往小的顺路服务；小于出发点的请求要等掉头后才服务。
- 易混对子：不要把返程顺序写成从小到大——返程是往回走，顺序是从大到小；不要把小于 105 的请求插到前面——出发方向是向大，先服务大于 105 的。

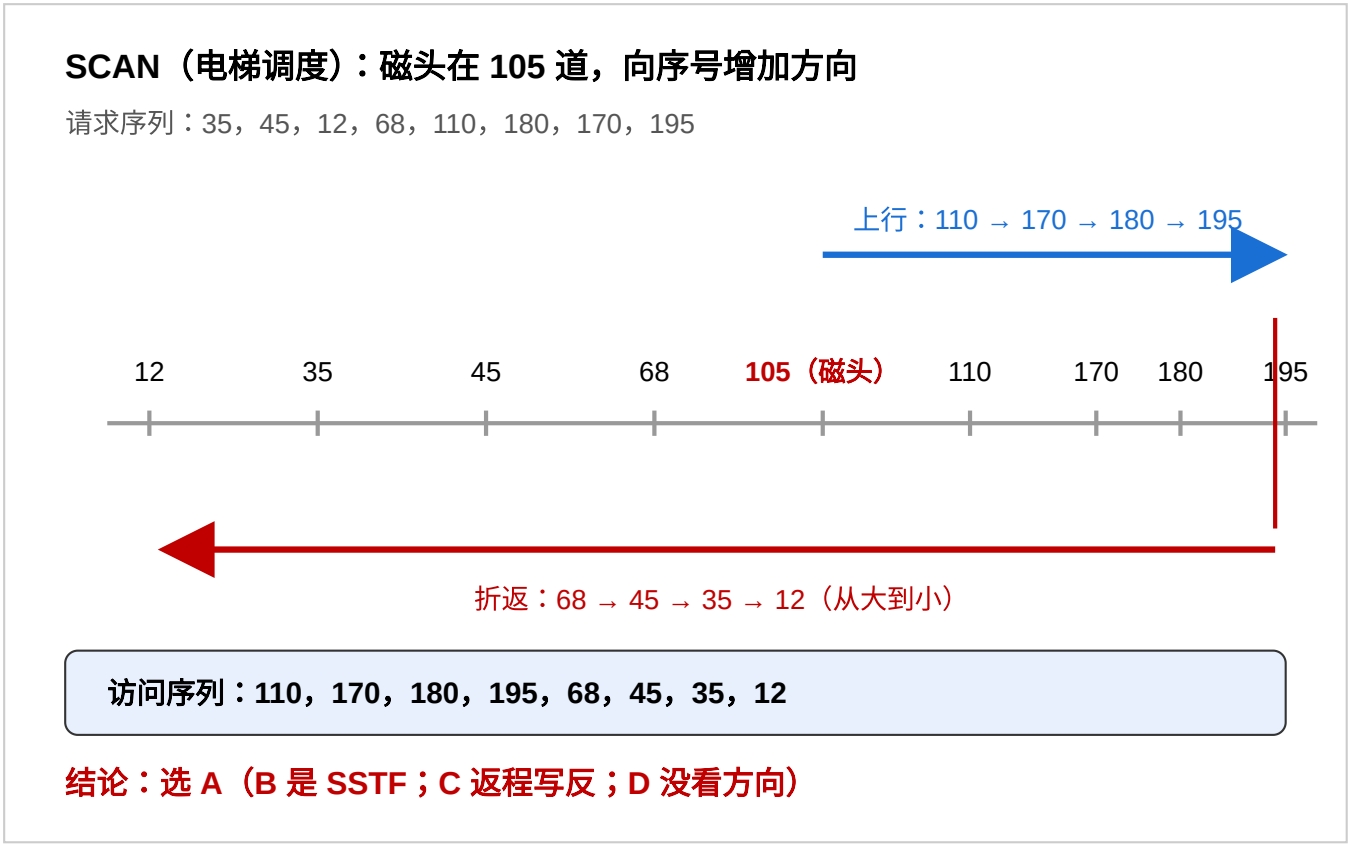
正确选项为什么对： 先往大走 110、170、180、195，掉头往回走 68、45、35、12，完整顺序就是 A，选 A。

错误选项为什么错＋举例：

- 返程顺序写成从小到大的选项：返程是往回走，该从大到小。教训：掉头后顺序反过来。
- 把小于 105 的请求排在前面的选项：出发方向是向大，小的要等掉头后。教训：先看出发方向。
- 其他顺序的选项：只要有一处顺序不对，就不是 SCAN 的走法。教训：先往大、再往回，两段分开排。

笔记本一句话： 第 29 题＋SCAN：从 105 往大走 110、170、180、195，掉头往回走 68、45、35、12。

【图像】



磁头从 105 道向序号增加方向移动，先服务 110、170、180、195 再折返服务 68、45、35、12 的 SCAN 电梯调度路径示意图

【定位】 操作系统 · 第5章 I/O 管理 · 磁盘调度算法（SCAN）

【易混对比】

- SCAN 和 SSTF 的区别：SSTF 只挑最近的、不管方向（本题会是 110 → 68 → 45 → 35 → 12 → 170 → 180 → 195）；SCAN 先认方向，一个方向走到底再折返。
- SCAN 和 CSCAN 的区别：CSCAN 是单向循环——走到一端直接跳回另一端、返程不服务；SCAN 是电梯式——返程也要顺路服务。
- "向序号增加方向"和"向序号减少方向"的区别：方向决定先服务哪一侧；本题是增加方向，所以先服务大于 105 的请求。

105 → 先上行服务 110、170、180、195 → 折返服务 68、45、35、12，选 A。

【坑位】

返程的请求也要服务，且顺序是从大到小，别写成从小到大。

最容易错的是返程顺序：折返后磁头是从 195 往回走，先遇到 68、再 45、35、12，顺序是 68、45、35、12（从大到小），不是 12、35、45、68。误选 C 的就是返程写反了；误选 B 的是把 SSTF（只看最近）当成了 SCAN；误选 D 的是完全没看方向、直接从小到大排。

【套路】

- 第一步，定方向：磁头在 105、向序号增加方向 → 先服务大于 105 的请求。
- 第二步，上行服务：105 → 110 → 170 → 180 → 195（从小到大）。
- 第三步，折返服务：从 195 往回 → 68 → 45 → 35 → 12（从大到小）。

认准方向走到底，顺路全接；折返再接另一侧，顺序反过来。

【类比】

就像电梯上行：你在 105 层进了上行电梯，电梯先把楼上按了上行键的人（110、170、180、195 层）一个个接上，到顶层后再下行，接楼下等着的人（68、45、35、12 层）——下行时先到 68 层、再到 45 层，顺序自然是从大到小。术语贴回去：楼层是"磁道号"，上行是"向序号增加方向"，到顶折返是"SCAN 返程服务"。

【口诀】 电梯认准一个方向，顺路全接再折返；返程顺序反过来，A 项序列记心上。

【考频】 「磁盘调度」在 2009–2022 年解析中出现 4 次 · 中频

第 30 题 · 为实现按名存取而为文件设置的数据结构及其信息类别（正确答案：A）

题干与选项：

文件系统中，文件访问控制信息存储的合理位置是_____。

- A. 文件控制块
- B. 文件分配表
- C. 用户口令表
- D. 系统注册表

入手点： 先想“按名存取”是什么意思：用户只给文件名，系统就得找到文件；系统得有个地方记着每个文件的信息。

专有名词从零解释：

- 先想“按名存取”：用户说文件名，系统找到对应的文件，不用用户记文件存在磁盘哪个角落。
- 再想“文件控制块（FCB）”：文件系统给每个文件设的、用来描述和控制文件的数据结构。
- 再想 FCB 里的三类信息：基本信息（文件名、位置、大小一类），存取控制信息（谁能读、谁能写一类），使用信息（什么时候建的、什么时候改过一类）。

考查知识点：

- 先想为什么需要 FCB：文件名和磁盘上的实际位置之间要有一座桥；FCB 就是这座桥，系统查 FCB 就知道文件在哪、多大、谁能动。
- 再想 FCB 里的三类信息：基本信息、存取控制信息、使用信息，三类齐了，文件才能被描述清楚、管起来。
- 拿生活例子验算：图书馆给每本书一张卡片，卡片上有书名、放哪个架子（基本信息），谁能借（存取控制信息），什么时候入库、借过几次（使用信息）；报书名就能找到书。
- 边界情况：FCB 是文件系统内部用的数据结构，用户平时看不到；按名存取靠的是 FCB，不是靠用户记磁盘地址。
- 易混对子：不要把 FCB 和文件本身混了——FCB 是描述文件的数据结构，不是文件的数据；不要把三类信息记漏——基本、存取控制、使用，三类都要有。

正确选项为什么对： 为按名存取设的数据结构是文件控制块，里面有基本信息、存取控制信息、使用信息三类，就是 A，选 A。

错误选项为什么错＋举例：

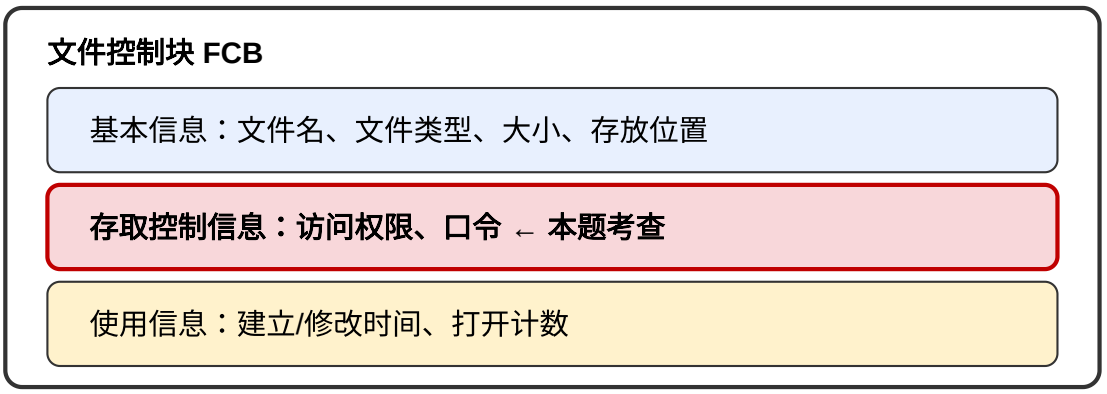
- 说其他数据结构名称的选项：按名存取靠的是文件控制块，不是别的数据结构。教训：先对名字，再对内容。
- 信息类别说不全的选项：三类信息缺一类，文件就描述不全、管不起来。教训：基本、存取控制、使用，三类一起记。

笔记本一句话： 第 30 题＋按名存取：靠文件控制块，里面有基本信息、存取控制信息、使用信息三类。

【图像】

FCB：描述和控制文件的三类信息

为实现"按名存取"，每个文件设置一个文件控制块（FCB）



文件分配表管"盘块分给谁"（空间分配），不管文件属性 → 干扰项

用户口令表、系统注册表都不是文件系统的数据结构

结论：访问控制信息的合理位置 = 文件控制块 → 选 A

文件控制块 FCB 包含基本信息、存取控制信息、使用信息三类信息的结构示意图

【定位】 操作系统 · 第4章 文件管理 · 文件控制块（FCB）

【易混对比】

- FCB 和文件分配表的区别：分配表管的是"盘块分给了谁"（空间分配），FCB 管的是"这个文件是谁、有什么属性"（描述和控制）；访问控制信息是文件的属性，住在 FCB 里。
- FCB 和索引结点的区别：索引结点是 FCB 瘦身后的核心（只留最常用的描述信息、常驻内存），FCB 是完整版；问"文件的数据结构"答 FCB 不会错。
- 三类信息的区别：基本信息答"文件是谁"（文件名、类型、大小、位置），存取控制信息答"谁能动它"（权限、口令），使用信息答"它被用得怎么样"（建立/修改时间、打开计数）。

按名存取靠 FCB，访问控制信息是 FCB 三类信息之一，选 A。

【坑位】

别被"文件分配表"带偏：分配表管盘块、不管文件属性。

看到"访问控制"容易联想到"控制"二字在分配表里也有管控意味，但文件分配表（FAT 一类）记录的是每个盘块空闲还是分配给了哪个文件，根本不存"谁可以读写这个文件"。访问控制信息（权限位、口令）是文件的属性信息，和文件名、大小住在一起——都在 FCB 里。

【套路】

- 第一步，看题干动作："按名存取" → 找描述和控制文件的数据结构 → FCB。
- 第二步，对三类信息：基本信息、存取控制信息、使用信息 → "访问控制信息"正好是第二类。
- 第三步，排除：分配表管空间、口令表和注册表都不是文件系统的数据结构 → 选 A。

按名存取找 FCB，三类信息对号入座。

【类比】

像一份人事档案袋（FCB）：基本信息是姓名、工号、部门（文件名、类型、大小）；存取控制信息是封面上的"密级：谁可以调阅"（访问权限）；使用信息是里面的借阅登记卡（创建时间、修改时间、被借过几次）。你要查"这份档案谁能看"，翻档案袋封面就行，不用去查公司车位分配表（文件分配表）。术语贴回去：档案袋是"文件控制块"，密级是"存取控制信息"，借阅卡是"使用信息"。

【口诀】 按名存取找 FCB，基本存取使用三类记；访问控制住第二类，答案选 A。

【考频】 「FCB」在 2009–2022 年解析中出现 60 次 · 高频

第 31 题 · 符号链接与硬链接建立、删除时的引用计数值变化（正确答案：B）

题干与选项：

设文件 F1 的当前引用计数值为 1，先建立 F1 的符号链接（软链接）文件 F2，再建立 F1 的硬链接文件 F3，然后删除 F1。此时，F2 和 F3 的引用计数值分别是_____。

- A. 0、1
- B. 1、1
- C. 1、2
- D. 2、1

入手点： 先把两种链接的区别想清楚：符号链接是复制计数值，硬链接是计数加 1；删除时，硬链接的计数减 1，符号链接的计数不动。

专有名词从零解释：

- 先想“引用计数”：有多少个名字指着同一个文件，计数就是几；计数到 0，文件才能真正删掉。
- 再想“符号链接”：新建一个文件，里面存的是原文件的路径，像快捷方式；建立时引用计数值直接复制。
- 再想“硬链接”：给同一个文件再起一个名字，两个名字指着同一个文件；建立时引用计数加 1。
- 再想“删除”：删掉一个名字，硬链接的计数减 1；符号链接的删除对原文件不可见，不影响计数。

考查知识点：

- 先想建立 F2：F2 是符号链接，建立时引用计数值直接复制；这时候 F1 和 F2 的引用计数值都为 1。
- 再想建立 F3：F3 是硬链接，建立时引用计数加 1；这时候 F1 和 F3 的引用计数值都变成了 2。
- 再想删除 F1：硬链接的计数减 1，F3 的引用计数值变为 2−1＝1；符号链接的删除对原文件不可见，F2 的引用计数值一直不变，仍为 1。
- 再想删除后的结果：F2 是 1，F3 是 1，两个都是 1，就是 B。
- 拿生活例子验算：F1 是原件；F2 是写着原件地址的纸条（符号链接），纸条上的数字照抄原件的；F3 是原件的另一个名字（硬链接），名字多一个，计数加 1；撕掉 F1 这个名字，F3 的计数减 1，纸条上的数字不动。
- 边界情况：计数不为 0，文件就不能真正删掉；符号链接删了，原文件还在；硬链接删到计数为 0，文件才真正删掉。
- 易混对子：不要把符号链接当成硬链接——符号链接是复制计数值，不是加 1；不要以为删除 F1 会让 F2 的计数也变——符号链接的删除对原文件不可见，F2 的计数不动。

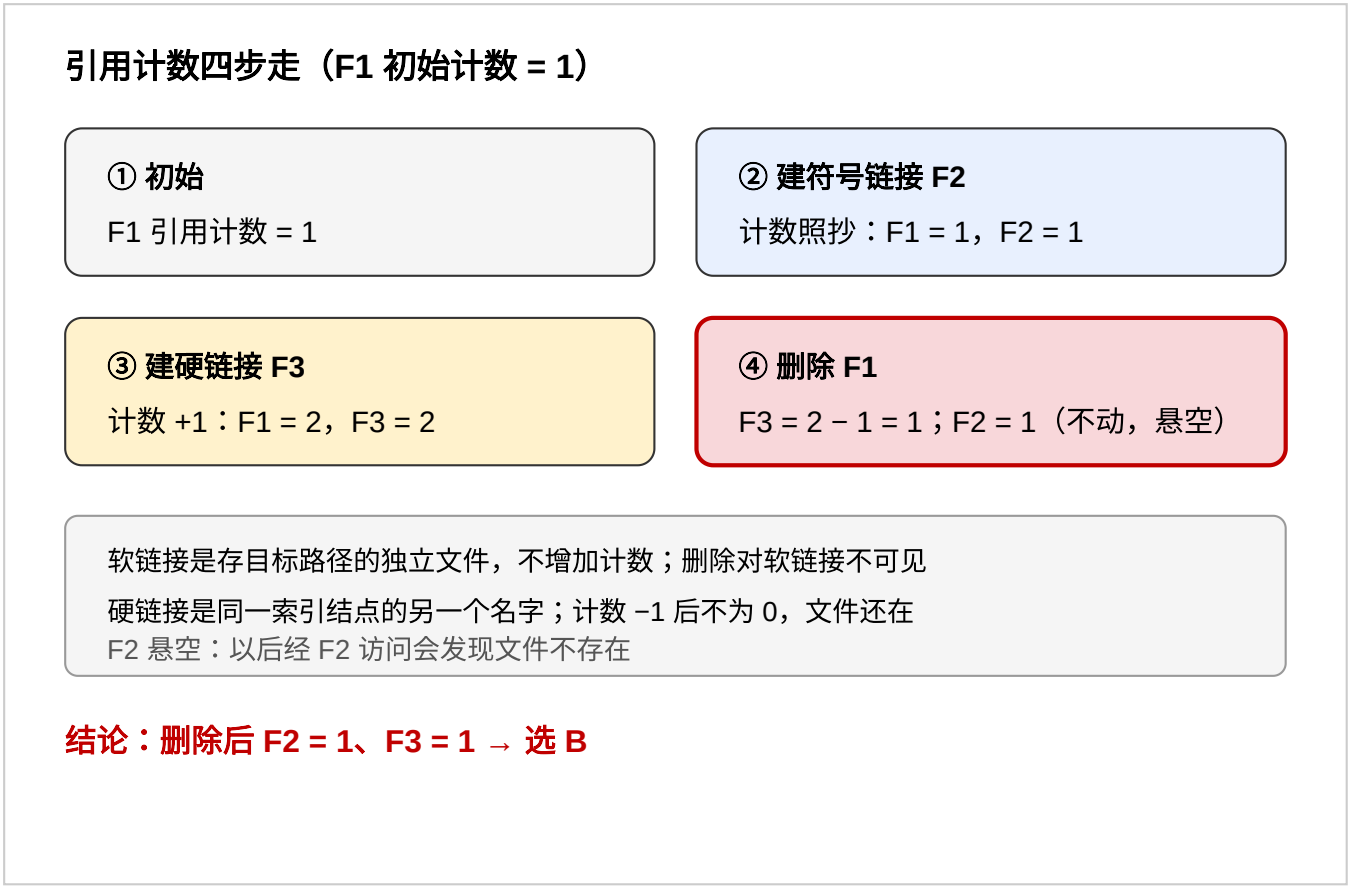
正确选项为什么对： 删除 F1 后，F3 的计数 2−1＝1，F2 的计数一直是 1，两个都是 1，就是 B，选 B。

错误选项为什么错＋举例：

- 说 F2 计数也变的选项：符号链接的删除对原文件不可见，F2 的计数不动。教训：符号链接和硬链接的删除效果不一样。
- 说 F3 计数还是 2 的选项：删掉 F1 这个硬链接名字，计数要减 1，变成 1。教训：硬链接删一个名字，计数减 1。
- 其他计数值组合的选项：只要 F2、F3 有一处对不上，就不是答案。教训：建立和删除分开算。

笔记本一句话： 第 31 题＋链接计数：符号链接复制计数，硬链接加 1；删 F1 后 F3 减到 1，F2 一直是 1。

【图像】



建立符号链接 F2（计数照抄）、硬链接 F3（计数加 1）、删除 F1（硬减 1、软不动）四阶段引用计数值变化示意图

【定位】 操作系统 · 第4章 文件管理 · 文件的链接（符号链接与硬链接）

【易混对比】

- 符号链接和硬链接的区别：软链接是新建一个存"目标路径"的独立文件，原文件的引用计数照抄不变；硬链接是给同一个索引结点再起个名字，原文件引用计数 +1。
- 删除源文件时两者的区别：删 F1 对软链接不可见——F2 的计数不动（仍为 1），但 F2 悬空（再访问发现文件不存在）；硬链接的计数 −1，只要不为 0 文件就还在。
- "引用计数"和"链接数"计的是谁：计的是指向同一索引结点的硬链接数，软链接不算进去。

软链接照抄（F2=1）、硬链接加 1（F1=F3=2）、删源硬减 1（F3=1）软不动（F2=1）→ 选 B。

【坑位】

软链接不让计数加 1，删源文件后软链接的计数也不变。

两个经典误区：一是以为"建立 F2"后 F1 的计数变成 2——那是硬链接的规矩，软链接只是复制计数值（F1 仍为 1，F2 也为 1）；二是以为"删除 F1"后 F2 的计数变 0——删除操作对软链接不可见，F2 的计数一直是它自己的 1，不受影响，只是从此悬空。F3 的计数则是 2 − 1 = 1，不是 0（还有 F3 指着，文件不删）。

【套路】

- 第一步，建软链接 F2：计数值直接复制 → F1 = 1，F2 = 1。
- 第二步，建硬链接 F3：引用计数 +1 → F1 = 2，F3 = 2。
- 第三步，删除 F1：硬链接 −1 → F3 = 2 − 1 = 1；软链接不可见 → F2 仍为 1。
- 第四步，组合：F2 = 1、F3 = 1 → 选 B。

软照抄、硬加一、删源硬减一软不动。

【类比】

像通讯录：硬链接是给同一个人再存一个名字（"张三"又存一个"三哥"，两个名字拨的都是同一个号码，联系人条目 +1）；删掉"张三"这个名字，"三哥"还能打通（计数 −1 但不为 0）。软链接是存一张纸条"找张三请拨 138xxxx"（独立纸条，计数照抄）；张三换号（删 F1）了，纸条还在、上面的号码打不通（悬空），但纸条本身的数量不受影响。术语贴回去：号码是"索引结点"，多存的名字是"硬链接"，纸条是"符号链接"，纸条/名字的个数是"引用计数"。

【口诀】 软链接照抄数，硬链接加一数；删源硬减一软不动，一一组合选 B。

【考频】 「链接」在 2009–2022 年解析中出现 12 次 · 高频

第 32 题 · 设备独立性与逻辑设备名向物理设备名的转换（正确答案：A）

题干与选项：

程序员利用系统调用打开 I/O 设备时，通常使用的设备标识是 _____。

- A. 逻辑设备名
- B. 物理设备名
- C. 主设备号
- D. 从设备号

入手点： 先想设备独立性是什么意思：用户程序不用管具体是哪台设备，只说要哪类设备；真正执行时，系统再把逻辑名字换成物理名字。

专有名词从零解释：

- 先想“设备独立性”：用户程序申请设备时，不用指定具体是哪一台，只要说要哪种设备。
- 再想“逻辑设备名”：用户程序里用的、统一的设备名字，不指具体哪台机器。
- 再想“物理设备名”：真正的那台设备的名字，指具体的机器。

考查知识点：

- 先想用户程序怎么写：操作系统以统一调用方式来请求某种设备时，使用的是逻辑设备名；程序里写“打印机”，不写“二楼那台打印机”。
- 再想实际执行时：程序真正跑起来，系统把逻辑设备名转换为对应的物理设备名，再去找那台具体的设备。
- 拿生活例子验算：点外卖只说“要一份宫保鸡丁”（逻辑名），平台再派给具体哪家店做（物理名）；用户不用管是哪家店。
- 边界情况：逻辑设备名是统一的，物理设备名是具体的；转换发生在程序实际执行时，不是写程序时。
- 易混对子：不要把逻辑设备名和物理设备名弄反——程序里写的是逻辑名，执行时换成物理名；不要把设备独立性当成“不用设备”——设备还是要用的，只是用户不用指定具体哪台。

正确选项为什么对： 程序实际执行时，逻辑设备名转换为对应的物理设备名，就是 A，选 A。

错误选项为什么错＋举例：

- 说“一直用逻辑设备名”的选项：逻辑名找不到具体的机器，执行时得换成物理名。教训：逻辑名是给程序看的，物理名是给机器找的。
- 说“程序里就写物理设备名”的选项：那样程序就和具体设备绑死了，换台设备就得改程序，设备独立性就没了。教训：设备独立性就是为了解耦。
- 其他名称的选项：只要没说到“逻辑设备名转物理设备名”，就没说到点子上。教训：转换的方向是逻辑到物理。

笔记本一句话： 第 32 题＋设备独立性：程序里写逻辑设备名，执行时系统换成物理设备名。

【图像】



应用程序用逻辑设备名经逻辑设备表映射到物理设备的设备独立性示意图

【定位】 操作系统 · 第5章 I/O 管理 · 设备独立性

【易混对比】

- 逻辑设备名和物理设备名的区别：逻辑名是程序员在程序里写的（与具体设备无关），物理名是实际执行时 OS 映射到的真实设备；换设备只改映射表，程序不用重写。
- 逻辑设备名和主/从设备号的区别：设备号是 OS 内部标识设备的编号（驱动层面用），程序员写程序、做系统调用时用的是逻辑设备名。
- 设备独立性和"设备无关性"表述的区别：同一个意思的不同说法，题干出现"与具体物理设备无关""程序可移植"都是在点它。

程序员系统调用打开设备用逻辑设备名，执行时再映射到物理设备，选 A。

【坑位】

题干强调"程序员利用系统调用"——站在程序员视角，只能看到逻辑设备名。

主设备号、从设备号、物理设备名都是 OS 内部或驱动层面的东西，程序员写 open() 时不会、也不该直接指定"第 3 台打印机"。设备独立性的全部意义就是让程序员摆脱具体设备：程序里写逻辑名，换一台物理设备时 OS 改映射表就行，程序原样能跑。

【套路】

- 第一步，看题干视角："程序员""系统调用" → 用户层面 → 逻辑设备名。
- 第二步，看考查点："打开 I/O 设备时用的设备标识" + 设备独立性 → 逻辑名，执行时映射物理名。
- 第三步，排除：物理设备名、主/从设备号都是 OS 内部用的 → 选 A。

用户层面只认逻辑名，映射是 OS 的事。

【类比】

像点外卖填"公司前台"（逻辑设备名）：今天前台在 A 栋、明天搬到 B 栋（物理设备变了），你下单只管写"公司前台"，骑手（操作系统）负责查映射表找到真正的楼栋（物理设备名）。公司搬家你不用重学点外卖——这就是设备独立性。术语贴回去：外卖地址栏是"系统调用参数"，骑手手里的楼栋对照表是"逻辑设备表"，真正的楼栋是"物理设备"。

【口诀】 程序只认逻辑名，执行映射物理名；换设备不改程序，答案选 A。

【考频】 「设备独立性」在 2009–2022 年解析中出现 3 次 · 中频

第 33 题 · 传输层、数据链路层与网络层各自的通信范围（正确答案：B）

题干与选项：

在 OSI 参考模型中，自下而上第一个提供端到端服务的层次是_____。

- A. 数据链路层
- B. 传输层
- C. 会话层
- D. 应用层

入手点： 先把三层的通信范围各想一遍：传输层是端到端，数据链路层是相邻结点，网络层是主机到主机；再看哪个选项把传输层说对了。

专有名词从零解释：

- 先想“端到端”：从一台端系统上的应用进程，到另一台端系统上的应用进程，像两个人直接通话。
- 再想“相邻结点”：一条链路上直接连着的两个设备，像隔壁邻居。
- 再想“主机到主机”：从一台主机到另一台主机，中间可以经过很多路由器。
- 再想“端系统”：真正跑应用的主机；交换机和路由器是中间设备，不算端系统。

考查知识点：

- 先想传输层：提供应用进程间的逻辑通信（通过端口号），即端到端的通信；两端的进程像直接通话，中间的细节不用管。
- 再想数据链路层：负责相邻结点之间的通信；这个结点包括了交换机和路由器等数据通信设备，这些设备不能称为端系统。
- 再想网络层：负责主机到主机的逻辑通信；从源主机到目的主机，中间经过的路由器不改变“主机到主机”这个范围。
- 拿生活例子验算：传输层像两个人打电话（端到端），数据链路层像把信交到隔壁邻居手里（相邻结点），网络层像把信从一座城市寄到另一座城市（主机到主机）。
- 边界情况：交换机和路由器是中间设备，不是端系统；说“端到端”时，两端一定是端系统上的进程。
- 易混对子：不要把“相邻结点”安到传输层头上——相邻结点是数据链路层的范围；不要把“主机到主机”安到传输层头上——主机到主机是网络层的范围；不要把交换机、路由器当成端系统。

正确选项为什么对： 传输层是端到端的通信，就是 B，选 B。

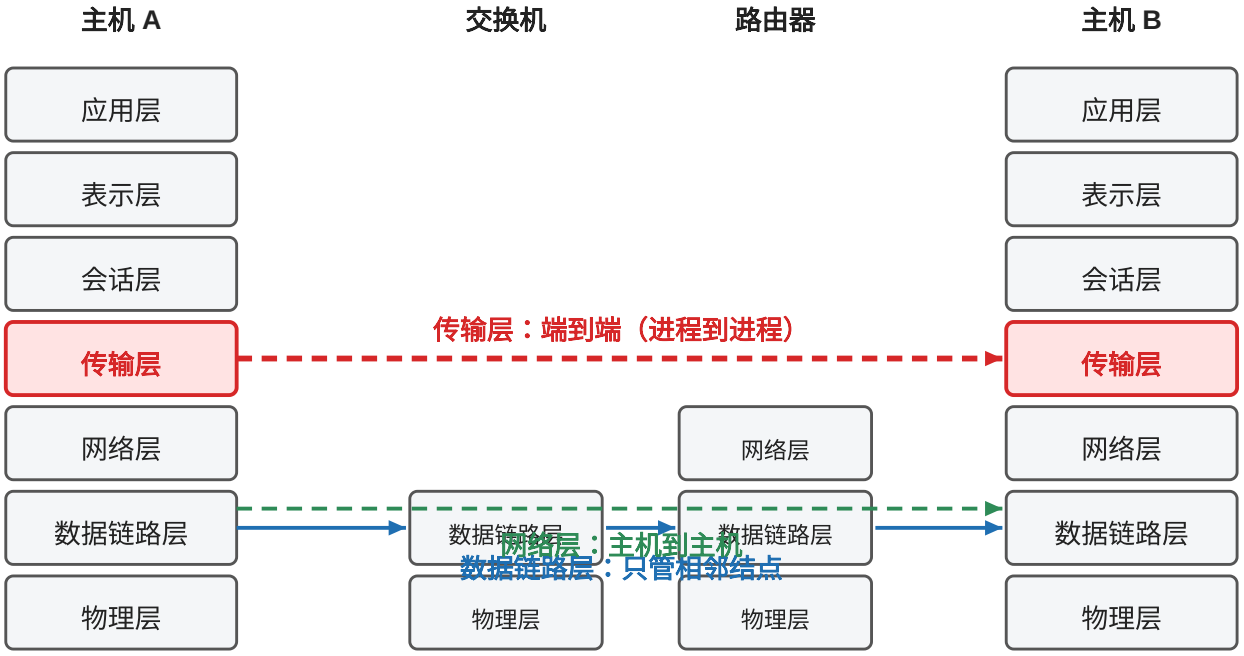
错误选项为什么错＋举例：

- 说传输层是“相邻结点间通信”的选项：相邻结点是数据链路层的范围。教训：三层的范围分开记。
- 说传输层是“主机到主机通信”的选项：主机到主机是网络层的范围。教训：传输层看的是进程，不是主机。
- 其他层选项逐字表述错的选项：只要范围和层次对不上，就不是答案。教训：先定层次，再定范围。

笔记本一句话： 第 33 题＋三层通信范围：传输层端到端，数据链路层相邻结点，网络层主机到主机。

【图像】

Q33 · 各层通信范围：传输层端到端



七层模型通信范围：传输层端到端直达对端进程，数据链路层只管相邻结点

【定位】 计算机网络 · 第 1 章计算机网络体系结构 · 各层通信范围：端到端 vs 相邻结点

【易混对比】

- 数据链路层和传输层是两回事：数据链路层只管相邻两个结点之间，中间每跳都要重新封装转发；传输层只看两端的端系统，中间有多少交换机、路由器都不管。
- 网络层和传输层也是两回事：网络层是主机到主机（找哪台机器），传输层是端到端进程到进程（找这台机器上的哪个程序，靠端口号区分）。
- "自下而上第一个"和"提供端到端服务的最高层"是两回事：会话层、表示层、应用层也有端到端语义，但自下往上数，传输层是第一个。

传输层是自下而上第一个端到端层，端口号就是端到端的标志。

【坑位】

别把交换机、路由器当成端系统：数据链路层的"相邻结点"包括它们，但端到端只认两端的主机。

最容易踩的坑，是看到"结点之间通信"就选数据链路层。数据链路层的结点是相邻结点，帧每到一个交换机、路由器都要被拆开重传；而"端到端"指的是发送方和接收方两个端系统直接的逻辑通信，中间设备只是透明转发。题干问的是第一个提供端到端服务的层次，答案是传输层。

【类比】

像寄快递：数据链路层是每个中转站之间的短途运输，包裹每到一站都要卸货再装车；传输层是发件人和收件人直接对接——不管经手多少中转站，真正"通话"的是两端的人。

【口诀】 端到端找传输，相邻结点数据链。

【考频】 「端到端」在 2009–2022 年解析中出现 16 次 · 高频

第 34 题 · 用奈奎斯特公式求 QAM 调制的最大信息传输速率（正确答案：B）

题干与选项：

在无噪声情况下，若某通信链路的带宽为 3kHz，采用 4 个相位，每个相位具有 4 种振幅的 QAM 调制技术，则该通信链路的最大数据传输速率是_____。

- A. 12kbps
- B. 24 kbps
- C. 48 kbps
- D. 96 kbps

入手点： 先算一个信号能带几位：相位数乘以幅度数就是状态数，状态数取对数就是位数；再用奈奎斯特公式乘带宽。

专有名词从零解释：

- 先想“QAM”：把相位和幅度结合起来的调制方法，一个信号可以表示好几种状态。
- 再想“状态数”：一个信号能表示的不同样子有几种；4 个相位、每个相位 4 种幅度，一共 4×4＝16 种状态。
- 再想“奈奎斯特公式”：最大信息传输速率等于 2 乘以带宽、再乘以每个信号带的位数。

考查知识点：

- 先想每个信号带几位：16 种状态，一个信号能表示 16 种样子，需要 4 位二进制（2^4＝16），所以每个信号传输 4bit 的数据。
- 再想带宽：信道带宽是 3kHz，就是 3000Hz。
- 再想最大速率：根据奈奎斯特公式，2×3k×4＝24kbps，就是 B。
- 拿生活例子验算：一个信号像一个有 16 个格子的转盘，转一次能表示 4 位信息；一秒转 6000 次（2 倍带宽），一秒就是 24000 位。
- 边界情况：奈奎斯特公式里的 2 是固定的（2 倍带宽）；位数是状态数取以 2 为底的对数，不是状态数本身。
- 易混对子：不要把 16 种状态当成 16 位——16 种状态只要 4 位；不要漏了公式里的 2——最大速率是 2 倍带宽乘以位数。

正确选项为什么对： 16 种状态带 4 位，2×3k×4＝24kbps，就是 B，选 B。

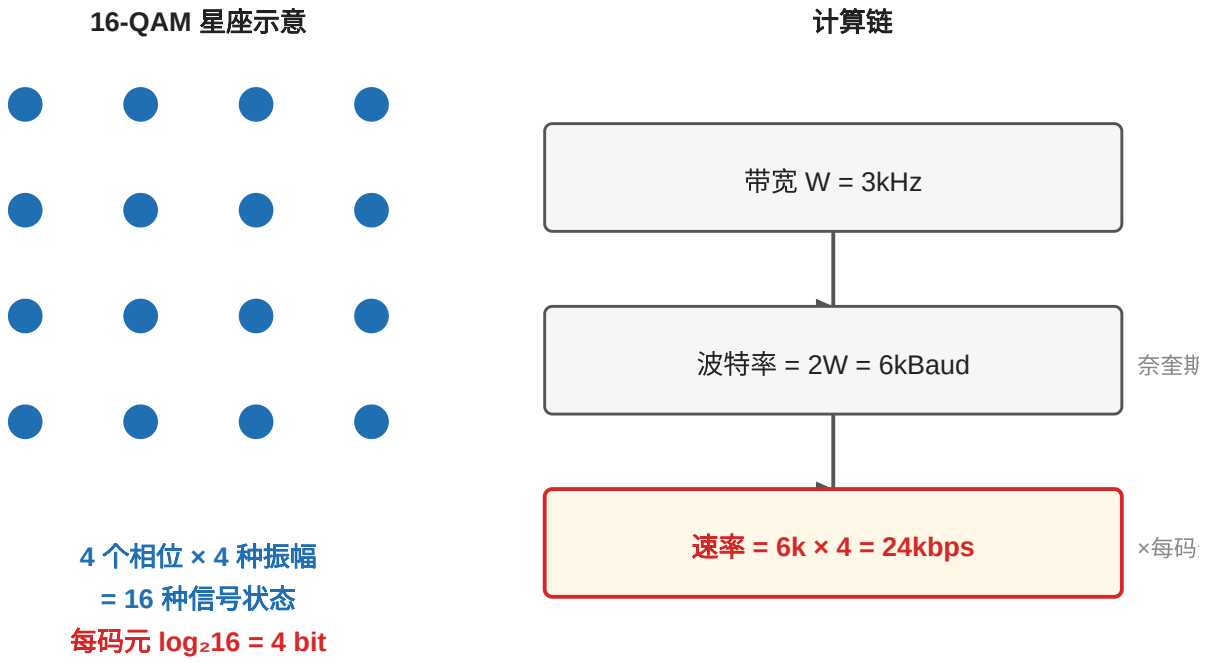
错误选项为什么错＋举例：

- 说 12kbps 的选项：那是漏了公式里的 2，或者位数算错了。教训：公式是 2 倍带宽乘位数。
- 说 48kbps 的选项：那是把 16 种状态当成 8 位，或者带宽用错了。教训：位数是状态数取对数，16 种状态是 4 位。
- 其他速率的选项：只要状态数、位数、带宽有一处算错，结果就错。教训：先算状态数，再算位数，最后乘公式。

笔记本一句话： 第 34 题＋奈奎斯特：4 相位×4 幅度＝16 种状态带 4 位，2×3k×4＝24kbps。

【图像】

Q34 · 奈奎斯特公式算 QAM 速率



16-QAM 星座图与奈奎斯特计算链：4×4=16 种状态，每码元 4bit，2×3k×4=24kbps

【定位】 计算机网络 · 第 2 章物理层 · 奈奎斯特公式与 QAM 每码元比特数

【易混对比】

- "4 个相位、每个相位 4 种振幅"和"8 种信号状态"是两回事：相位和振幅是两个独立维度，要相乘，4×4=16 种，不是 4+4=8。
- 奈奎斯特和香农是两回事：题干说"无噪声情况下"就是点名用奈奎斯特公式（$2W \cdot \log_2 V$）；有噪声才用香农公式。
- 波特率和比特率是两回事：2×3k=6k 是波特率（码元/秒），再乘每码元 4bit 才是 24kbps 的信息速率。

无噪声+带宽→奈奎斯特；信号状态数=相位种数×振幅种数。

【坑位】

"4 相位、4 振幅"直接当成 8 种状态或 3bit：信号状态数是相乘不是相加。

最容易踩的坑，是看到"4 个相位、4 种振幅"就写成 8 种状态、每码元 3bit，得出 2×3k×3=18kbps。相位和振幅组合出的是 16 种不同的波形，每种波形携带 $\log_2 16=4$bit。记住：几个维度就乘几次，再取 $\log_2$。

【套路】

- 第一步，算信号状态数 V=4×4=16。
- 第二步，每码元比特数=$\log_2 V=\log_2 16=4$。
- 第三步，套奈奎斯特公式：2×3k×4=24kbps。

V 先乘后取 $\log_2$，最后再乘 2W，顺序别乱。

【类比】

像用 4 种颜色的墨水写在 4 种颜色的纸上：组合出来的是 4×4=16 种花样，不是 8 种；每写一个符号，就传递了 4bit 的信息。

【口诀】 相位乘振幅，$\log_2$ 取码元，无噪奈奎斯特，二倍带宽乘。

【考频】 「QAM」在 2009–2022 年解析中出现 4 次 · 中频

第 35 题 · 后退 N 帧协议中累积确认丢失时需要重传的帧号（正确答案：C）

题干与选项：

数据链路层采用后退 N 帧（GBN）协议，发送方已经发送了编号为 0~7 的帧。当计时器超时时，若发送方只收到 0、2、3 号帧的确认，则发送方需要重发的帧数是_____。

- A. 2
- B. 3
- C. 4
- D. 5

入手点： 先想累积确认是什么意思：收到几号确认，就表示几号及以前的帧都收到了；再看发送方实际收到了几号确认。

专有名词从零解释：

- 先想“后退 N 帧”：发送方一次发好几帧，接收方发现某一帧出错，就把这帧和它后面的帧全丢掉，发送方过后把这帧和后面的全重传。
- 再想“累积确认”：接收方对按序到达的最后一个分组发送确认；收到 3 号确认，就表示 0、1、2、3 号帧都收到了。
- 再想“确认帧丢失”：确认帧在返回的路上丢了，发送方没收到，不等于数据帧没到接收方。

考查知识点：

- 先想接收方的情况：接收方对按序到达的最后一个分组发送确认；本题中收到 3 的确认帧，表示编号为 0、1、2、3 的帧已接收。
- 再想发送方没收到 1 号帧的确认：这只能代表确认帧在返回的过程中丢失了，而不代表 1 号帧未到达接收方。
- 再想发送方要重传什么：接收方已经确认到 3 号，发送方过后需重传 3 号之后的帧，也就是 4、5、6、7，就是 C。
- 拿生活例子验算：寄了 8 封信，对方回信说“前 4 封都收到了”（累积确认）；中间有一封回信在路上丢了，不等于那封信没到；对方已经确认到第 4 封，接下来只要重寄第 5、6、7、8 封。
- 边界情况：累积确认是“到几号为止都收到了”，不是“只收到了几号”；确认帧丢失和数据帧丢失是两回事。
- 易混对子：不要以为“没收到 1 号确认”就是“1 号帧没到”——确认帧丢了，数据帧可能早就到了；不要从 1 号开始重传——接收方已经确认到 3 号，重传从 4 号开始。

正确选项为什么对： 累积确认到 3 号，0、1、2、3 都已接收，重传 4、5、6、7，就是 C，选 C。

错误选项为什么错＋举例：

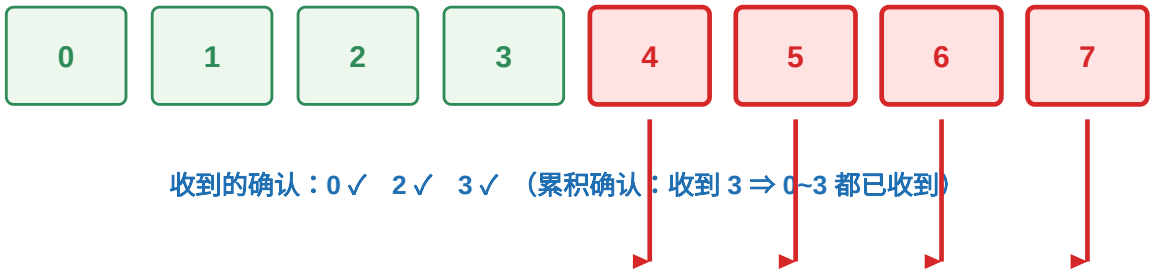
- 从 1 号开始重传的选项：把确认帧丢失当成数据帧丢失，1 号其实已经到了。教训：确认丢了不等于数据丢了。
- 从 0 号开始重传的选项：0 到 3 号都已经确认收到，重传是浪费。教训：累积确认到几号，就从下一号开始重传。
- 其他帧号组合的选项：只要起始帧号不对，组合就错。教训：先看确认到几号，再定重传起点。

笔记本一句话： 第 35 题＋后退 N 帧累积确认：确认到 3 号就是 0 到 3 都到了，确认帧丢了不等于数据丢了，重传 4、5、6、7。

【图像】

Q35 · 后退 N 帧：累积确认到 3，重传 4~7

发送方已发送 0~7 号帧，计时器超时



重传帧：4、5、6、7 —— 共 4 帧

注意：没收到 1 的确认 ≠ 1 丢失（只是确认帧在回程丢了）

GBN 重传的是“第一个未确认帧及之后全部”，不是“没收到确认的帧”

GBN 超时重传：已发 0~7，累积确认到 3，重传 4、5、6、7 共 4 帧

【定位】 计算机网络 · 第 3 章数据链路层 · 后退 N 帧协议与累积确认

【易混对比】

- 后退 N 帧和选择重传是两回事：GBN 超时重传出错帧及其之后全部已发帧；选择重传（SR）只重传出错的那一帧。
- "没收到 1 号帧的确认"和"1 号帧没送到"是两回事：累积确认下，收到 3 的确认就表示 0、1、2、3 都已正确接收；1 的确认只是回程路上丢了。
- "收到 0、2、3 的确认"不是按序点名：累积确认只认最大的那个序号（3），0 和 2 的确认信息被 3 完全覆盖。

GBN 重传的是"第一个未确认帧及之后全部"，不是"没收到确认的那几帧"。

【坑位】

别把没收到 1 号确认当成 1 号帧丢失：累积确认里，3 的确认已经把 1 捎带确认了。

最容易踩的坑，是数"没收到确认的帧"：0、2、3 收到了，1、4、5、6、7 没收到，于是选 5（D）。但累积确认的语义是"3 之前（含 3）的都收到了"，所以真正需要重传的只有 4、5、6、7，共 4 帧，选 C。

【套路】

- 第一步，找出已收确认中的最大序号：3。
- 第二步，累积确认的含义：0~3 都已正确接收。
- 第三步，GBN 重传 4 及之后全部已发帧：4、5、6、7，共 4 帧。

先找最大确认号，再往后数全部重传。

【类比】

像快递签收：快递员说"3 号包裹之前的都收到了"，1 号包裹的签收条在回程路上丢了，不等于 1 号包裹没送到；超时之后，就得把 4 号及之后的包裹重新再寄一遍。

【口诀】 累积确认看最大，后退重传全跟上。

【考频】 「后退N帧」在 2009–2022 年解析中出现 2 次 · 低频

第 36 题 · 交换机工作层次与转发时使用的 PDU 地址（正确答案：A）

题干与选项：

以太网交换机进行转发决策时使用的 PDU 地址是_____。

- A. 目的物理地址
- B. 目的 IP 地址
- C. 源物理地址
- D. 源 IP 地址

入手点： 先想交换机是什么：多端口网桥；网桥工作在哪一层，交换机就工作在哪一层；那一层用的地址，就是转发时用的地址。

专有名词从零解释：

- 先想“交换机”：实质上是一个多端口网桥，能把好几个设备连起来，数据从哪个口进、往哪个口出，由它决定。
- 再想“数据链路层”：负责相邻结点之间的数据传送，用物理地址来区分设备。
- 再想“PDU 地址”：这一层的数据单元里用来寻址的地址；数据链路层用的是物理地址，转发时看的是目的地址。

考查知识点：

- 先想交换机的工作层次：交换机实质上是一个多端口网桥，工作在数据链路层。
- 再想转发用的地址：数据链路层使用物理地址进行转发，而转发到目的地通常是使用目的地址；所以 PDU 地址是目的物理地址，就是 A。
- 拿生活例子验算：小区门卫（交换机）按门牌号（物理地址）分信，信封上写的目的门牌号（目的物理地址）决定信往哪栋楼送。
- 边界情况：交换机看的是目的地址，不是源地址；源地址是用来学路径的，不是用来转发这一帧的。
- 易混对子：不要把交换机说成网络层设备——网络层设备是路由器；不要把目的物理地址说成源物理地址——转发看的是目的地。

正确选项为什么对： 交换机是多端口网桥，工作在数据链路层，转发用目的物理地址，就是 A，选 A。

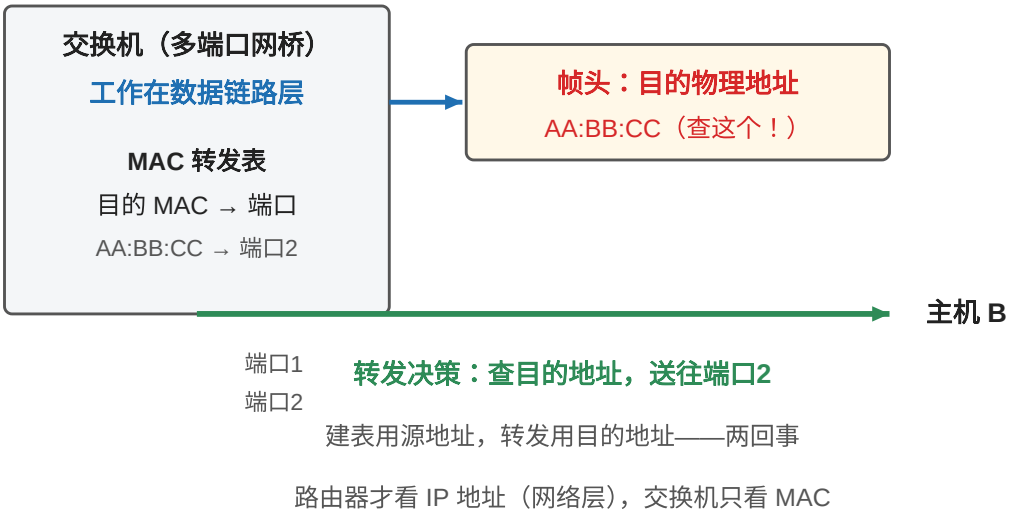
错误选项为什么错＋举例：

- 说网络层的选项：网络层设备是路由器，不是交换机。教训：交换机是网桥，不是路由器。
- 说源物理地址的选项：源地址是用来学习路径的，转发看的是目的地址。教训：转发问“往哪送”，看目的地址。
- 其他地址类型的选项：只要层次或地址类型有一处不对，就不是答案。教训：先定层次，再定地址。

笔记本一句话： 第 36 题＋交换机：多端口网桥，数据链路层，转发看目的物理地址。

【图像】

Q36 · 交换机：数据链路层设备，查目的物理地址转发



交换机转发：查 MAC 表，用帧头中的目的物理地址决定输出端口

【定位】 计算机网络 · 第 3 章数据链路层 · 交换机工作层次与转发依据

【易混对比】

- 交换机和路由器是两回事：交换机是多端口网桥，工作在数据链路层，看物理地址转发；路由器工作在网络层，看 IP 地址转发。
- 目的地址和源地址是两回事：转发决策查的是目的地址；源地址只用来学习、建立转发表。
- 物理地址和 IP 地址是两回事：数据链路层 PDU（帧）的转发依据是物理地址（MAC 地址），IP 地址是网络层的事。

交换机=多端口网桥=数据链路层设备，转发查目的物理地址。

【坑位】

别把"学源建表"和"转发查目的"搞混：建表用源地址，转发用目的地址，题干问的是转发决策。

最容易踩的坑，是记得"交换机要学习源地址"，就选了源物理地址（C）。学习建表和转发决策是两个动作：收到帧时用源地址更新转发表，决定从哪个端口发出去时查的是目的地址。题干问的是"转发决策时使用的 PDU 地址"，所以是目的物理地址。

【类比】

像小区门卫按门牌号放行：只看你要去几号楼（目的物理地址）决定放哪个门；顺手把你住几号楼记下来（源地址学习），下次有人找你就知道往哪指。

【口诀】 交换机看二层，目的物理地址转发。

【考频】 「交换机」在 2009–2022 年解析中出现 25 次 · 高频

第 37 题 · 最短帧长缩短时为保持冲突窗口而需缩短的站点间距离（正确答案：D）

题干与选项：

在一个采用 CSMA/CD 协议的网络中，传输介质是一根完整的电缆，传输速率为 1Gbps，电缆中的信号传播速度是 200 000km/s。若最小数据帧长度减少 800 比特，则最远的两个站点之间的距离至少需要_____。

- A. 增加 160m
- B. 增加 80m
- C. 减少 160m
- D. 减少 80m

入手点： 先想冲突窗口是什么：收发端点间的往返时延；帧长变短，发送时延变短，往返时延也要跟着变短，距离就得缩短。

专有名词从零解释：

- 先想“冲突窗口”：CSMA/CD 里，发送方能发现冲突的时间窗口，等于网络中收发端点间的往返时延，也就是 2 倍传播时延。
- 再想“发送时延”：把一帧数据推到链路上要的时间，等于帧长除以数据传输速率。
- 再想“传播时延”：信号从一端跑到另一端要的时间，等于距离除以传播速度。

考查知识点：

- 先想为什么距离要缩短：最短帧长减少，发送时延跟着减少；要保持冲突窗口，往返时延也要减少同样的量，往返时延是 2 倍传播时延，传播时延要减半个量，距离就得缩短。
- 再想公式：减少的往返时延等于减少的发送时延，即 2×(s÷(2×10^8))＝800÷(1×10^9)；其中 s 是需要减少的最小距离，800 是帧长减少的位数，1×10^9 是数据传输速率，2×10^8 是传播速度。
- 再想解出来：s＝80，即最远的两个站点之间的距离最少需要减少 80m，就是 D。
- 再想补充：CSMA/CD 的碰撞窗口是 2 倍传播时延，报文发送时间远大于碰撞窗口。
- 拿生活例子验算：喊话的人一句话变短了，要保证对方在自己喊完之前能回话，两人之间的距离就得缩短；一句话短了多少秒，往返跑的时间就得短多少秒。
- 边界情况：减少的是“最远的两个站点之间的距离”，不是随便两个站点；单位换算要对齐，位和字节、米和秒别混了。
- 易混对子：不要把往返时延当成单程传播时延——冲突窗口是往返的，有个 2 倍；不要把帧长减少的位数直接当成距离——要先换算成发送时延，再换算成距离。

正确选项为什么对： 减少的往返时延等于减少的发送时延，算出来距离最少减少 80m，就是 D，选 D。

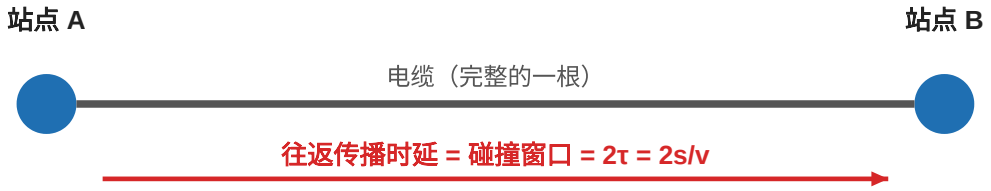
错误选项为什么错＋举例：

- 说 40m 的选项：那是把往返时延当成单程，漏了 2 倍。教训：冲突窗口是往返时延。
- 说 160m 的选项：那是把 2 倍多乘了一次，或者单位换算错了。教训：公式一步一步代，别凭感觉翻倍。
- 其他距离的选项：只要公式里的量有一处代错，结果就错。教训：先列公式，再代数字。

笔记本一句话： 第 37 题＋冲突窗口：减少的往返时延等于减少的发送时延，距离最少减少 80m。

【图像】

Q37 · 碰撞窗口 = 2τ：帧长减少 800bit ⇒ 距离减少 80m



最短帧发送时延 ≥ 碰撞窗口

帧长减少 800bit ⇒ 发送时延减少 $800 \div 10^9 = 0.8\mu s$

$$2 \times (s \div 2\times10^8) = 0.8\times10^{-6} \quad \Rightarrow \quad s = 80m$$

帧变短 ⇒ 碰撞窗口变小 ⇒ 电缆必须缩短（减少 80m）

坑：往返是 2 倍单程，别漏乘 2（否则错算 160m）

CSMA/CD 碰撞窗口：帧长减少 800bit，则最远站点距离减少 80m

【定位】 计算机网络 · 第 3 章数据链路层 · CSMA/CD 碰撞窗口与最短帧长

【易混对比】

- 碰撞窗口和帧发送时延是两回事：碰撞窗口是往返传播时延 2τ；CSMA/CD 要求最短帧的发送时延不小于 2τ，这样冲突才能在发完之前被检测到。
- "帧长减少"和"速率变化"是两回事：速率 1Gbps 不变，帧变短→发送时延变小→允许的往返时延变小→电缆必须变短。
- 增加和减少是两回事：帧变短，碰撞窗口跟着变小，只有缩短电缆才能保证冲突被发现，所以是减少 80m，不是增加。

最短帧发送时延=2τ；帧长和距离同向变化，帧短则距离短。

【坑位】

往返时延是 2 倍单程：距离变化 s 带来的往返变化是 2s/v，别漏乘这个 2。

最容易踩的坑有两个：一是把碰撞窗口当成单程传播时延，算出 160m（A）；二是帧长减少却选了"增加距离"（A/B）。记住：帧长减少→发送时延减少→往返时延必须同步减少→距离缩短。方程是 $2\times(s\div(2\times10^8))=800\div(1\times10^9)$，解得 s=80m。

【套路】

- 第一步，帧长减少对应的发送时延=$800\div(1\times10^9)=0.8\mu s$。
- 第二步，令往返时延减少量相等：$2\times s\div(2\times10^8)=0.8\times10^{-6}$。
- 第三步，解得 s=80m，帧变短了距离必须缩短，选 D。

减少的发送时延=距离减少带来的往返时延减少量。

【类比】

像两个人隔着山谷喊话确认：话变短了（帧短），就得站近点（距离短），才能在对方开口前听到回声、发现撞话了；站太远，话没说完回声还没回来。

【口诀】 帧长短则距离短，往返两倍别漏算。

【考频】 「CSMA/CD」在 2009–2022 年解析中出现 5 次 · 中频

第 38 题 · TCP 接收方返回的确认序号含义与计算（正确答案：D）

题干与选项：

主机甲与主机乙之间已建立一个 TCP 连接，主机甲向主机乙发送了两个连续的 TCP 段，分别包含 300 字节和 500 字节的有效载荷，第一个段的序列号为 200，主机乙正确接收到两个段后，发送给主机甲的确认序列号是 _____。

- A. 500
- B. 700
- C. 800
- D. 1000

入手点： 先想确认序号是什么意思：接收方期待收到的下一个字节的序号；再把两个段的字节数加起来。

专有名词从零解释：

- 先想“确认序号”：接收方告诉发送方“我接下来想收第几个字节”，这个数字就是确认序号。
- 再想“报文段”：TCP 传送的一段数据，每个字节都有序号。
- 再想“期待收到”：已经收到的字节就不用再确认了，确认序号指向的是下一个还没收到的字节。

考查知识点：

- 先想确认序号的含义：返回的确认序号是接收端期待收到对方下一个报文段数据的第一个字节的序号。
- 再想本题的数字：乙在正确收到两个段后，返回给甲的确认序号是 200+300+500＝1000，就是 D。
- 拿生活例子验算：甲从第 200 页开始寄书，先寄了 300 页，再寄了 500 页；乙都收到了，接下来想收第 1000 页，就回信说“请从 1000 页开始寄”。
- 边界情况：确认序号是“下一个期待的字节”，不是“最后一个收到的字节”；两个段都收到了，序号要加两个段的长度。
- 易混对子：不要把确认序号写成最后一个收到的字节的序号——要加 1 指向下一个；不要只加一个段的长度——两个段都收到了，两个长度都要加。

正确选项为什么对： 两个段都收到了，期待的下一个字节序号是 200+300+500＝1000，就是 D，选 D。

错误选项为什么错＋举例：

- 说 500 或 700 的选项：那是只算了一个段，或者把起始序号弄错了。教训：两个段的长度都要加。
- 说 999 的选项：那是最后一个收到的字节的序号，确认序号要指向下一个。教训：确认序号是“下一个期待的字节”。
- 其他序号的选项：只要起始序号或段长度有一处加错，结果就错。教训：先定起始序号，再把两个段的长度都加上。

笔记本一句话： 第 38 题＋TCP 确认序号：期待收到的下一个字节的序号，200+300+500＝1000。

【图像】

Q38 · TCP 确认序号 = 期望的下一个字节序号



确认序号指向“下一个期望字节”，不是“最后一个已收字节”（999）

起始序号 + 已收总字节数，一步到位

TCP 确认序号：seq=200 发 300+500 字节，返回 ACK=1000（期望的下一个字节）

【定位】 计算机网络 · 第 5 章传输层 · TCP 确认序号的含义与计算

【易混对比】

- 确认序号和"已收到最后一个字节的序号"是两回事：确认序号是"我期望收到的下一个字节的序号"（1000），不是已收到的最后一个字节（999）。
- 累加的是字节数不是段数：两个段共 300+500=800 字节，ACK=200+800=1000。
- 第一个段的序列号是起始偏移：从 200 开始数，不是默认从 0 或 1 开始。

ACK=起始序号+已收总字节数，指向下一个期望字节。

【坑位】

别答 999：确认序号是"下一个"不是"最后一个"，永远多 1。

最容易踩的坑，是把确认序号理解成"最后一个收到的字节"，算出 200+300+500−1=999。TCP 的确认序号定义是期望收到的下一个字节序号，所以是 200+800=1000，选 D。记住口诀里的"指下家"。

【套路】

- 第一步，已收总字节数=300+500=800。
- 第二步，ACK=起始序号+总字节数=200+800=1000。

起始序号加总字节，一步到位。

【类比】

像收快递报数："200 号到 999 号我都收到了，下一件请从 1000 号开始发"——确认的是下一件的编号，不是上一件的。

【口诀】 确认序号指下家，起始序号加字节。

【考频】 「确认序号」在 2009–2022 年解析中出现 36 次 · 高频

第 39 题 · 发生超时后慢开始与拥塞避免切换时的拥塞窗口变化（正确答案：C）

题干与选项：

一个 TCP 连接总是以 1KB 的最大段长发送 TCP 段，发送方有足够多的数据要发送。当拥塞窗口为 16KB时发生了超时，如果接下来的 4 个 RTT（往返时间）时间内的 TCP 段的传输都是成功的，那么当第 4 个RTT 时间内发送的所有 TCP 段都得到肯定应答时，拥塞窗口大小是_____。

- A. 7 KB
- B. 8 KB
- C. 9 KB
- D. 16 KB

入手点： 先想超时后的两步：门限减半、窗口归 1；再想慢开始怎么涨：每个 RTT 翻倍，涨到门限就转拥塞避免、改成加法增大。

专有名词从零解释：

- 先想“拥塞窗口”：发送方一次能发多少数据还没收到确认，窗口越大，发得越多。
- 再想“慢开始门限”：慢开始和拥塞避免的分界线；窗口小于门限用慢开始，大于等于门限用拥塞避免。
- 再想“RTT”：从发数据到收到确认的一轮时间。
- 再想“慢开始”：窗口每个 RTT 翻倍地涨，像滚雪球。
- 再想“拥塞避免”：窗口每个 RTT 只加 1，像慢慢爬坡。

考查知识点：

- 先想超时后的变化：慢开始门限 ssthresh 变为 16KB÷2＝8KB，拥塞窗口变为 1KB。
- 再想接下来的 3 个 RTT：执行慢开始算法，拥塞窗口大小依次为 2KB、4KB、8KB。
- 再想切换：窗口涨到 8KB，等于慢开始门限 8KB，之后转而执行拥塞避免算法，拥塞窗口开始“加法增大”。
- 再想第 4 个 RTT 结束：拥塞避免加法增大，窗口从 8KB 加到 9KB，就是 C。
- 拿生活例子验算：堵车后重新起步，先翻倍加速（1、2、4、8），到了限速线（8），改成每次只加一点（9）。
- 边界情况：超时后门限是原来的一半，不是窗口的一半；窗口是归 1，不是减半；涨到门限就切换，不是超过门限才切换。
- 易混对子：不要把超时后的窗口记成减半——超时是归 1，收到 3 个重复确认才是减半；不要把切换点记成超过门限——涨到门限就切换。

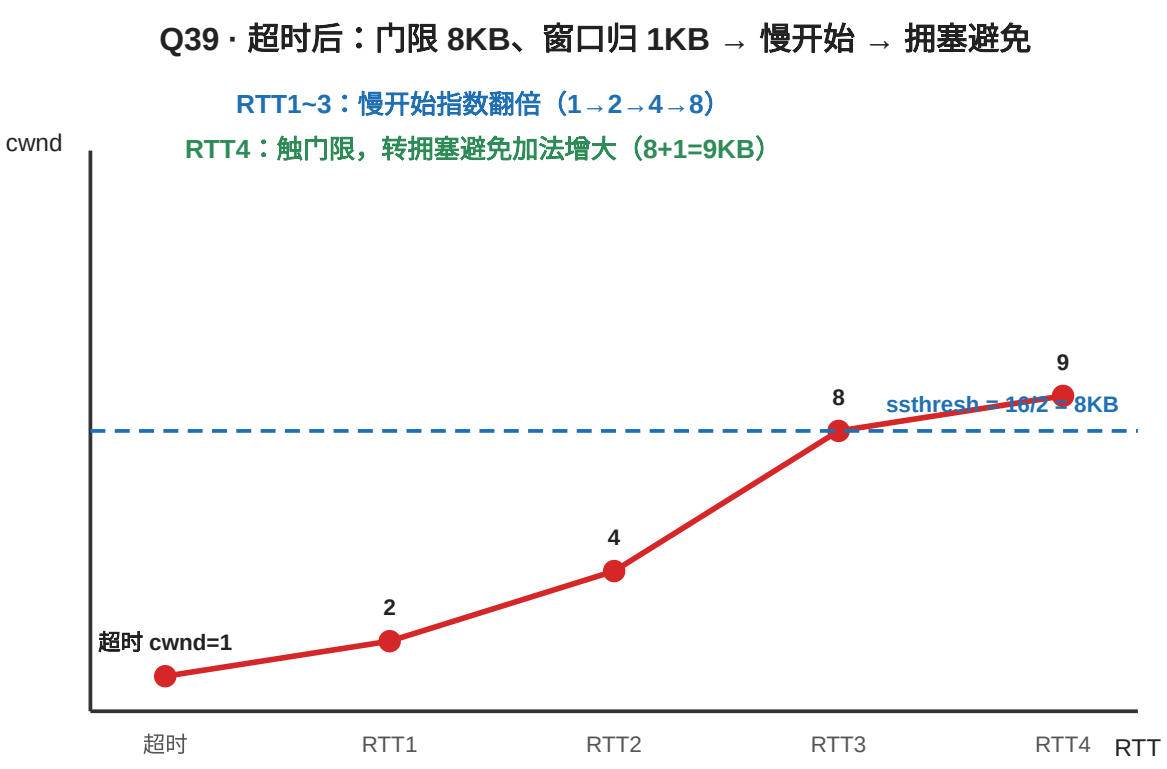
正确选项为什么对： 超时后门限 8KB、窗口 1KB，慢开始涨到 8KB 转拥塞避免，第 4 个 RTT 结束是 9KB，就是 C，选 C。

错误选项为什么错＋举例：

- 说 8KB 的选项：那是第 3 个 RTT 结束的窗口，没算第 4 个 RTT 的加法增大。教训：切换后还要再加 1。
- 说 16KB 的选项：那是把慢开始的翻倍算过头了，或者门限没减半。教训：超时后门限先减半，涨到门限就切换。
- 其他窗口值的选项：只要门限、窗口起点、切换点有一处错，结果就错。教训：超时归 1、门限减半、到门限切换，三步分开算。

笔记本一句话： 第 39 题＋超时后窗口：门限减半 8KB、窗口归 1KB，慢开始涨到 8KB 转拥塞避免，第 4 个 RTT 结束 9KB。

【图像】



拥塞窗口变化：超时后门限 8KB、窗口归 1KB，慢开始 2→4→8，第 4 个 RTT 拥塞避免到 9KB

【定位】 计算机网络 · 第 5 章传输层 · 慢开始与拥塞避免的切换

【易混对比】

- 超时和收到 3 个重复 ACK 是两回事：超时→门限减半、窗口归 1MSS，走慢开始；3 个重复 ACK→快重传+快恢复，窗口减半但不归 1。
- 慢开始和拥塞避免是两回事：窗口小于门限时指数翻倍（每 RTT 翻一番），达到门限后改加法增大（每 RTT 只加 1MSS）。
- "第 4 个 RTT 结束"和"第 3 个 RTT 结束"是两回事：前 3 个 RTT 走完慢开始（1→2→4→8），第 4 个 RTT 才进入拥塞避免加 1 得 9。

超时→门限减半、窗口归 1；到门限转拥塞避免加法增大。

【坑位】

数 RTT 别数错：第 1~3 个 RTT 是 2、4、8KB，第 4 个 RTT 才到 9KB。

最容易踩的坑，是把第 4 个 RTT 也算成翻倍得 16KB（D），或者在第 3 个 RTT 到 8KB 时提前加 1。规则是：窗口达到门限才切换算法。第 3 个 RTT 结束窗口正好 8KB=门限，第 4 个 RTT 走拥塞避免：8+1=9KB，选 C。

【套路】

- 第一步，超时：门限 ssthresh=16÷2=8KB，窗口 cwnd=1KB（MSS=1KB）。
- 第二步，慢开始指数增长：RTT1→2KB，RTT2→4KB，RTT3→8KB，触门限。
- 第三步，转拥塞避免加法增大：RTT4→8+1=9KB。

门限减半窗口归一，指数到顶改加法。

【类比】

像开车试探路况：出事故（超时）后先慢速起步、每段路程翻倍提速（慢开始），到了之前事故时一半的速度（门限）就改成每次只加一档（拥塞避免），稳着开。

【口诀】 超时门限折半归一，指数到顶加法续。

【考频】 「拥塞控制」在 2009–2022 年解析中出现 3 次 · 中频

第 40 题 · FTP 为保证可靠性所选协议及其控制信息的传送方式（正确答案：A）

题干与选项：

FTP 客户和服务器间传递 FTP 命令时，使用的连接是_____。

- A. 建立在 TCP 之上的控制连接
- B. 建立在 TCP 之上的数据连接
- C. 建立在 UDP 之上的控制连接
- D. 建立在 UDP 之上的数据连接

入手点： 先想可靠性要什么协议：要可靠传输的协议；再想控制信息走哪条连接：是和数据走同一条，还是单独一条。

专有名词从零解释：

- 先想“FTP”：文件传输协议，专门用来在两台机器之间传文件。
- 再想“TCP”：提供可靠传输的协议，数据丢了会重传，顺序也不会乱。
- 再想“带外传送”：控制信息不和数据走同一条连接，而是用一条单独的控制连接来传送命令。

考查知识点：

- 先想可靠性：FTP 文件传输要保证可靠性，选择 TCP 协议；不用不可靠传输的协议。
- 再想控制信息：FTP 的控制信息是带外传送的，也即 FTP 使用了一个分离的控制连接来传送命令；数据走数据连接，命令走控制连接，两条连接分开。
- 再想四个选项：A 是 TCP 协议加分离控制连接，就是答案；C、D 用了不提供可靠传输的协议，先排除。
- 拿生活例子验算：搬家公司运家具（数据）走货车，指挥调度（控制信息）走电话，两条线分开；货车要保证东西不丢不乱，就得选靠谱的运输方式。
- 边界情况：控制连接在整个会话期间一直开着，数据连接传完就关；命令和数据分开走，互不干扰。
- 易混对子：不要把控制信息当成和数据走同一条连接——FTP 的控制是带外的；不要把“可靠”随便配个协议——可靠传输要选 TCP。

正确选项为什么对： FTP 选 TCP 保证可靠，控制信息用分离的控制连接带外传送，就是 A，选 A。

错误选项为什么错＋举例：

- C、D：用了不提供可靠传输的协议，文件传丢了、传乱了都没人管，不符合 FTP 要可靠的要求。教训：先看可靠性要求，再挑协议。
- 说控制信息和数据走同一条连接的选项：FTP 的控制是带外传送，命令走单独的控制连接。教训：控制和数据分开走。
- 其他组合的选项：只要协议或控制方式有一处不对，就不是答案。教训：协议和控制方式分开验。

笔记本一句话： 第 40 题＋FTP：选 TCP 保证可靠，控制信息用分离的控制连接带外传送。

【图像】

Q40 · FTP 双连接：命令走控制连接（带外），文件走数据连接



控制信息带外传送：命令和文件走不同的连接

FTP 为保可靠选 TCP —— 排除一切 UDP 选项（C、D）

FTP 双连接：控制连接（TCP 21）传命令，数据连接（TCP 20）传文件，控制信息带外传送

【定位】 计算机网络 · 第 6 章应用层 · FTP 的双连接与带外控制

【易混对比】

- 控制连接和数据连接是两回事：命令走控制连接，文件走数据连接；题干问的是"传递 FTP 命令"，不是传文件。
- FTP 和 TFTP 是两回事：FTP 用 TCP 保证可靠，TFTP 才用 UDP。
- 带内和带外是两回事：FTP 的控制信息是带外传送（走独立的控制连接），不是混在数据流里一起传。

FTP 命令走 TCP 上的控制连接，文件另走数据连接。

【坑位】

看到"文件传输"就选数据连接：题问的是传命令，命令走控制连接。

最容易踩的坑，是看到 FTP 是文件传输协议就选了数据连接（B）。FTP 确实传文件，但题干明确问"传递 FTP 命令时使用的连接"。FTP 用一个分离的控制连接专门传命令（带外控制），数据连接只传文件内容，所以选 A。

【类比】

像餐厅点菜：服务员（控制连接）负责传菜单、记点单，传菜员（数据连接）负责上菜；要点菜只跟服务员说，不会拦传菜员下单。

【口诀】 FTP 双连接，命令走控制。

【考频】 「FTP」在 2009–2022 年解析中出现 4 次 · 中频

附录 · 答案速查表

- 1–8：B、C、D、B、C、B、A、D
- 9–16：A、B、C、D、D、C、D、C
- 17–24：A、A、D、B、D、A、D、D
- 25–32：C、A、C、B、A、A、B、A
- 33–40：B、B、C、A、D、D、C、A